# Recurrent Neural Networks and Transformer

Instructor: Hao Wang

AI Thrust, Information hub

Credit: part of the slides are modified from cs231n in standford

# Recap of last week

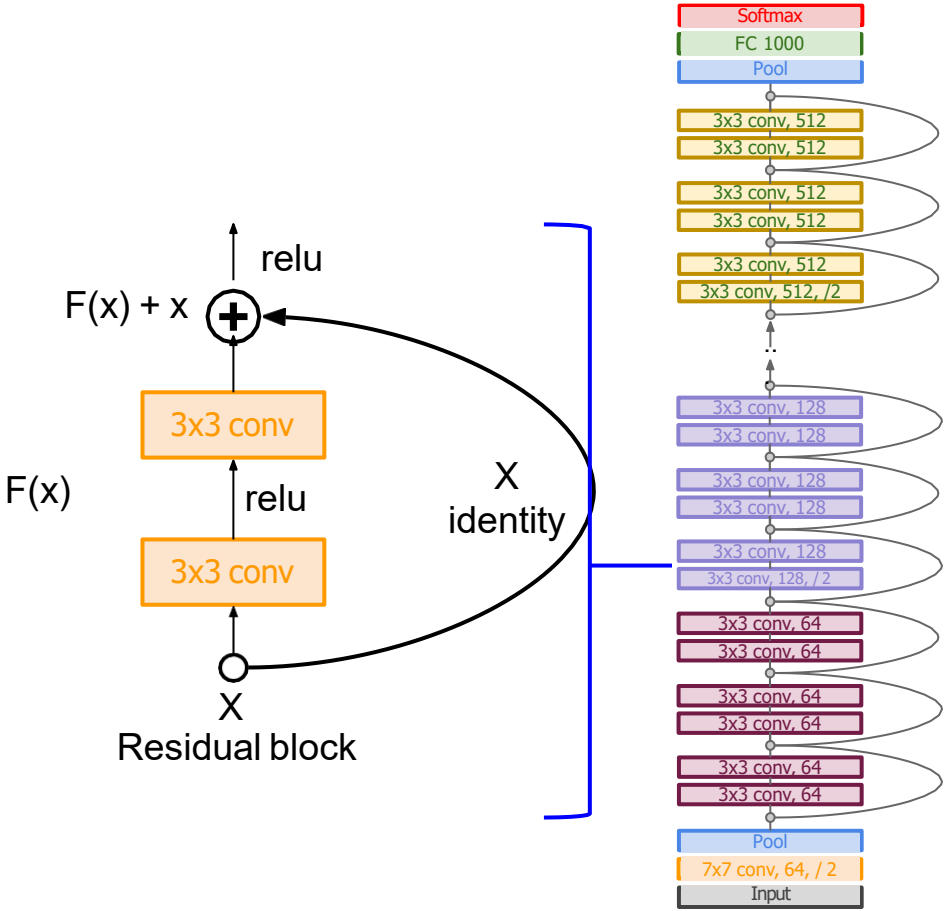


# ResNet

*[He et al., 2015]*

Full ResNet architecture:

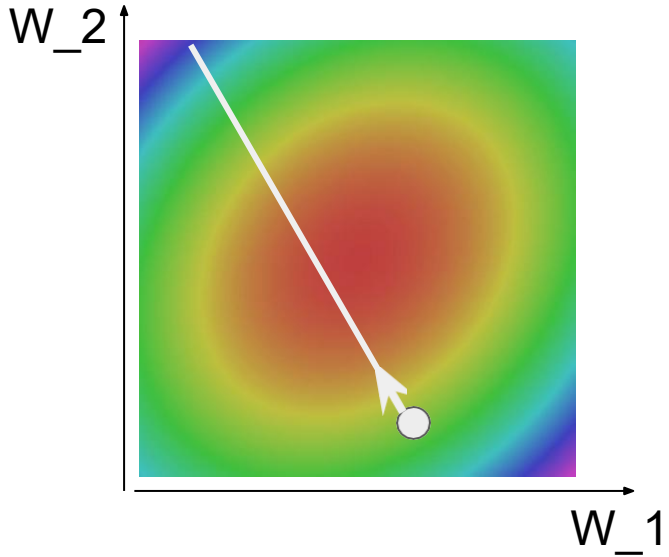
- Stack residual blocks
- Every residual block has two 3x3 conv layers



# Optimization introduced previously

```
# Vanilla Gradient Descent

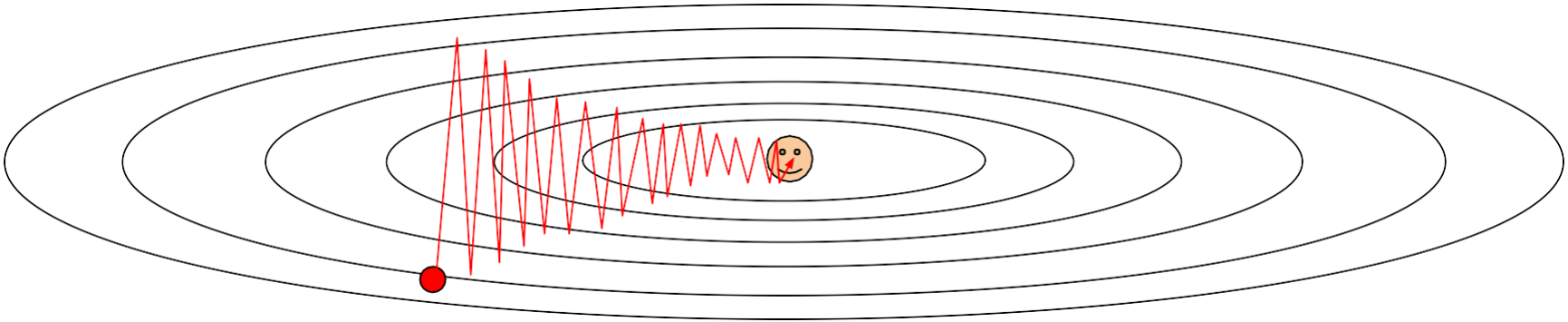
while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```






# Optimization: Problem #1 with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?
Very slow progress along shallow dimension, jitter along steep direction

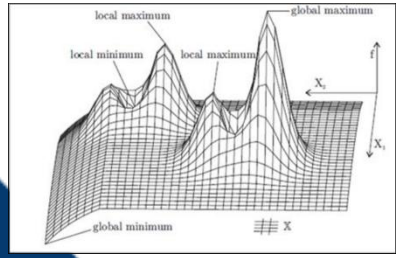


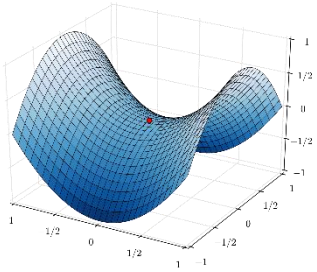
Loss function has high **condition number**: ratio of largest to smallest singular value of the Hessian matrix is large

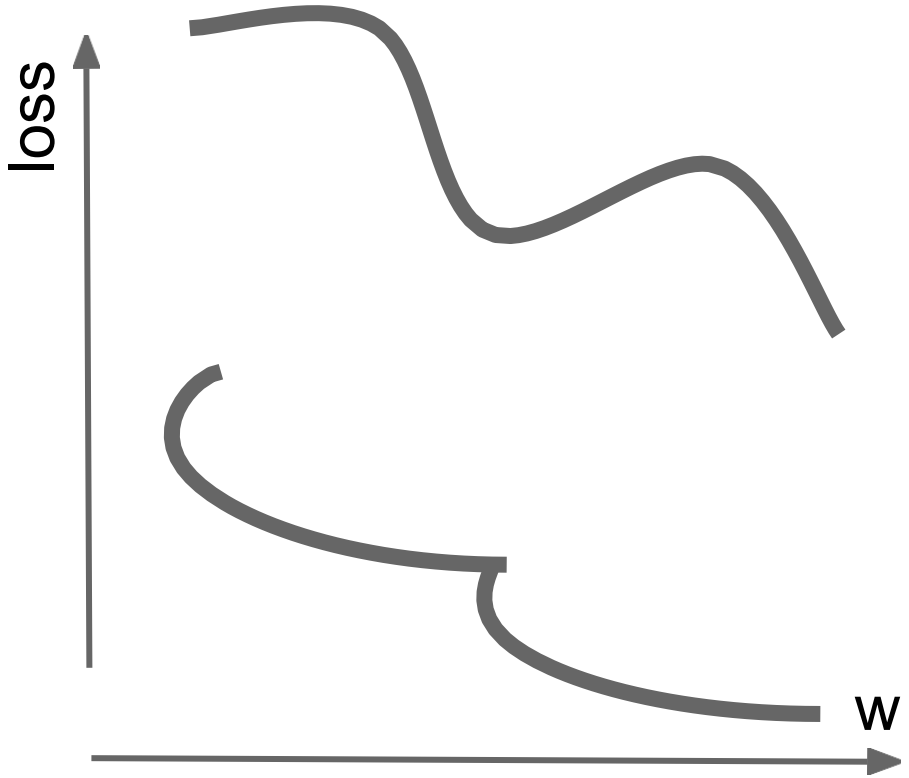


# Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?



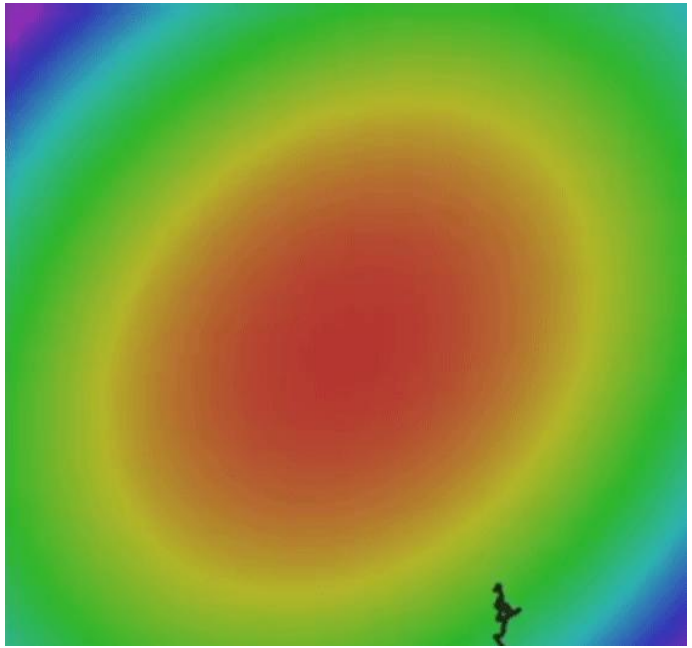



# Optimization: Problem #3 with SGD

Our gradients come from minibatches, so they can be noisy!

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W)$$





# SGD + Momentum

continue moving in the general direction as the previous iterations

## SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

## SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
  dx = compute_gradient(x)
  vx = rho * vx + dx
  x -= learning_rate * vx
```

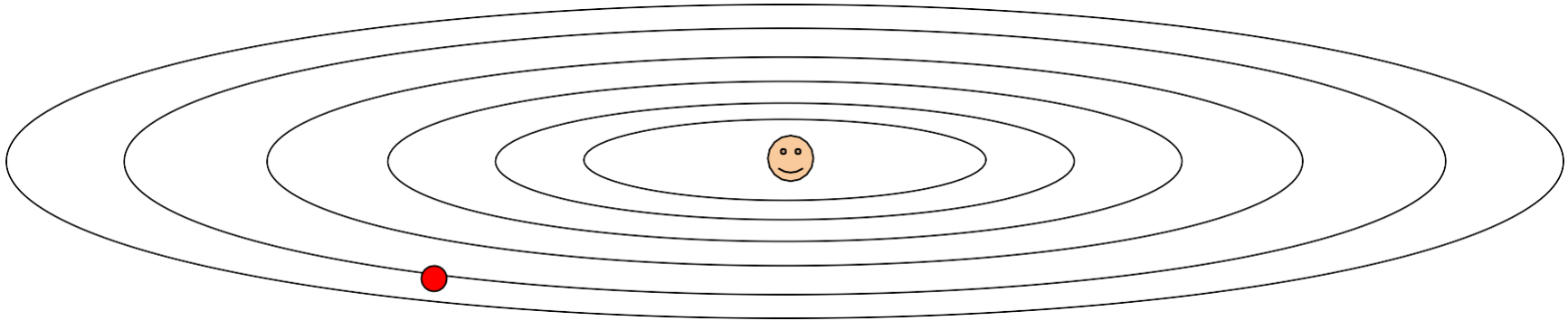
- Build up “velocity” as a running mean of gradients
- Rho gives “friction”; typically rho=0.9 or 0.99



Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

# AdaGrad

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Q: What happens with AdaGrad?

Progress along “steep” directions is damped;
progress along “flat” directions is accelerated

If dx is large, np.sqrt(grad_squared) will increased rapidly,
Denominator will help decrease the learning rate

# RMSProp

AdaGrad

Step size decays to zero over long time

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared += dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



RMSProp

```
grad_squared = 0
while True:
  dx = compute_gradient(x)
  grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
  x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```



Tieleman and Hinton, 2012

# Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7))
```



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

# Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

AdaGrad / RMSProp

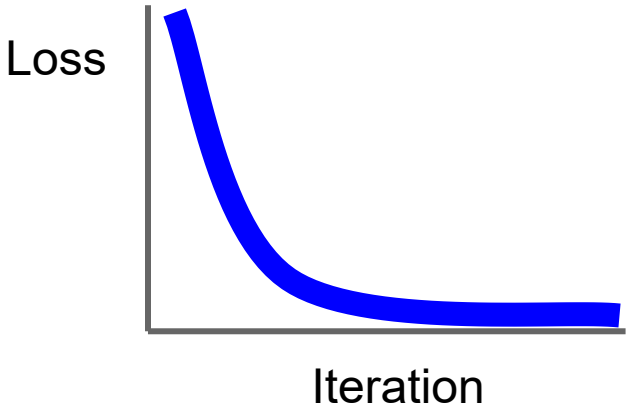
Bias correction for the fact that first and second moment estimates start at zero

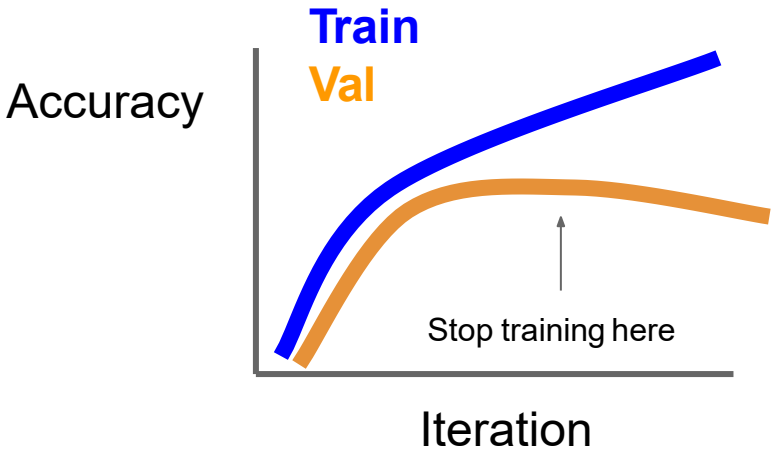
Adam with beta1 = 0.9,
beta2 = 0.999, and learning_rate = 1e-3 or 5e-4
is a great starting point for many models!



Kingma and Ba, “Adam: Amethod for stochastic optimization”, ICLR 2015

# Early Stopping



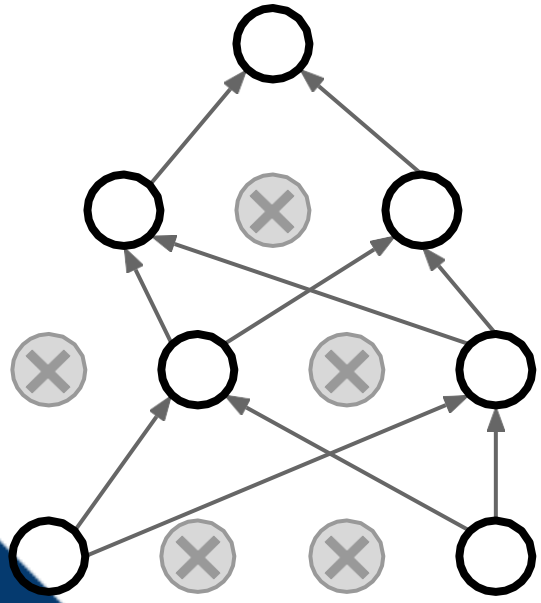

Stop training the model when accuracy on the validation set decreases

Or train for a long time, but always keep track of the model snapshot that worked best on val



# Regularization: Dropout

How can this possibly be a good idea?



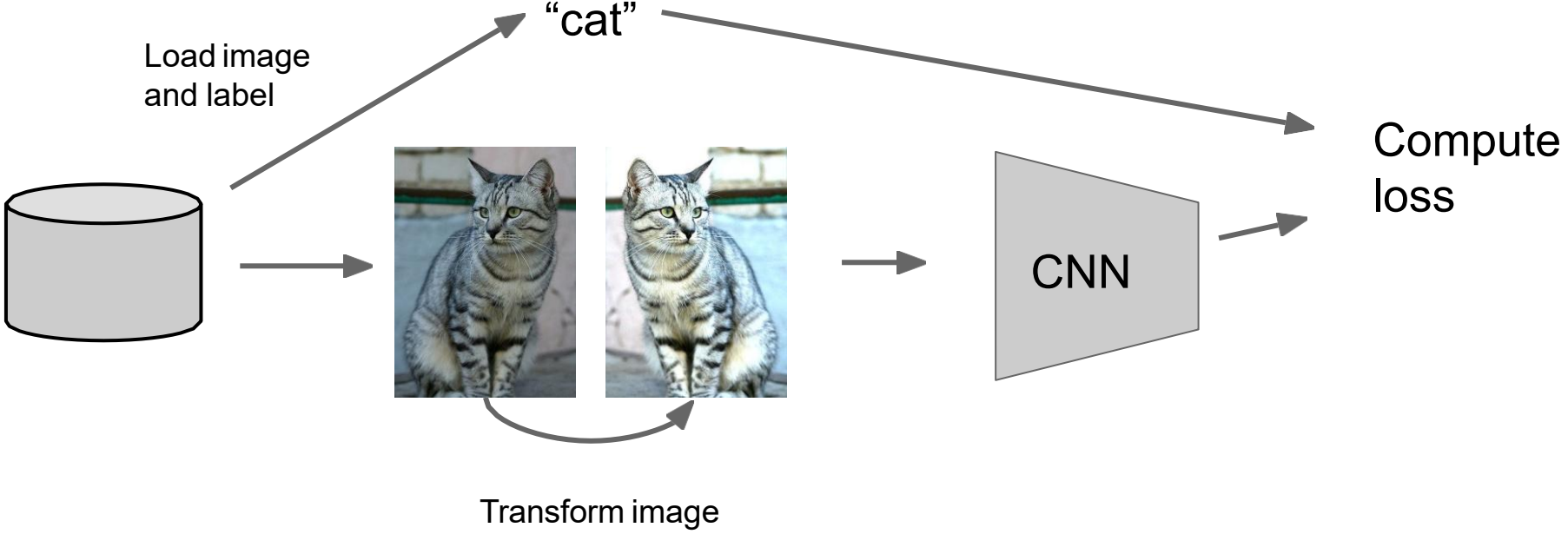
Another interpretation:

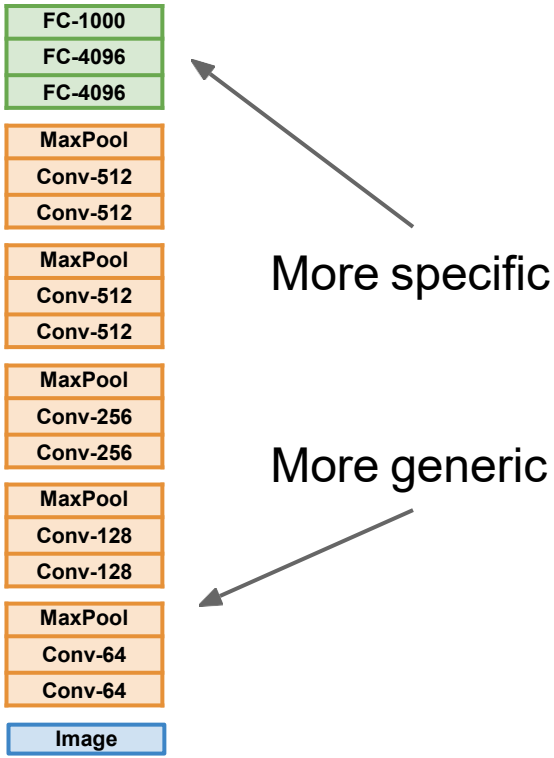
Dropout is training a large **ensemble** of models (that share parameters).

Each binary mask is one model

An FC layer with 4096 units has
$2^{4096} \sim 10^{1233}$ possible masks!
Only $\sim 10^{82}$ atoms in the universe...

# Regularization: Data Augmentation






| | **very similar dataset** | **very different dataset** |
|---|---|---|
| **very little data** | Use Linear Classifier on top layer | You're in trouble… Try linear classifier from different stages |
| **quite a lot of data** | Finetune a few layers | Finetune a larger number of layers |

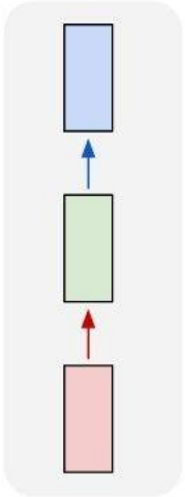
# Outline

- **Brief Introduction of RNN**
- **Attention with RNNs**
- **General Attention Layer**
  - Self-attention
  - Positional encoding
  - Masked attention
  - Multi-head attention
- **Transformers**



# “Vanilla” Neural Network

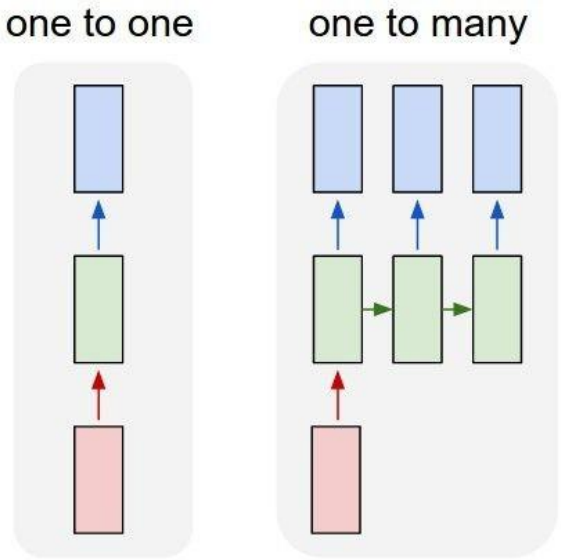
one to one



**Vanilla Neural Networks**

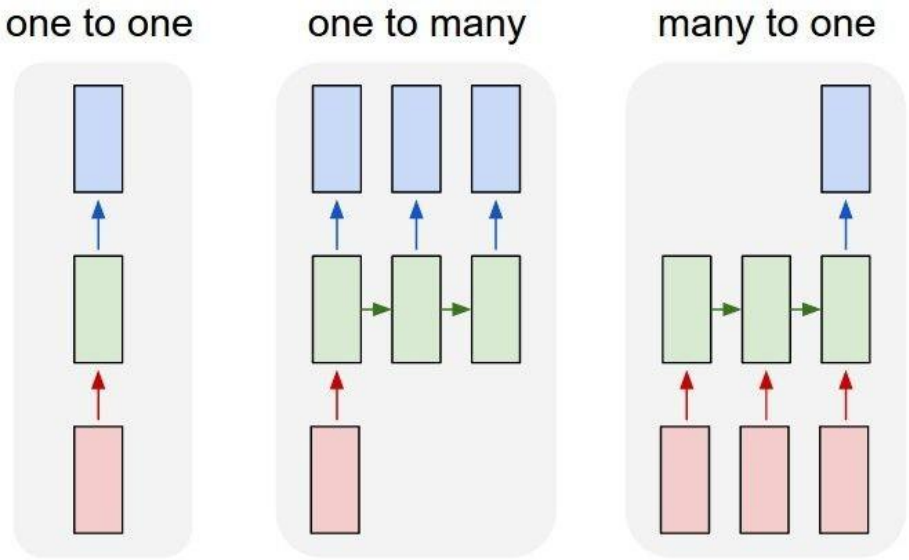


# Recurrent Neural Networks: Process Sequences



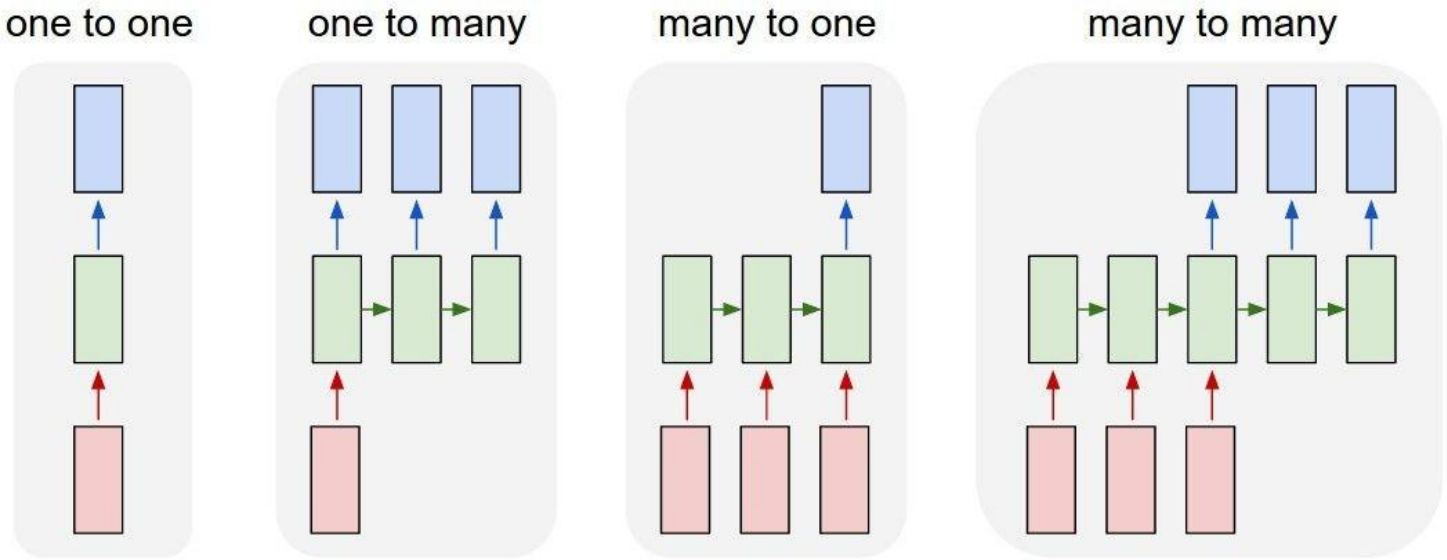

e.g. **Image Captioning**
image -> sequence of words

# Recurrent Neural Networks: Process Sequences



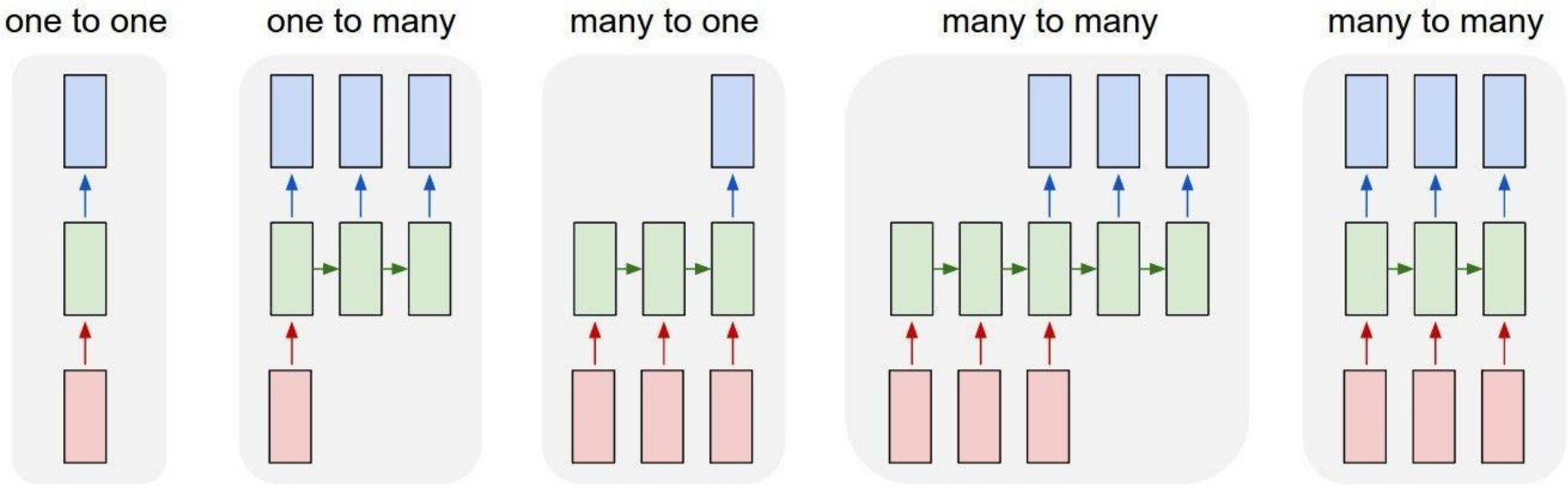

e.g. **action prediction**
sequence of video frames -> action class

# Recurrent Neural Networks: Process Sequences




E.g. **Video Captioning**
Sequence of video frames -> caption

# Recurrent Neural Networks: Process Sequences




e.g. **Video classification on frame level**



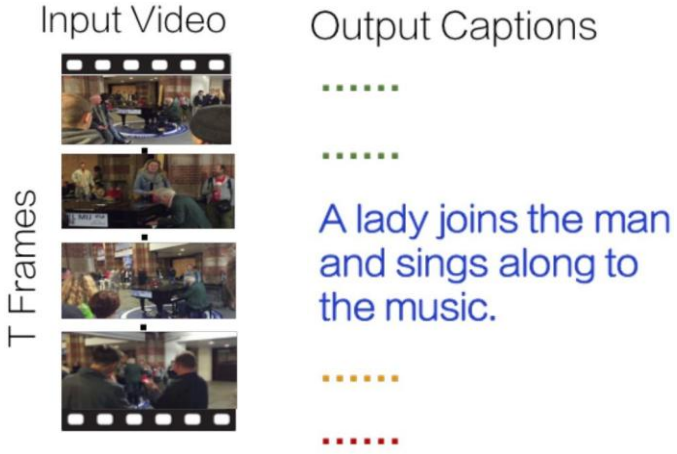
# Why existing ConvNets are insufficient?

## Variable sequence length inputs and outputs!

**Example task**: video captioning

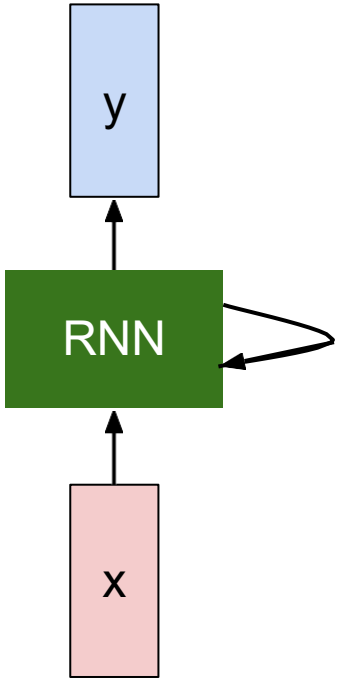
**Input** video can have variable number of frames

**Output** captions can be variable length.

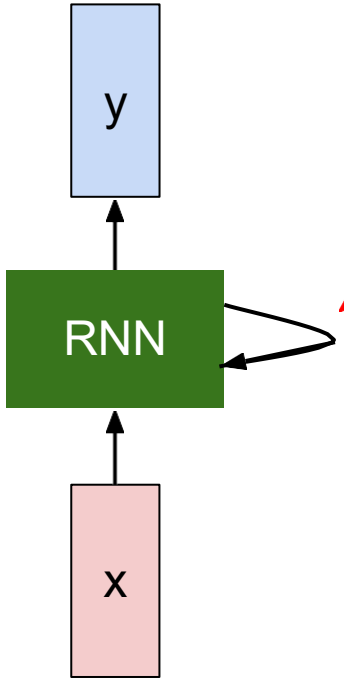



Krishna, Hata, Ren, Fei-Fei, Niebles. Dense captioning Events in Videos. ICCV 2019

# Recurrent Neural Network



# Recurrent Neural Network




Key idea: RNNs have an "internal state" that is updated as a sequence is processed



# RNN hidden state update

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

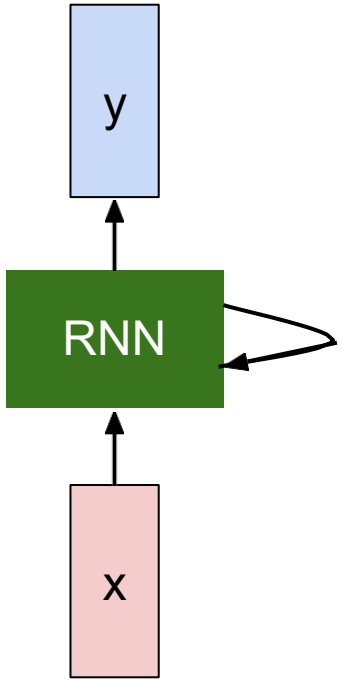
$$h_t = f_W(h_{t-1}, x_t)$$

new state

some function with parameters W

old state

input vector at some time step



# RNN output generation

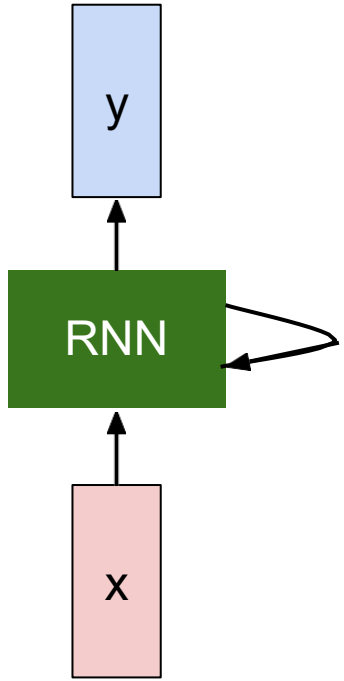
We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$y_t = f_{W_{hy}}(h_t)$$

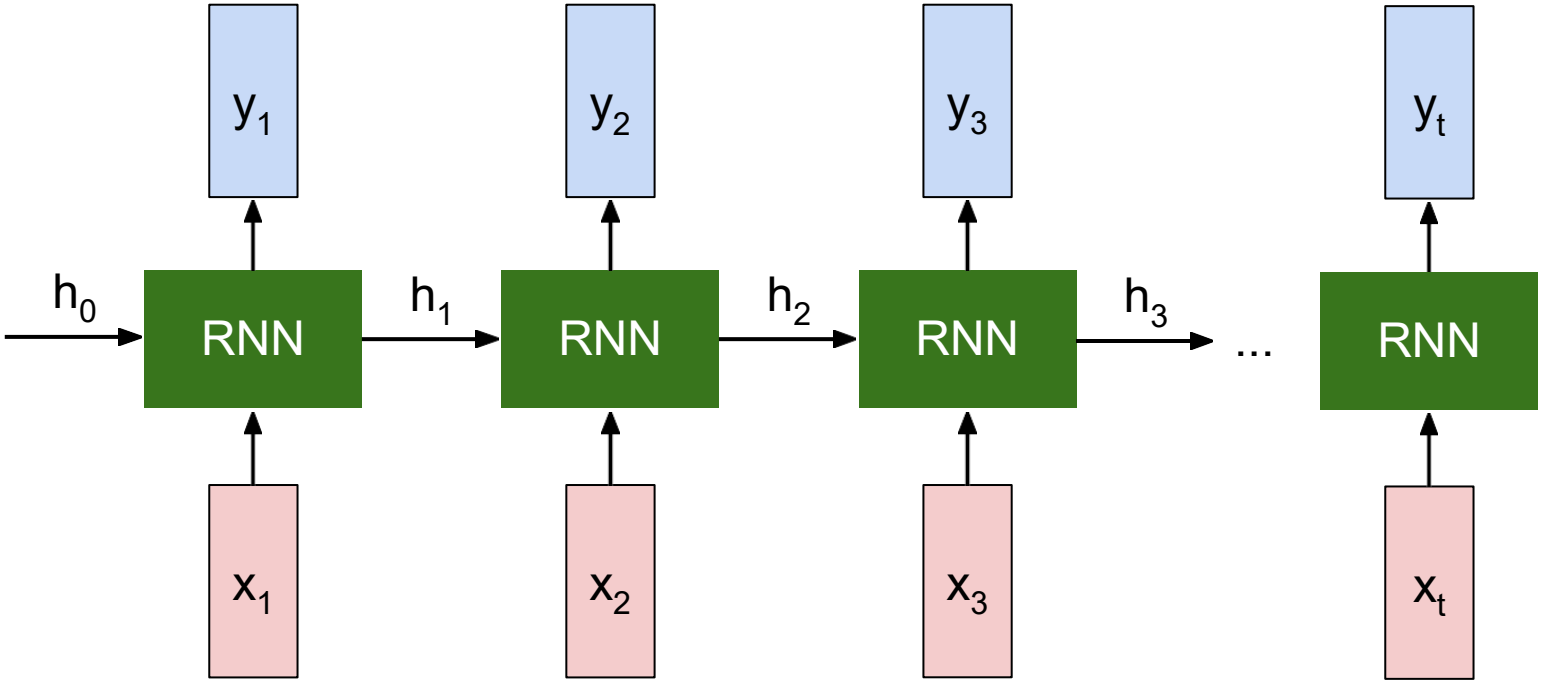
output

new state

another function with parameters $W_o$



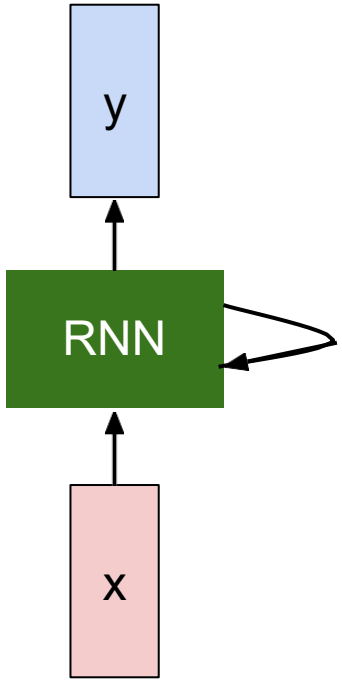
# Recurrent Neural Network



# Recurrent Neural Network

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

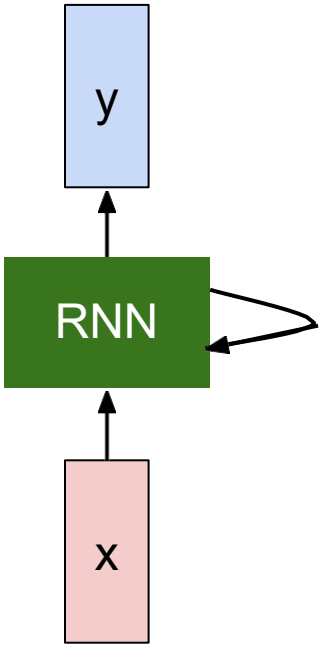



Notice: the same function and the same set of parameters are used at every time step.



# (Simple) Recurrent Neural Network

The state consists of a single *"hidden"* vector **h**:



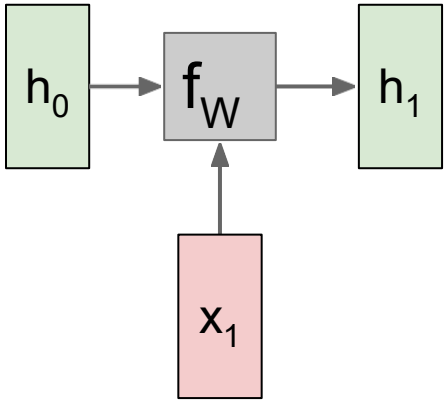

$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

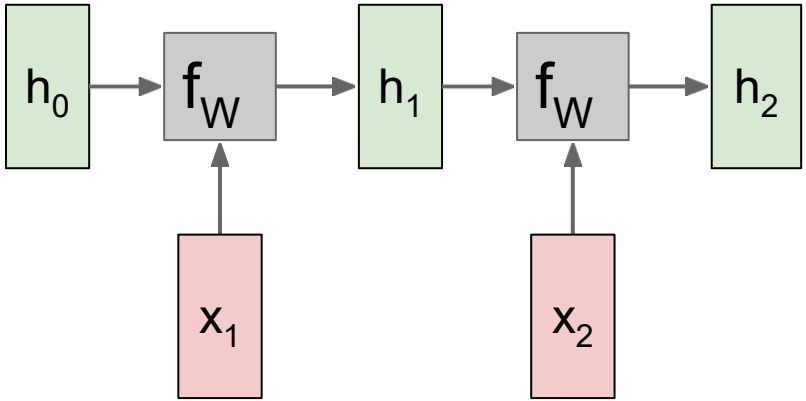
Sometimes called a "Vanilla RNN" or an "Elman RNN" after Prof. Jeffrey Elman

# RNN: Computational Graph



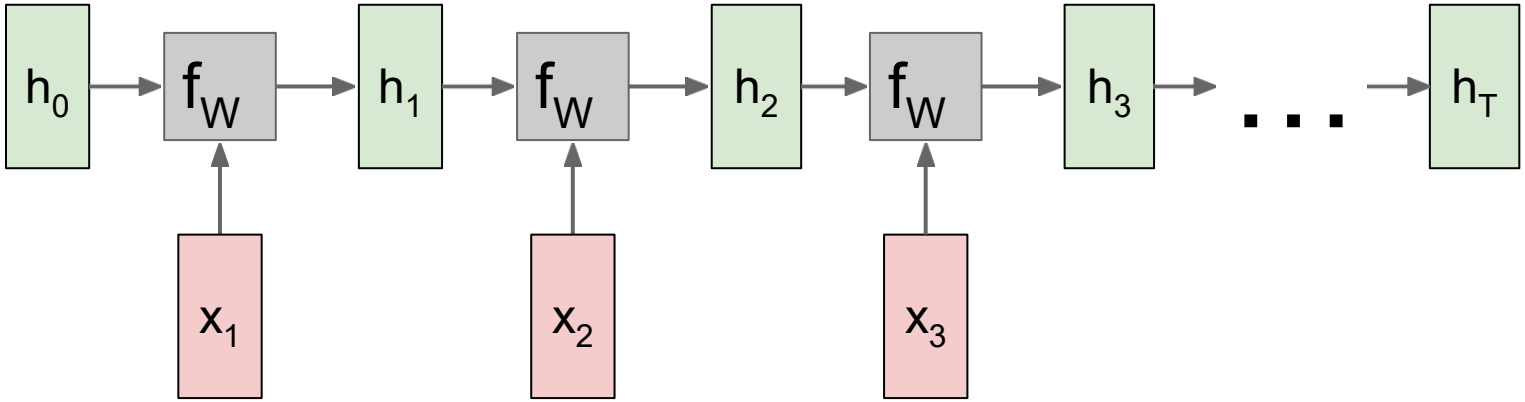



# RNN: Computational Graph





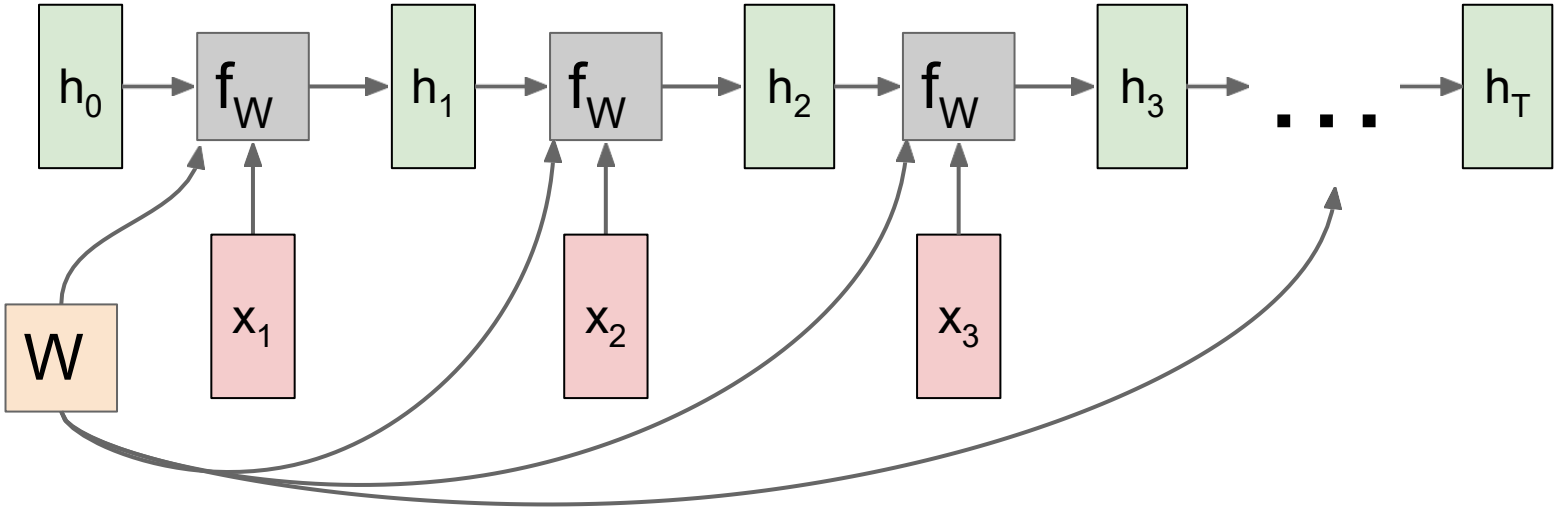

# RNN: Computational Graph



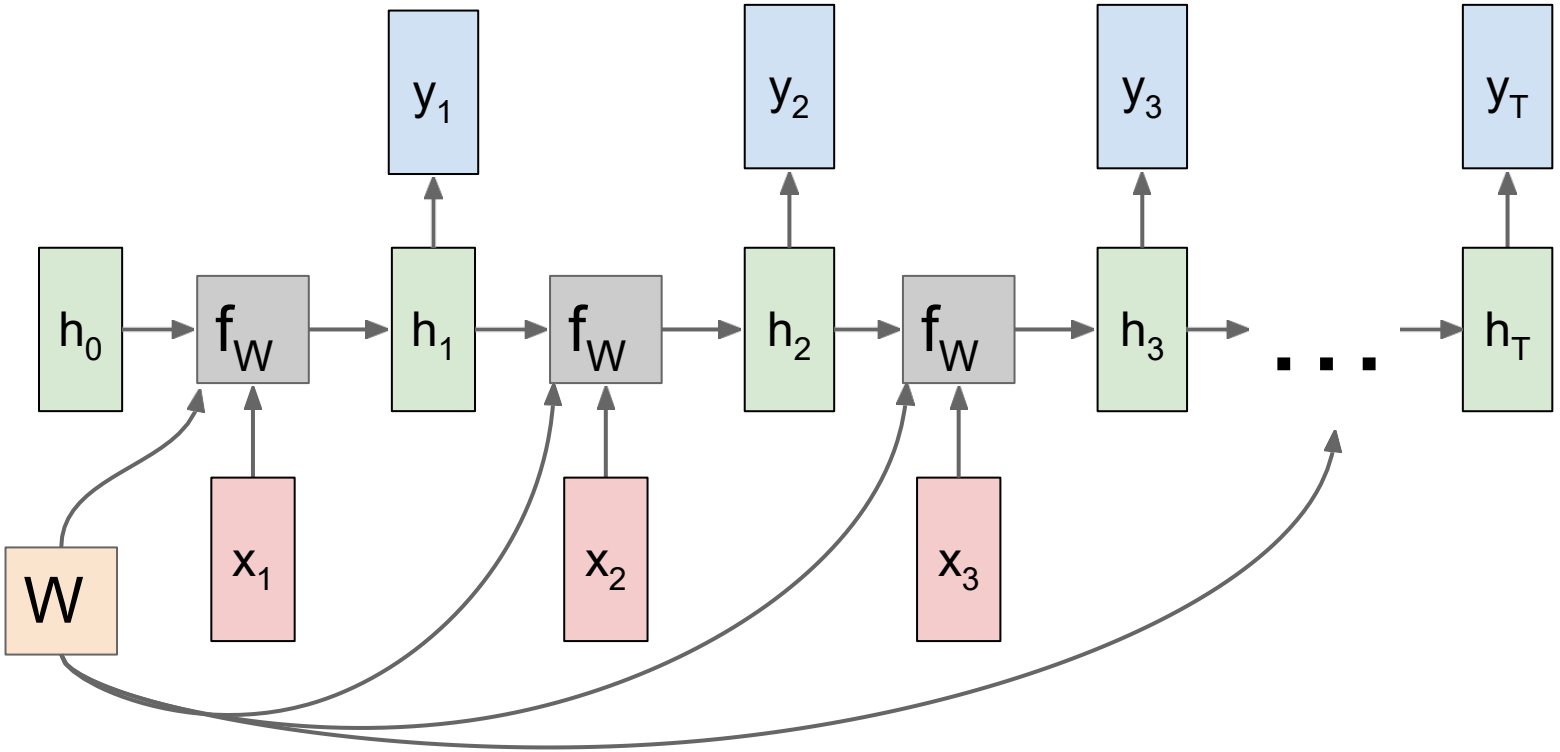



# RNN: Computational Graph

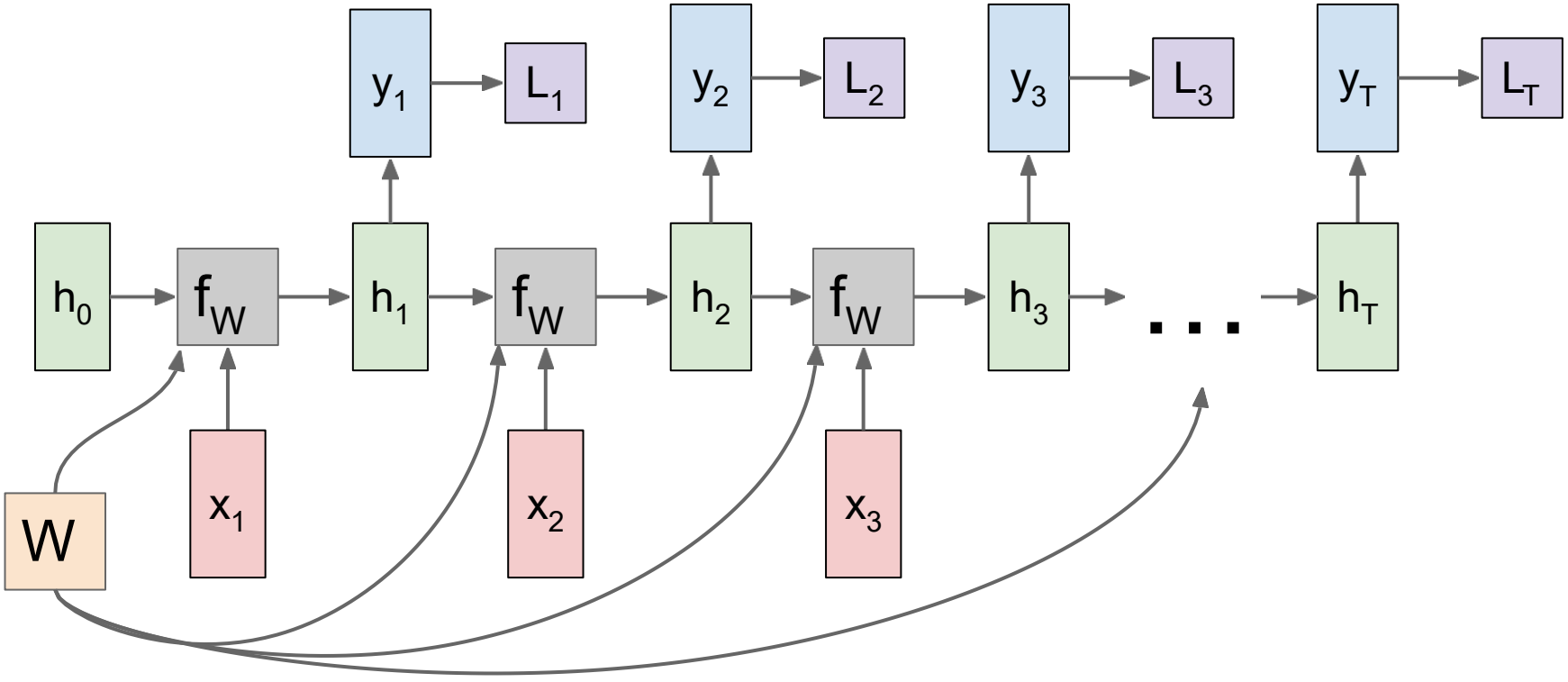
Re-use the same weight matrix at every time-step



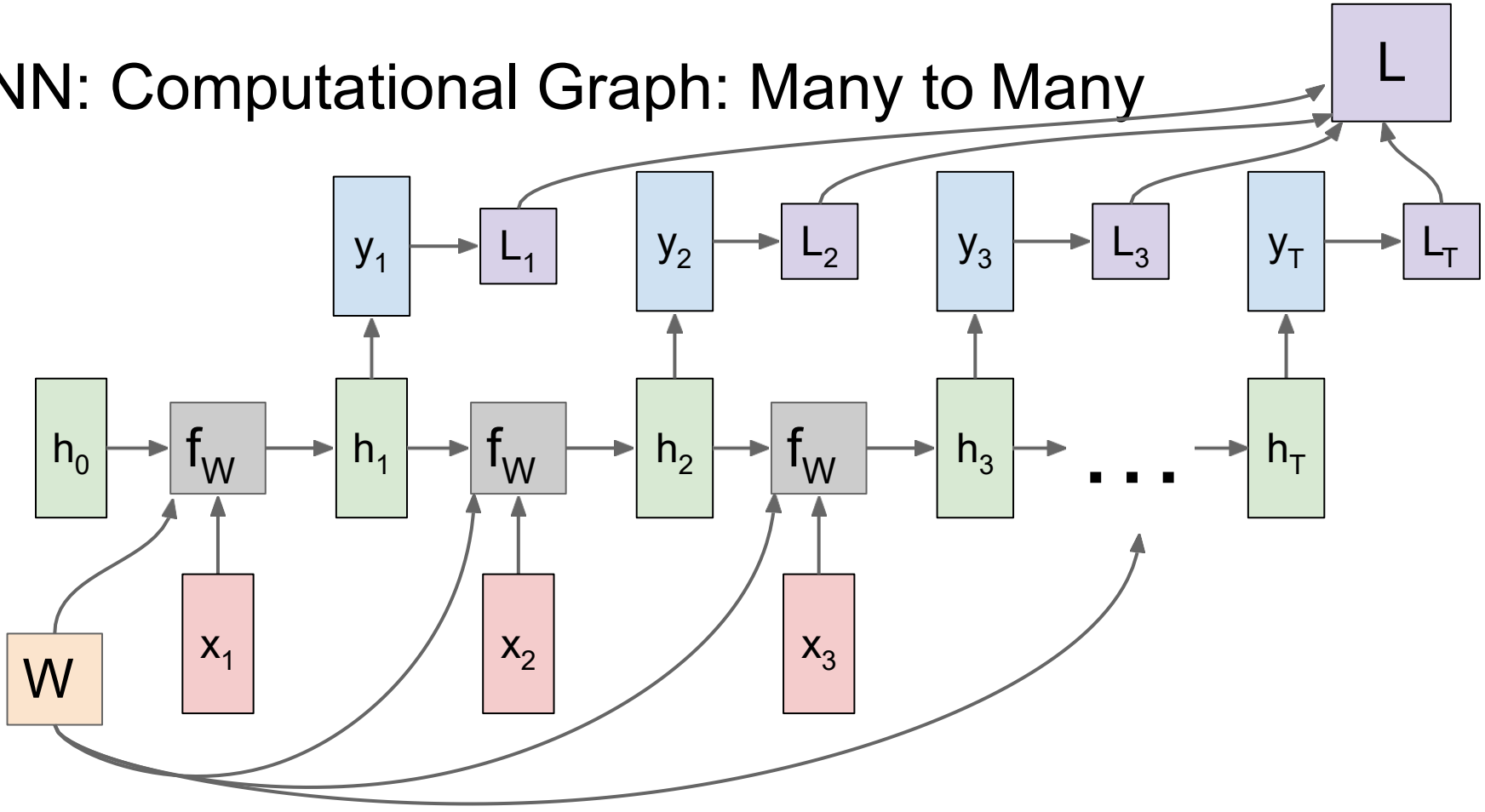
# RNN: Computational Graph: Many to Many



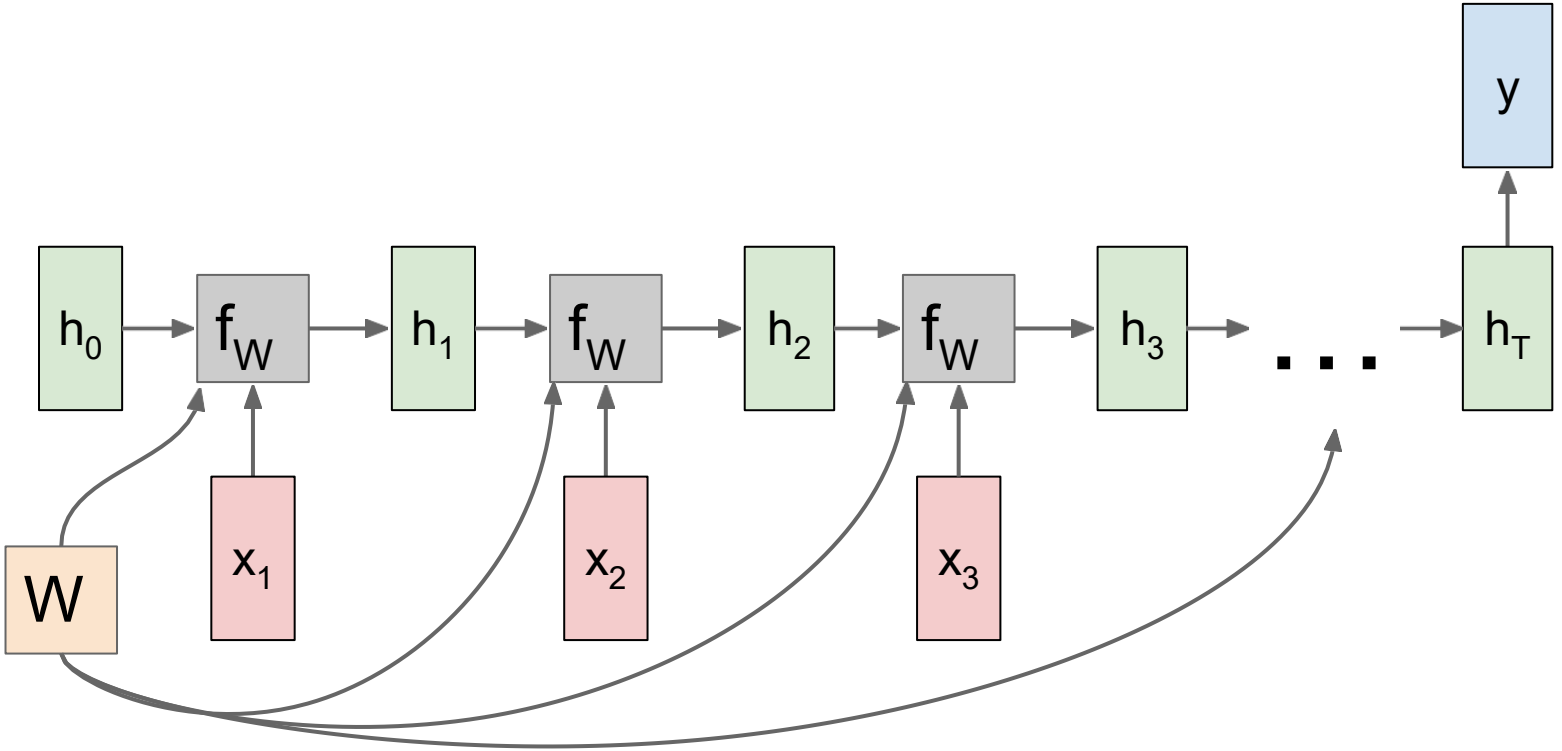
# RNN: Computational Graph: Many to Many



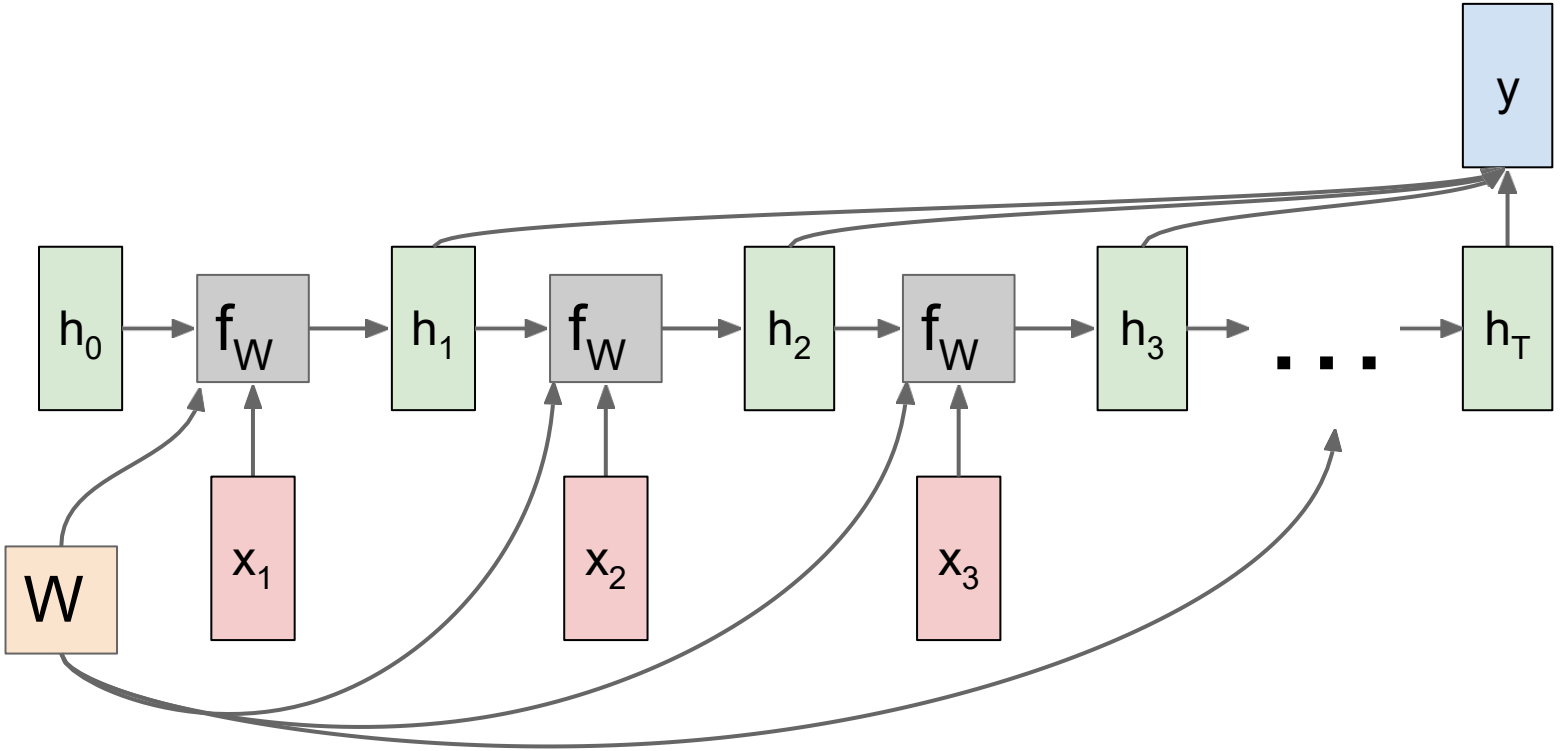
# RNN: Computational Graph: Many to Many



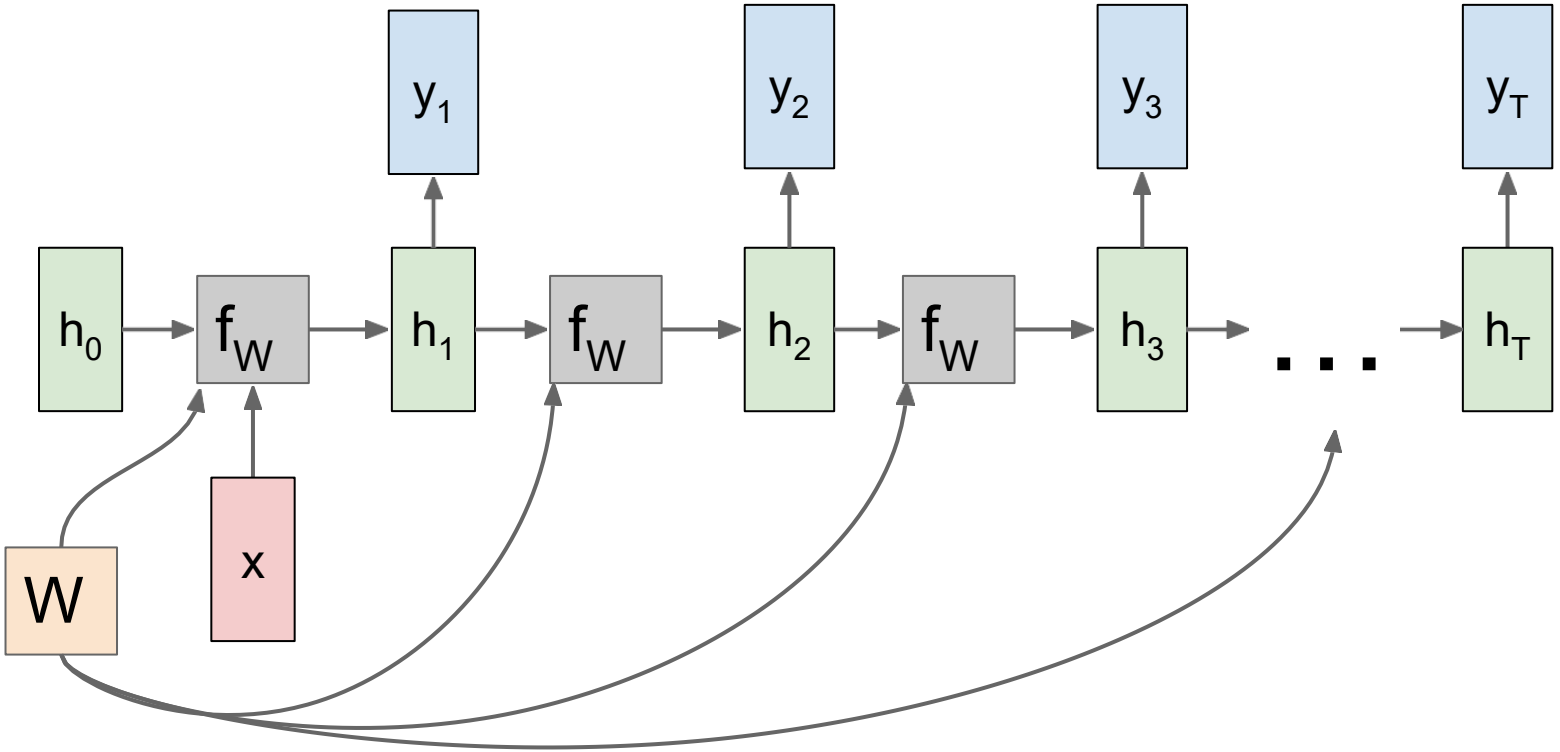
# RNN: Computational Graph: Many to One



# RNN: Computational Graph: Many to One

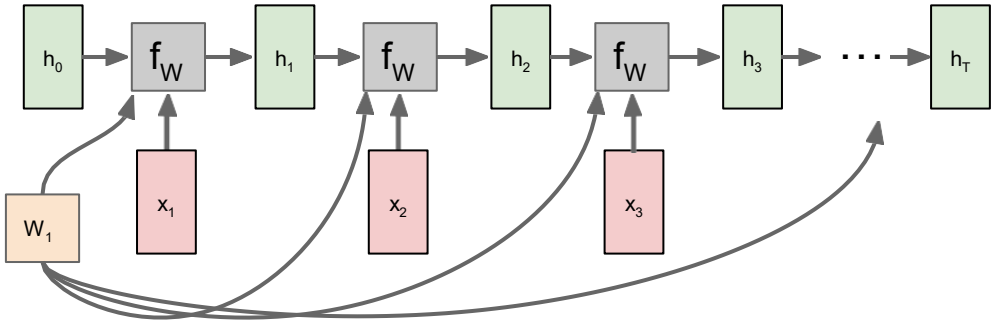


# RNN: Computational Graph: One to Many



# Sequence to Sequence: Many-to-one + one-to-many

**Many to one**: Encode input sequence in a single vector



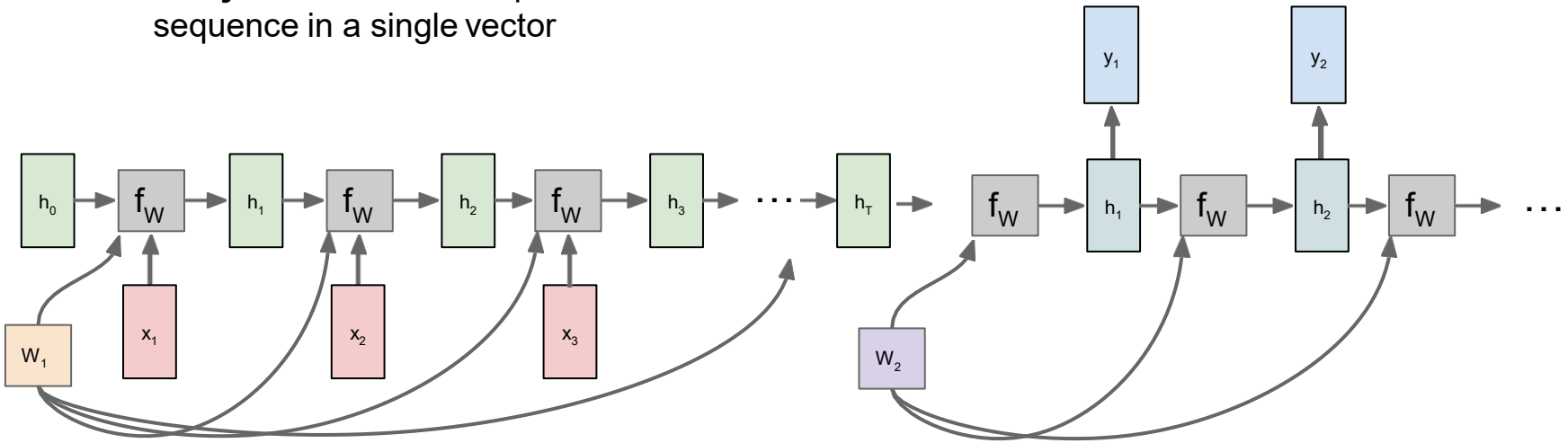

Sutskever et al, “Sequence to Sequence Learning with Neural Networks”, NIPS 2014



# Sequence to Sequence: Many-to-one + one-to-many

**Many to one**: Encode input sequence in a single vector

**One to many**: Produce output sequence from single input vector



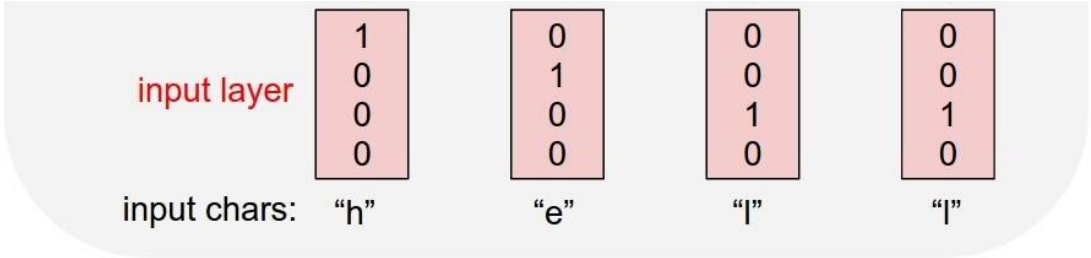

Sutskever et al, “Sequence to Sequence Learning with Neural Networks”, NIPS 2014



# Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training sequence:
**“hello”**

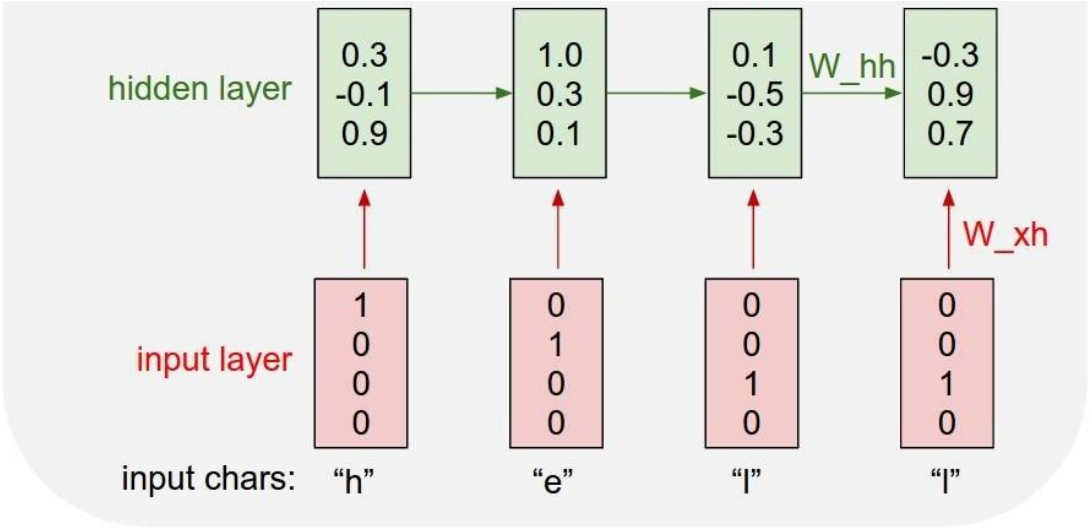


**Example: Character-level Language Model**

Vocabulary:
[h,e,l,o]

Example training sequence:
**“hello”**

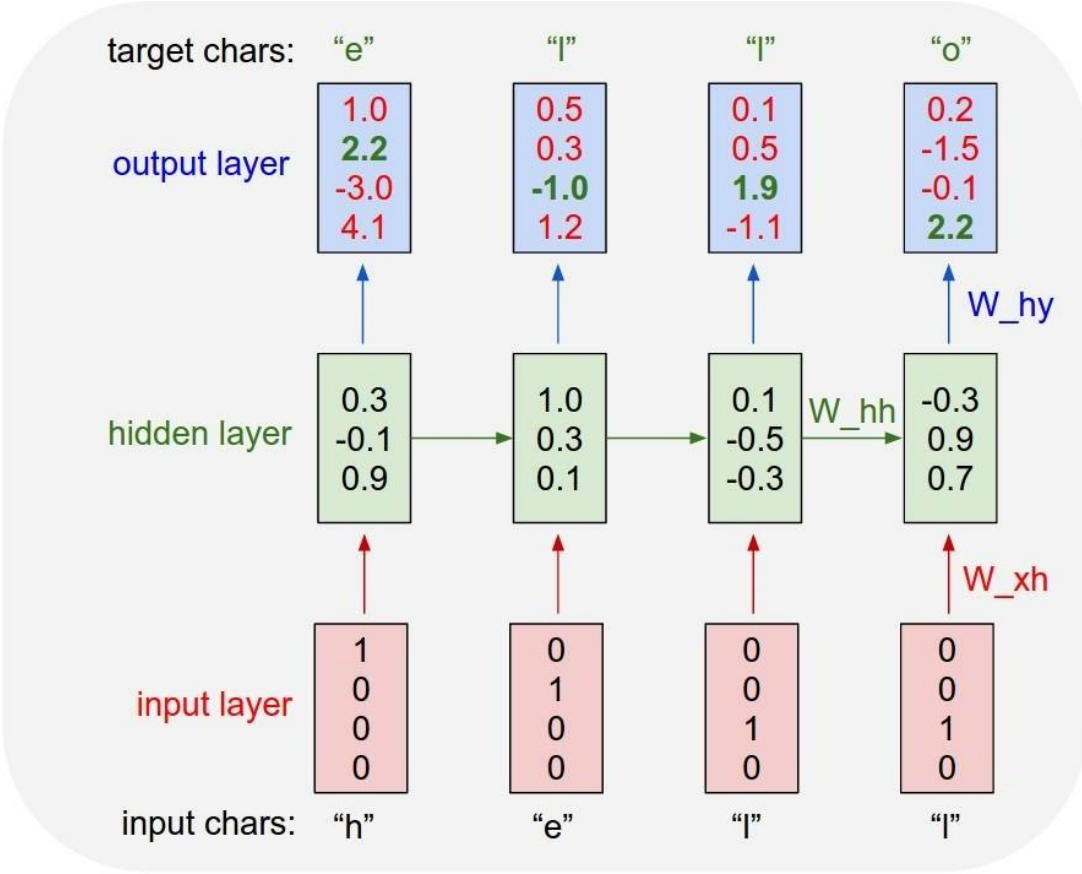
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



# Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

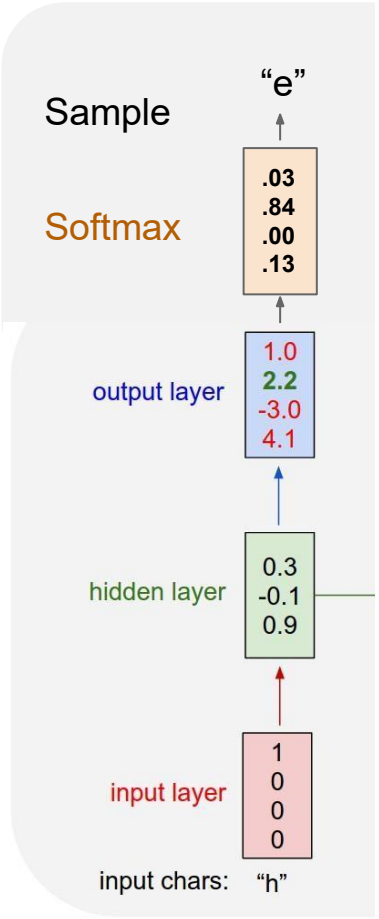
Example training sequence:
**“hello”**



# Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

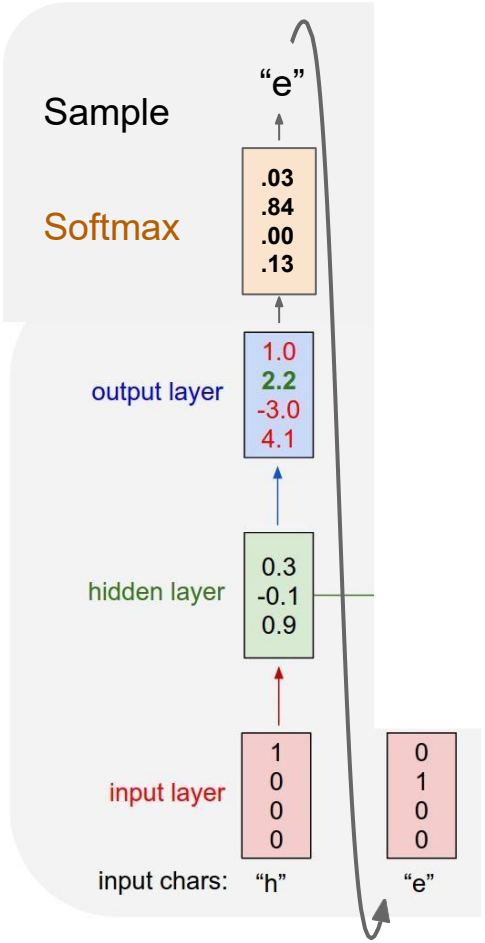
At test-time sample characters one at a time, feed back to model



# Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

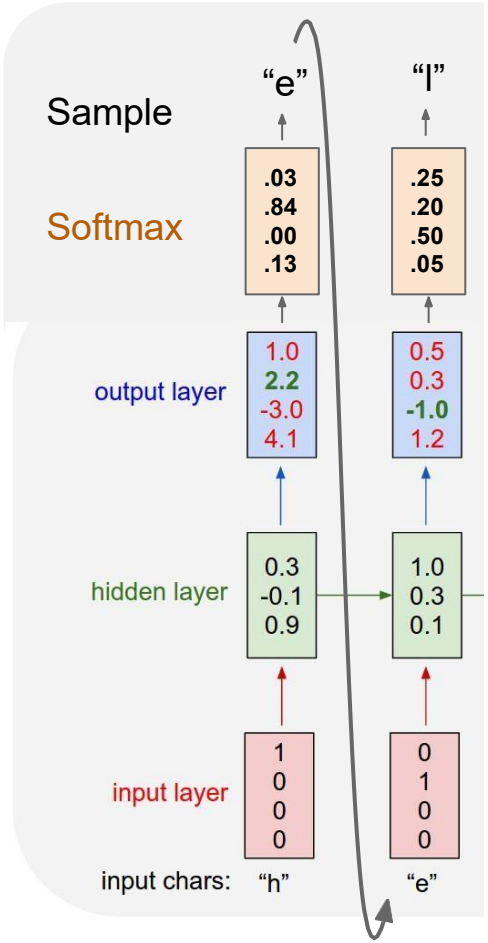
At test-time sample characters one at a time, feed back to model



# Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

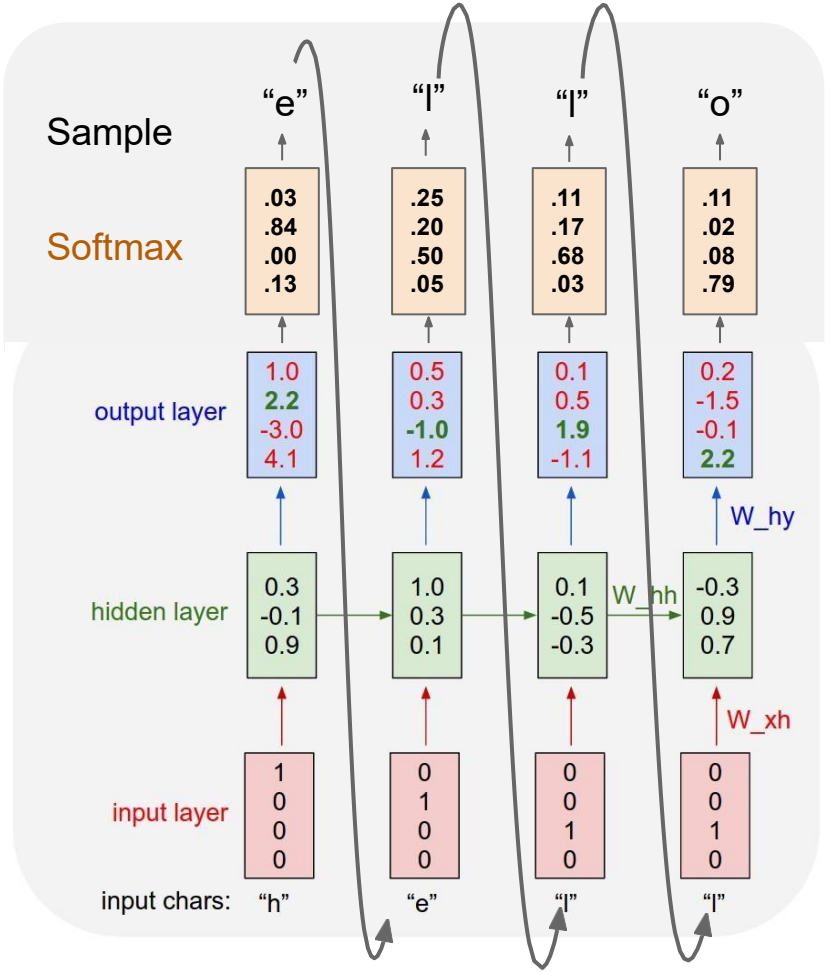
At test-time sample characters one at a time, feed back to model



# Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

At test-time sample characters one at a time, feed back to model



# Computer Vision Application

- Visual Captioning – Describe the content of an image or video with a natural language sentence.



A cat is sitting next to a pine tree, looking up.



A dog is playing piano with a girl.

# test image



This image is CC0 public domain



image

conv-64
conv-64
maxpool

conv-128
conv-128
maxpool

conv-256
conv-256
maxpool

conv-512
conv-512
maxpool

conv-512
conv-512
maxpool

FC-4096
FC-4096
FC-1000
softmax



test image

image

conv-64
conv-64
maxpool

conv-128
conv-128
maxpool

conv-256
conv-256
maxpool

conv-512
conv-512
maxpool

conv-512
conv-512
maxpool

FC-4096
FC-4096
FC-1000
softmax

X



test image

image

conv-64
conv-64
maxpool

conv-128
conv-128
maxpool

conv-256
conv-256
maxpool

conv-512
conv-512
maxpool

conv-512
conv-512
maxpool

FC-4096
FC-4096



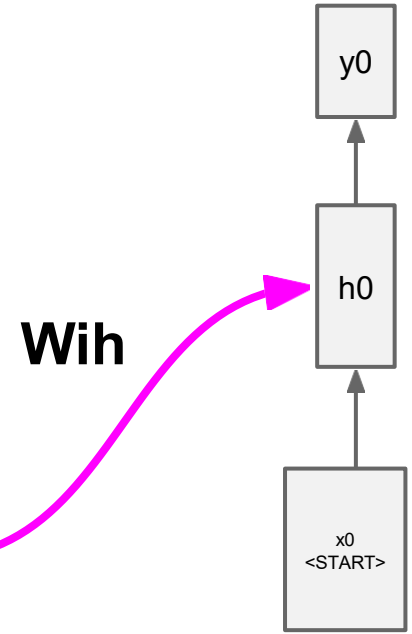
test image

x0
<START>






test image




**before:**

$h = \tanh(W_{xh} * x + W_{hh} * h)$

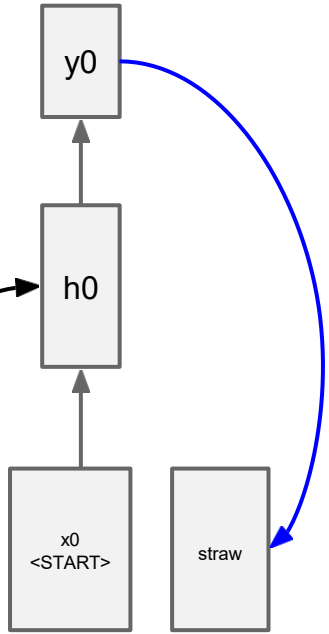
**now:**

$h = \tanh(W_{xh} * x + W_{hh} * h + \mathbf{W_{ih} * v})$






test image

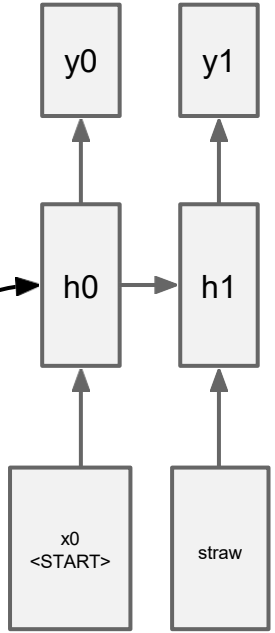



sample!


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096



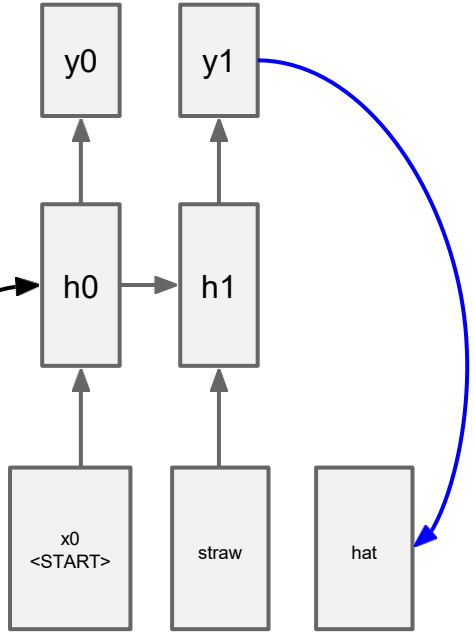
test image


y0
y1
h0
h1
x0
<START>
straw

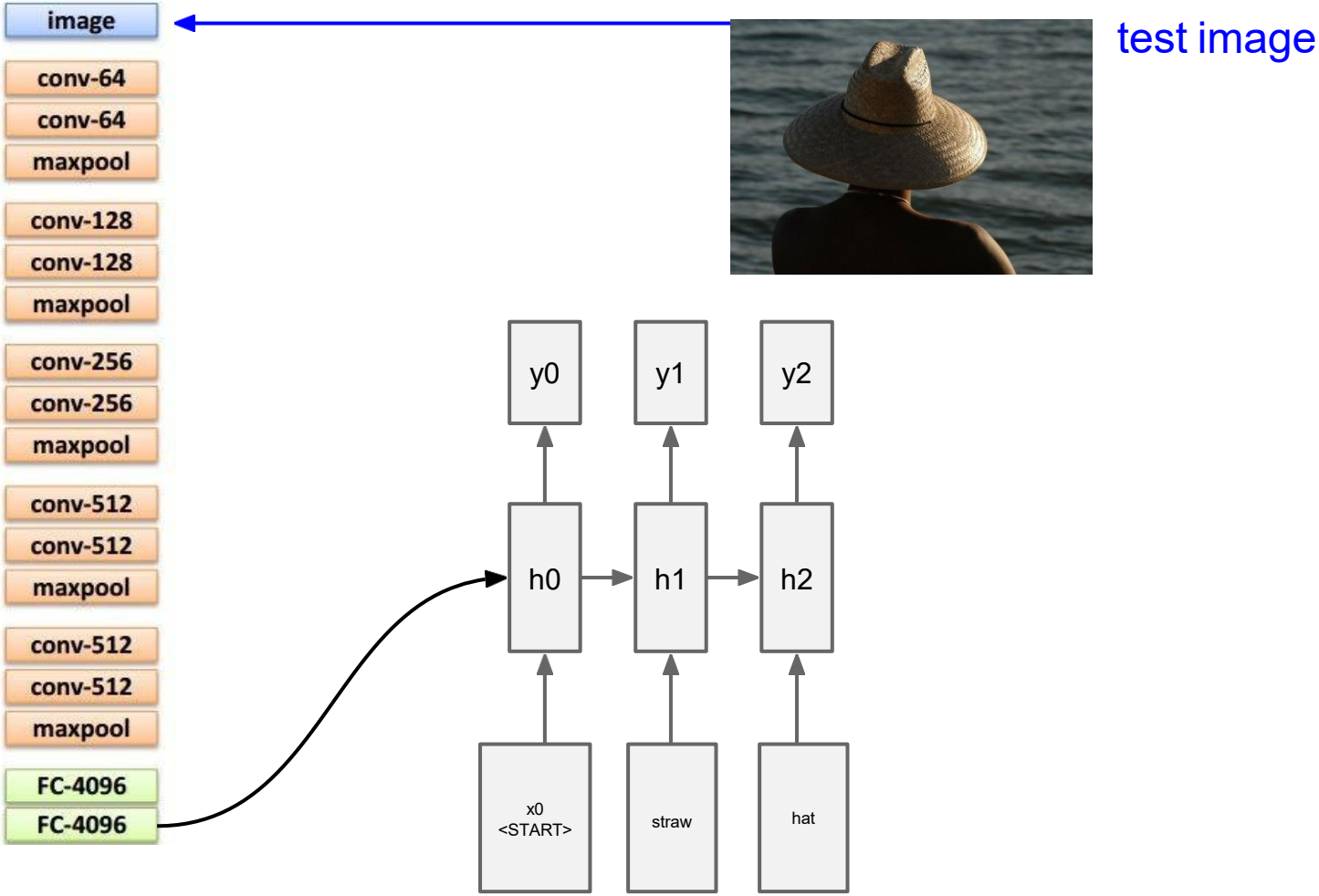


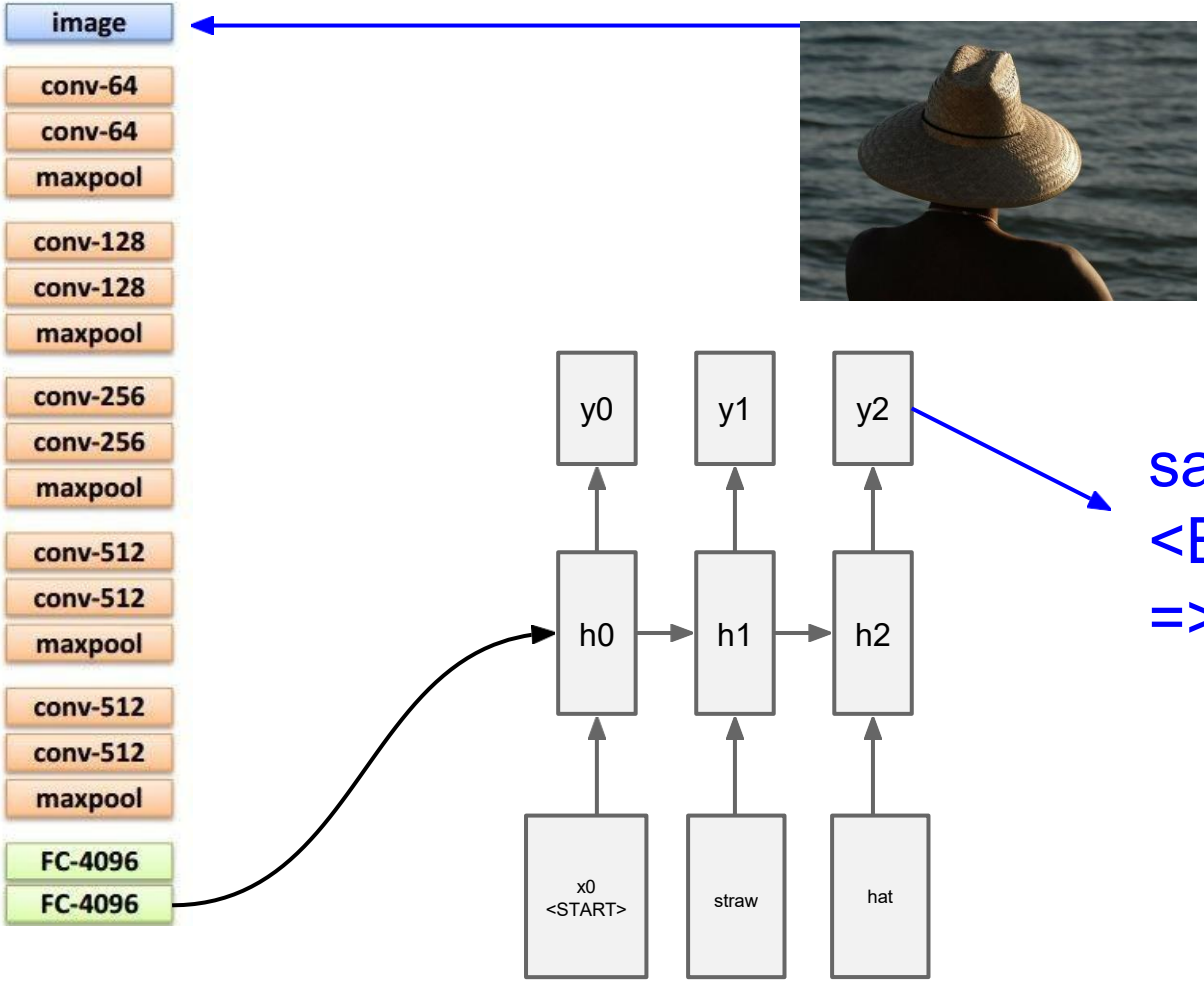



test image




sample!


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
test image
y0
y1
y2
h0
h1
h2
x0
<START>
straw
hat


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
test image
y0
y1
y2
h0
h1
h2
x0
<START>
straw
hat
sample
<END> token
=> finish.

# Image Captioning: Example Results

Captions generated using neuraltalk2
All images are CC0 Public domain:
cat suitcase, cat tree, dog, bear,
surfers, tennis, giraffe, motorcycle



*A cat sitting on a suitcase on the floor*



*A cat is sitting on a tree branch*



*A dog is running in the grass with a frisbee*



*A white teddy bear sitting in the grass*



*Two people walking on the beach with surfboards*




*A tennis player in action on the court*



*Two giraffes standing in a grassy field*



*A man riding a dirt bike on a dirt track*

# Image Captioning: Failure Cases

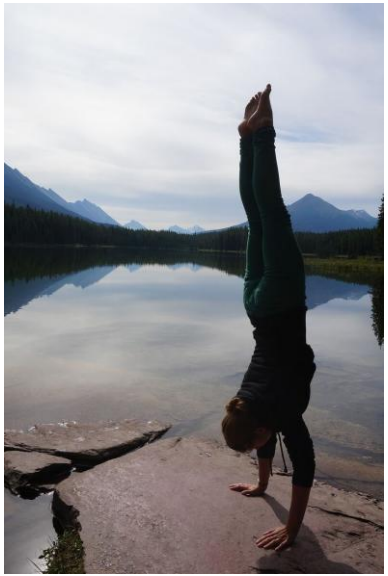
Captions generated using neuraltalk2
All images are CC0 Public domain: fur coat, handstand, spider web, baseball



*A woman is holding a cat in her hand*



*A person holding a computer mouse on a desk*



*A woman standing on a beach holding a surfboard*



*A bird is perched on a tree branch*



*A man in a baseball uniform throwing a ball*

# Outline

- **Attention with RNNs**
  - In Computer Vision
  - In NLP
- **General Attention Layer**
  - Self-attention
  - Positional encoding
  - Masked attention
  - Multi-head attention
- **Transformers**



# Outline

- **Attention with RNNs**
  - In Computer Vision
  - In NLP
- **General Attention Layer**
  - Self-attention
  - Positional encoding
  - Masked attention
  - Multi-head attention
- **Transformers**



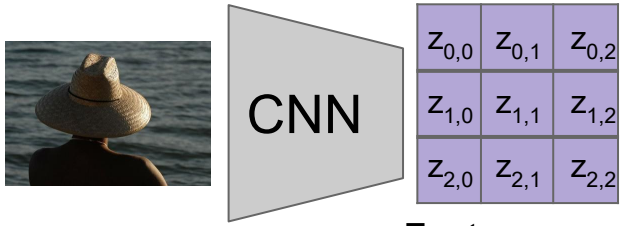
Image Captioning with Vanilla RNN

What’s the problem?



# Image Captioning using spatial features

**Input**: Image **I**
**Output:** Sequence **y** = $y_1, y_2, ..., y_T$




Extract spatial features from a pretrained CNN

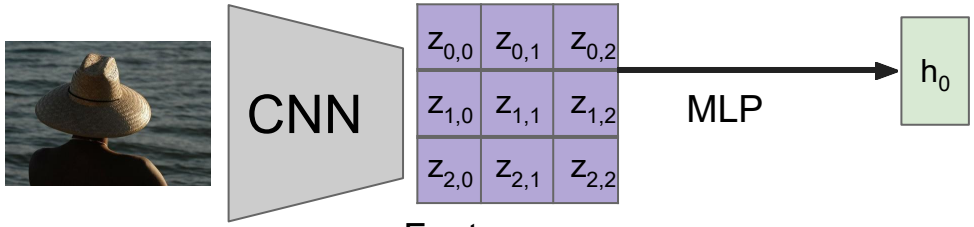
Features: H x W x D

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning using spatial features

**Input**: Image **I**
**Output:** Sequence **y** = $y_1, y_2, ..., y_T$

**Encoder**: $h_0 = f_{\mathbf{W}}(\mathbf{z})$
where **z** is spatial CNN features
$f_{\mathbf{W}}(.)$ is an MLP




Extract spatial features from a pretrained CNN

Features: H x W x D

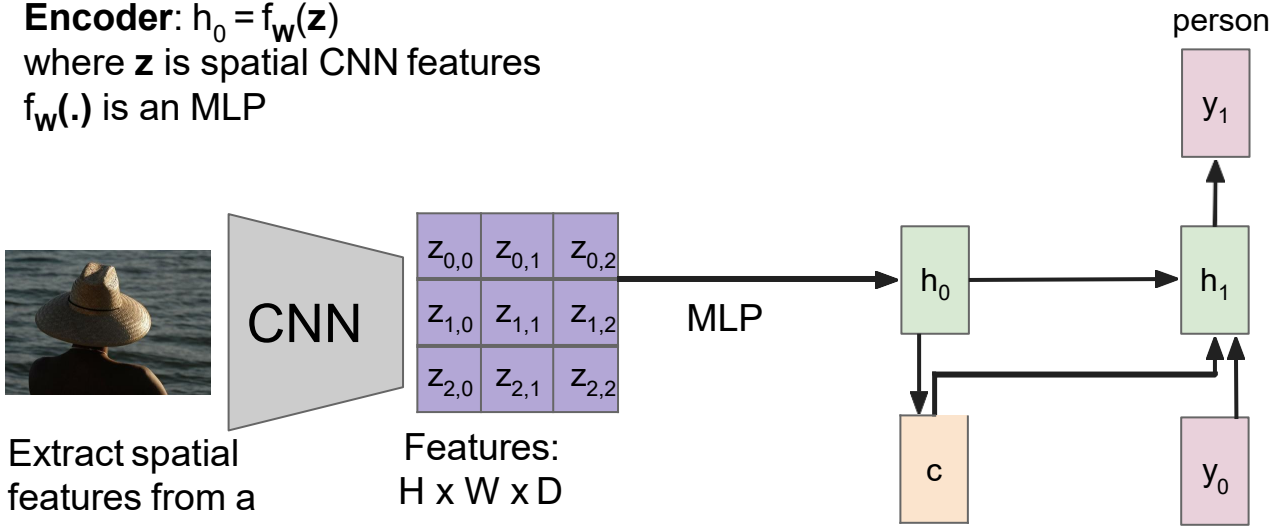
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning using spatial features

**Input**: Image **I**
**Output:** Sequence **y** = $y_1, y_2, \ldots, y_T$

**Decoder**: $y_t = g_{\mathbf{V}}(y_{t-1}, h_{t-1}, c)$
where context vector c is often $c = h_0$

**Encoder**: $h_0 = f_{\mathbf{W}}(\mathbf{z})$
where **z** is spatial CNN features
$f_{\mathbf{W}}(.)$ is an MLP




Extract spatial features from a pretrained CNN

Features: H x W x D

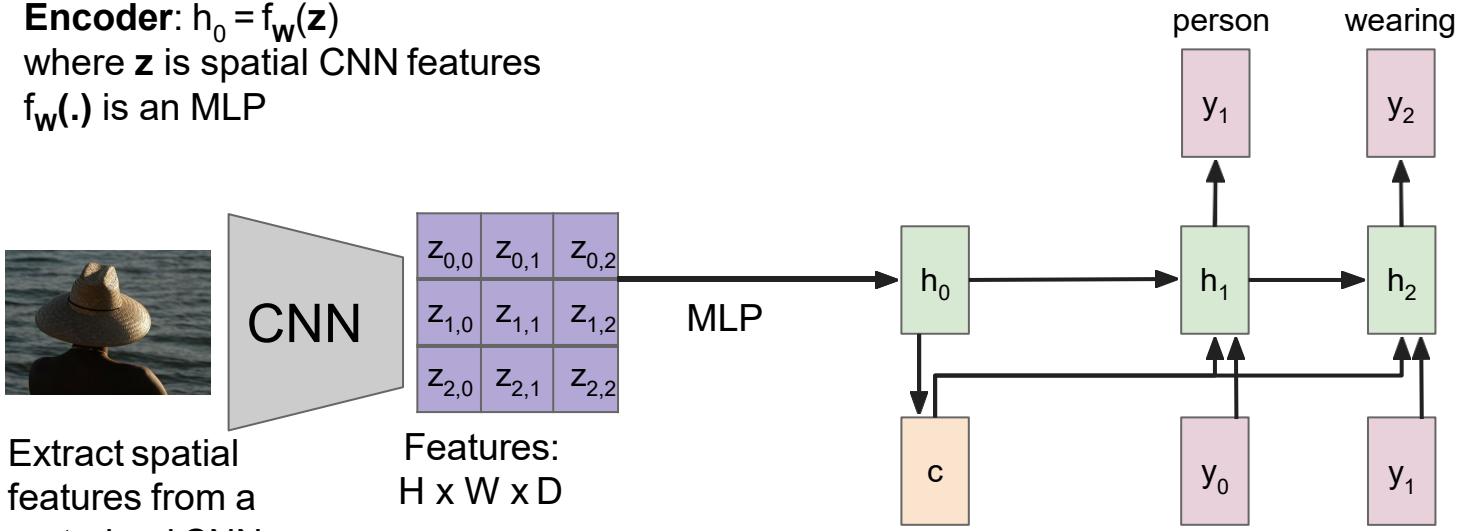
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning using spatial features

**Input**: Image **I**
**Output:** Sequence **y** = $y_1, y_2 \ldots, y_T$

**Decoder**: $y_t = g_{\mathbf{V}}(y_{t-1}, h_{t-1}, c)$
where context vector c is often $c = h_0$

**Encoder**: $h_0 = f_{\mathbf{W}}(\mathbf{z})$
where **z** is spatial CNN features
$f_{\mathbf{W}}(.)$ is an MLP




Extract spatial features from a pretrained CNN

Features: H x W x D

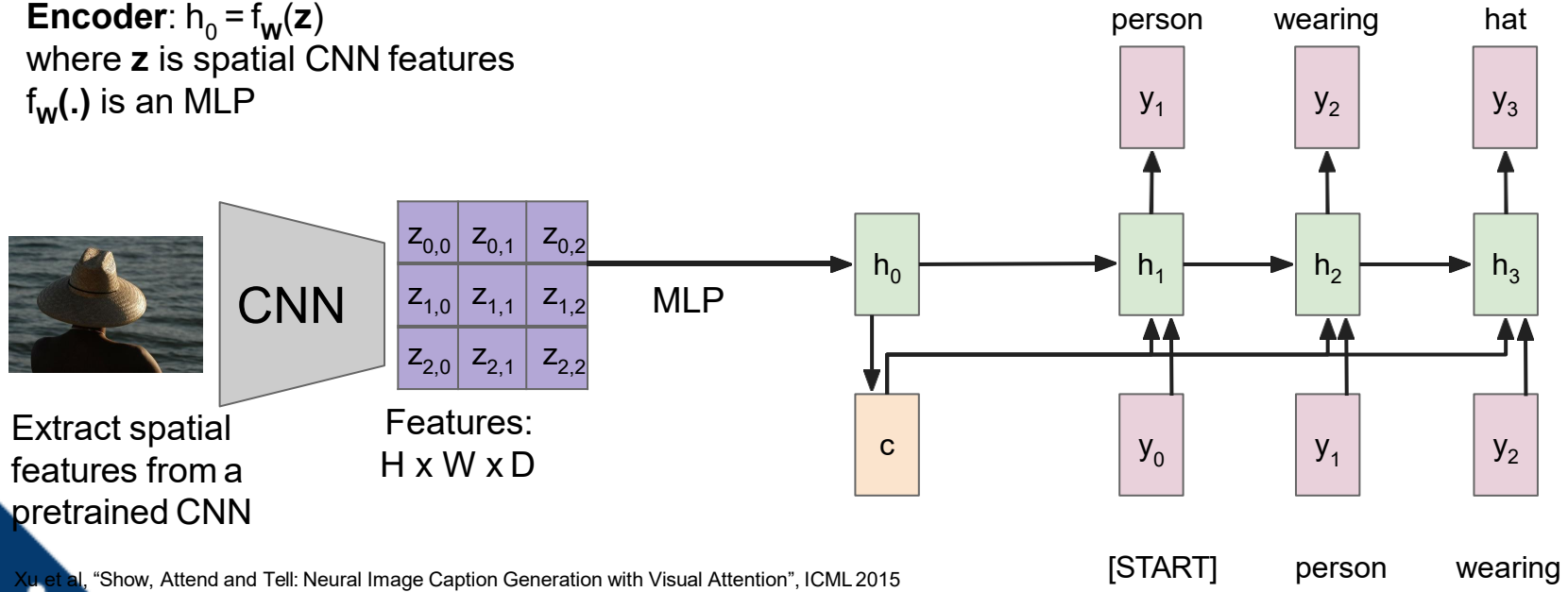
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning using spatial features

**Input**: Image **I**
**Output:** Sequence **y** = $y_1, y_2, \ldots, y_T$

**Decoder**: $y_t = g_{\mathbf{V}}(y_{t-1}, h_{t-1}, c)$
where context vector c is often $c = h_0$

**Encoder**: $h_0 = f_{\mathbf{W}}(\mathbf{z})$
where **z** is spatial CNN features
$f_{\mathbf{W}}(.)$ is an MLP



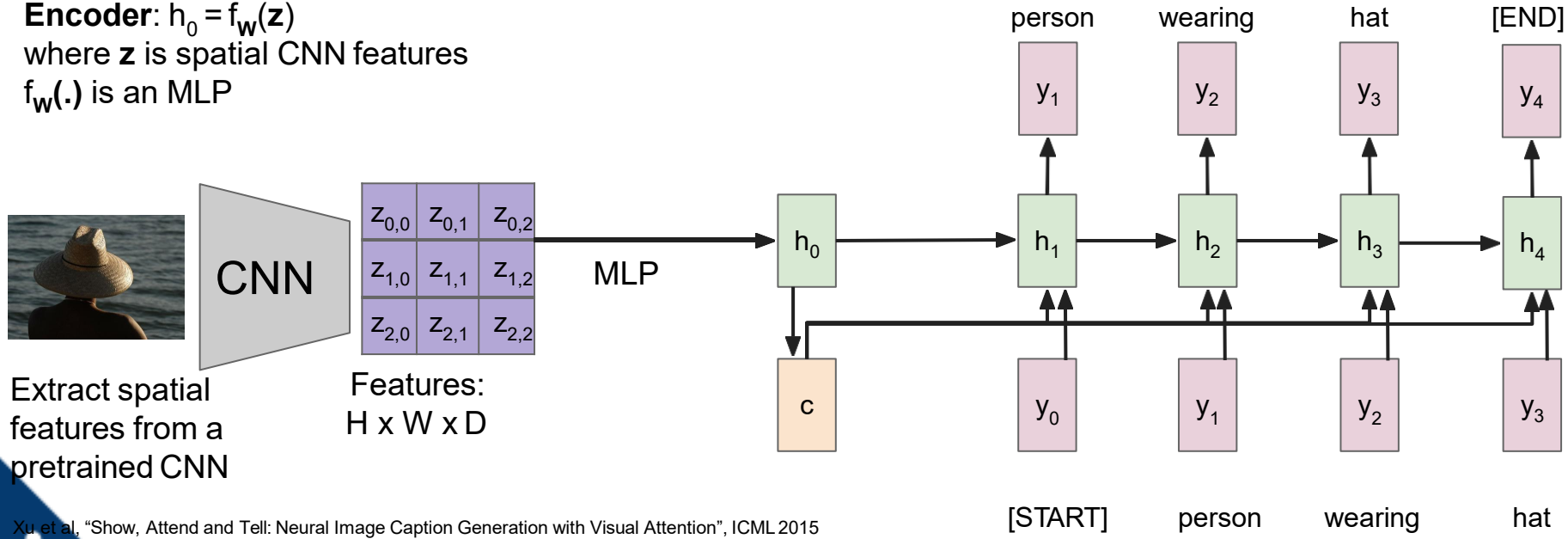

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning using spatial features

**Input**: Image **I**
**Output:** Sequence **y** = $y_1, y_2, \ldots, y_T$

**Decoder**: $y_t = g_{\mathbf{V}}(y_{t-1}, h_{t-1}, c)$
where context vector c is often $c = h_0$

**Encoder**: $h_0 = f_{\mathbf{W}}(\mathbf{z})$
where **z** is spatial CNN features
$f_{\mathbf{W}}(.)$ is an MLP




Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning using spatial features

**Problem: Input is "bottlenecked" through c**

- Model needs to encode everything it wants to say within c

**This is a problem if we want to generate really long descriptions? hundreds of words long**



CNN

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$
$z_{1,0}$ $z_{1,1}$ $z_{1,2}$
$z_{2,0}$ $z_{2,1}$ $z_{2,2}$

Extract spatial features from a pretrained CNN

Features: H x W x D

MLP

$h_0$ $h_1$ $h_2$ $h_3$ $h_4$

c

$y_0$ $y_1$ $y_2$ $y_3$

$y_1$ $y_2$ $y_3$ $y_4$

[START] person wearing hat

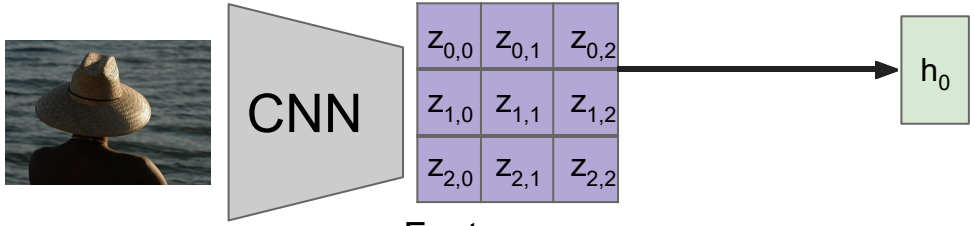
person wearing hat [END]

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with RNNs & Attention

**Attention idea: New context vector at every time step.**

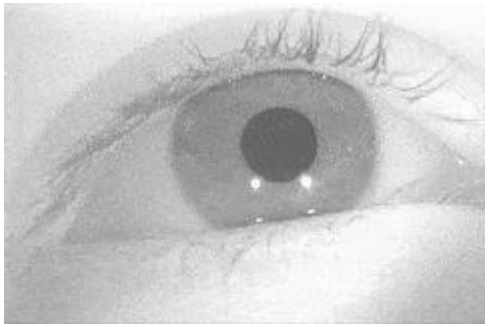
**Each context vector will attend to different image regions**




Extract spatial features from a pretrained CNN

Features: H x W x D

gif source



**Attention Saccades in humans**

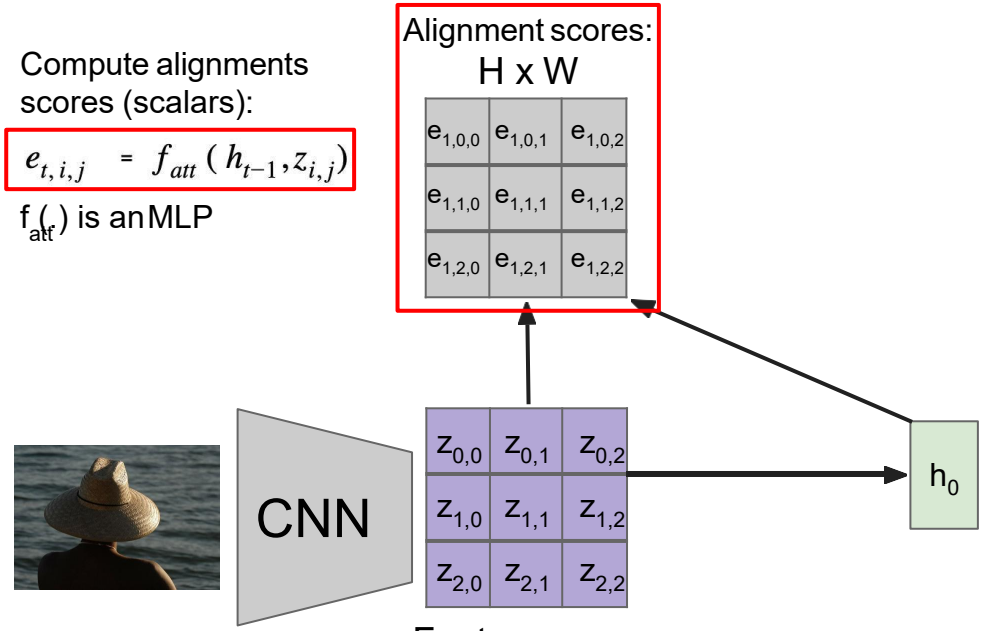
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with RNNs & Attention

Compute alignments scores (scalars):

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$f_{att}(.)$ is an MLP




Extract spatial features from a pretrained CNN

Features:
H x W x D

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with RNNs & Attention

Compute alignments scores (scalars):

$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$

$f_{att}(.)$ is an MLP

Alignment scores: H x W

| | | |
|---|---|---|
| $e_{1,0,0}$ | $e_{1,0,1}$ | $e_{1,0,2}$ |
| $e_{1,1,0}$ | $e_{1,1,1}$ | $e_{1,1,2}$ |
| $e_{1,2,0}$ | $e_{1,2,1}$ | $e_{1,2,2}$ |

Attention: H x W

| | | |
|---|---|---|
| $a_{1,0,0}$ | $a_{1,0,1}$ | $a_{1,0,2}$ |
| $a_{1,1,0}$ | $a_{1,1,1}$ | $a_{1,1,2}$ |
| $a_{1,2,0}$ | $a_{1,2,1}$ | $a_{1,2,2}$ |

Normalize to get attention weights:

$a_{t,:,:} = softmax(e_{t,:,:})$

$0 < a_{t,i,j} < 1$,
attention values sum to 1



CNN

| | | |
|---|---|---|
| $z_{0,0}$ | $z_{0,1}$ | $z_{0,2}$ |
| $z_{1,0}$ | $z_{1,1}$ | $z_{1,2}$ |
| $z_{2,0}$ | $z_{2,1}$ | $z_{2,2}$ |

$h_0$

Features: H x W x D

Extract spatial features from a pretrained CNN

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015



# Image Captioning with RNNs & Attention

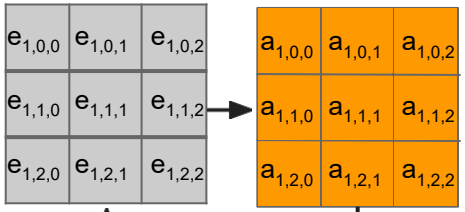
Compute alignments scores (scalars):

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$f_{att}(.)$ is an MLP

Alignment scores: H x W

Attention: H x W







Normalize to get attention weights:

$$a_{t,:,:} = softmax(e_{t,:,:})$$

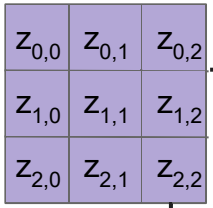
$0 < a_{t,i,j} < 1$,
attention values sum to 1

Compute context vector:

$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$

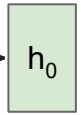


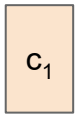
CNN




Features: H x W x D

Extract spatial features from a pretrained CNN







Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with RNNs & Attention

Each timestep of decoder uses a different context vector that looks at different parts of the input image

**Decoder**: $y_t = g_V(y_{t-1}, h_{t-1}, c_t)$
New context vector at every time step

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:} = softmax(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



CNN

Extract spatial features from a pretrained CNN

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$
$z_{1,0}$ $z_{1,1}$ $z_{1,2}$
$z_{2,0}$ $z_{2,1}$ $z_{2,2}$

Features: H x W x D

$h_0$ $h_1$

person

$y_1$

$c_1$ $y_0$

[START]

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with RNNs & Attention

**Decoder**: $y_t = g_V(y_{t-1}, h_{t-1}, c_t)$
New context vector at every time step

Alignment scores: H x W

Attention: H x W

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$$a_{t,:,:} = softmax(e_{t,:,:})$$

$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



CNN

Extract spatial features from a pretrained CNN

Features: H x W x D

person

[START]

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with RNNs & Attention

Each timestep of decoder uses a different context vector that looks at different parts of the input image

**Decoder**: $y_t = g_V(y_{t-1}, h_{t-1}, c_t)$
New context vector at every time step

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:} = softmax(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



CNN

Extract spatial features from a pretrained CNN

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$
$z_{1,0}$ $z_{1,1}$ $z_{1,2}$
$z_{2,0}$ $z_{2,1}$ $z_{2,2}$

Features: H x W x D

$h_0$ $h_1$ $h_2$

$y_1$ $y_2$

person wearing

$c_1$ $y_0$ $c_2$ $y_1$

[START] person

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with RNNs & Attention

Each timestep of decoder uses a different context vector that looks at different parts of the input image

**Decoder**: $y_t = g_V(y_{t-1}, h_{t-1}, c_t)$
New context vector at every time step

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:} = softmax(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



CNN

Extract spatial features from a pretrained CNN

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$
$z_{1,0}$ $z_{1,1}$ $z_{1,2}$
$z_{2,0}$ $z_{2,1}$ $z_{2,2}$

Features: H x W x D

$h_0$ $h_1$ $h_2$ $h_3$

$y_1$ $y_2$ $y_3$

person wearing hat

$c_1$ $y_0$ $c_2$ $y_1$ $c_3$ $y_2$

[START] person wearing

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with RNNs & Attention

Each timestep of decoder uses a different context vector that looks at different parts of the input image

**Decoder**: $y_t = g_V(y_{t-1}, h_{t-1}, c_t)$
New context vector at every time step

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:} = softmax(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



CNN

Extract spatial features from a pretrained CNN

$z_{0,0}$ $z_{0,1}$ $z_{0,2}$
$z_{1,0}$ $z_{1,1}$ $z_{1,2}$
$z_{2,0}$ $z_{2,1}$ $z_{2,2}$

Features: H x W x D

$h_0$ $h_1$ $h_2$ $h_3$ $h_4$

$y_1$ $y_2$ $y_3$ $y_4$

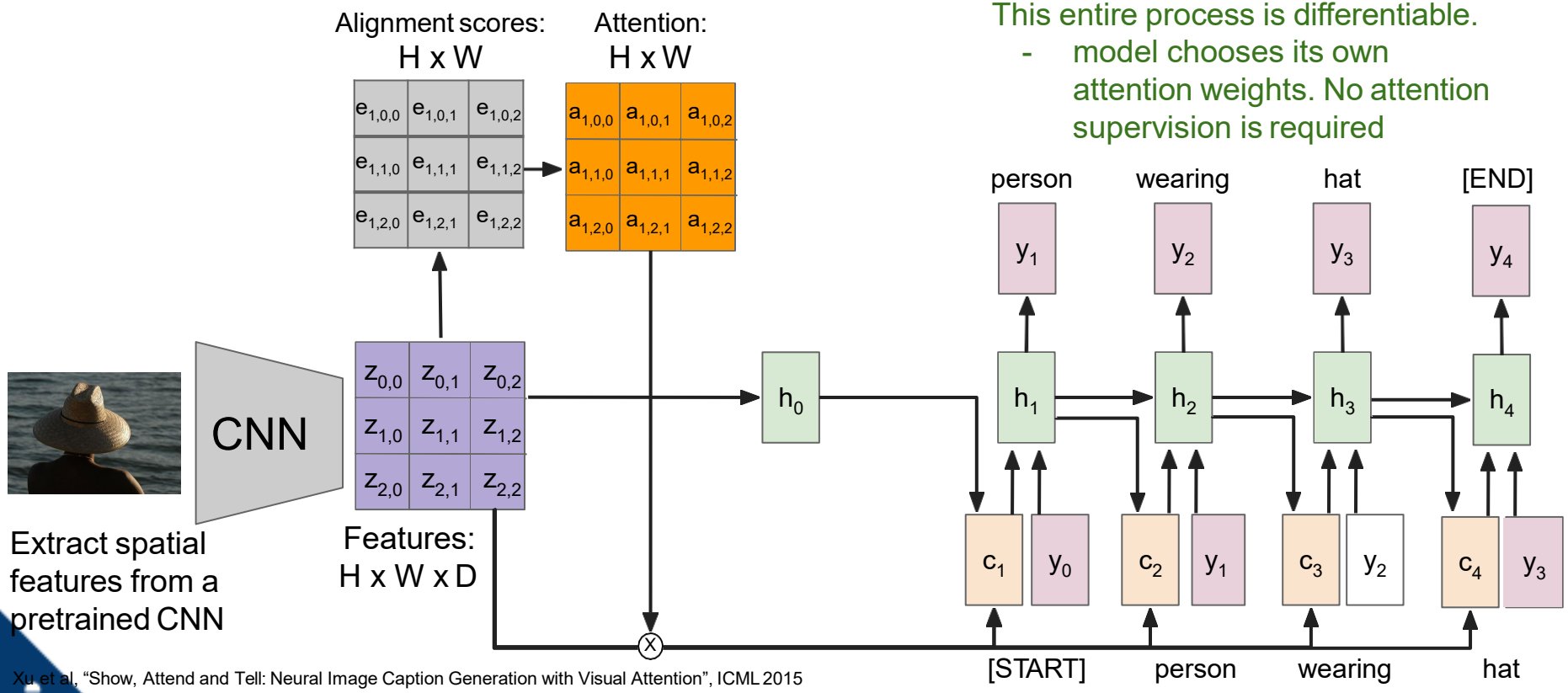
person wearing hat [END]

$c_1$ $y_0$ $c_2$ $y_1$ $c_3$ $y_2$ $c_4$ $y_3$

[START] person wearing hat

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with RNNs & Attention




This entire process is differentiable.

- model chooses its own attention weights. No attention supervision is required

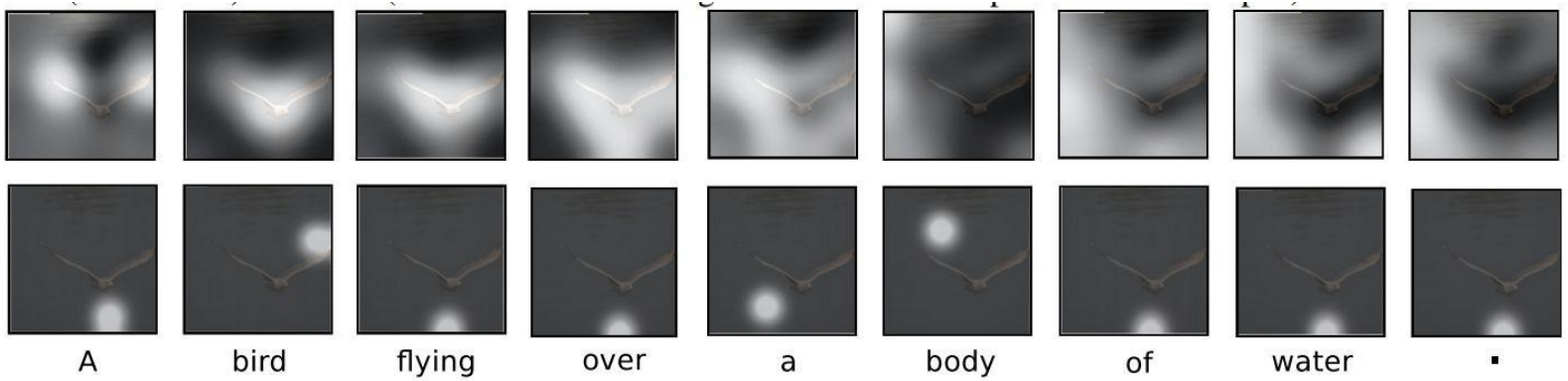
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

# Image Captioning with Attention

Soft attention



Hard attention
(requires reinforcement learning)




Xu et al, “Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention”, ICML 2015

Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard S. Zemel, and Yoshua Benchio, 2015. Reproduced with permission.



# Image Captioning with Attention



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.




A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

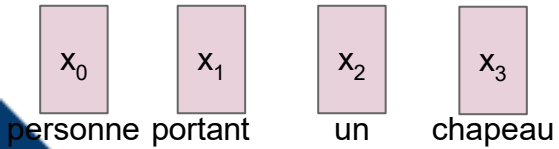
Xu et al, "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard S. Zemel, and Yoshua Benchio, 2015. Reproduced with permission.



# Similar tasks in NLP - Language translation example

**Input**: Sequence **x** = $x_1, x_2, \ldots, x_T$
**Output:** Sequence **y** = $y_1, y_2, \ldots, y_T$

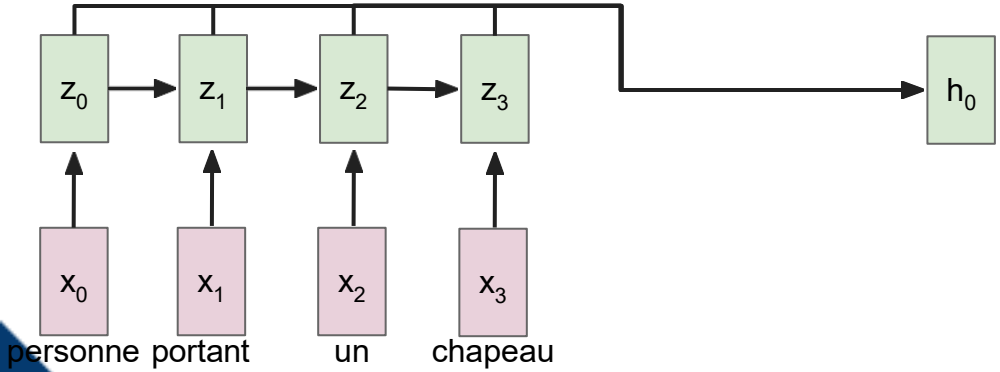





# Similar tasks in NLP - Language translation example

**Input**: Sequence **x** = $x_1, x_2, ..., x_T$
**Output:** Sequence **y** = $y_1, y_2, ..., y_T$

**Encoder**: $h_0 = f_{\mathbf{W}}(\mathbf{z})$
where $z_t = RNN(x_t, u_{t-1})$
$f_{\mathbf{W}}(.)$ is MLP
u is the hidden RNN state

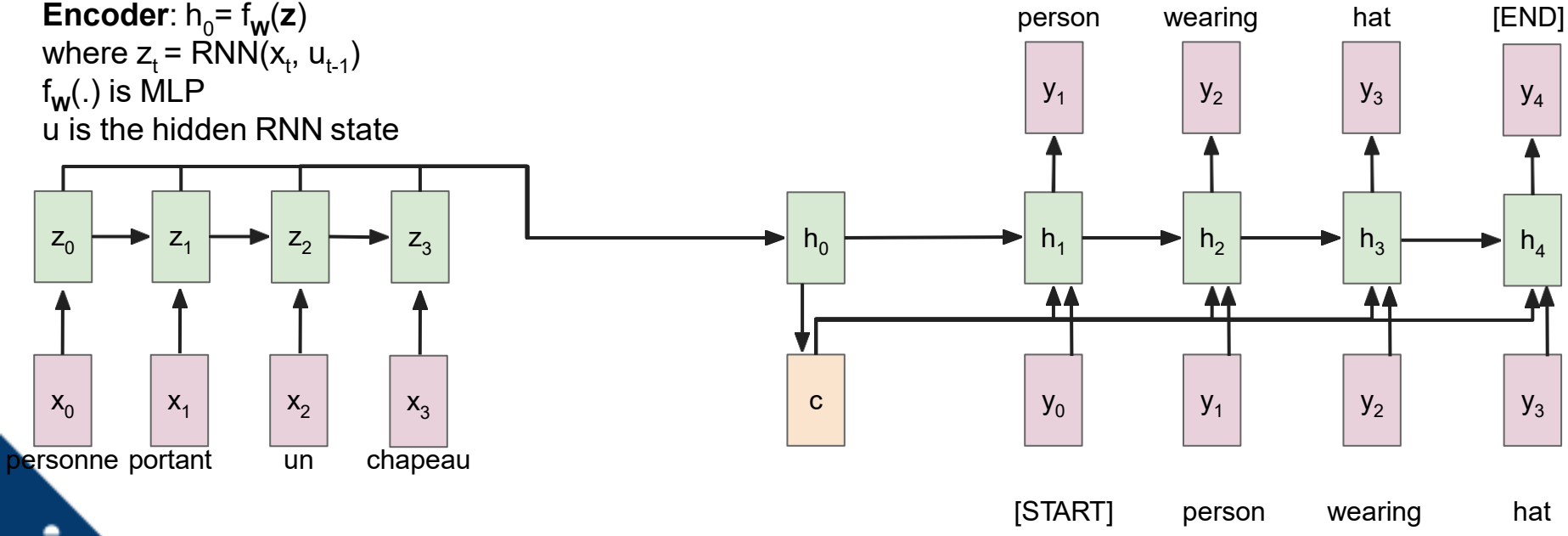


# Similar tasks in NLP - Language translation example

**Input**: Sequence **x** = $x_1, x_2, \ldots, x_T$
**Output:** Sequence **y** = $y_1, y_2, \ldots, y_T$

**Decoder**: $y_t = g_{\mathbf{V}}(y_{t-1}, h_{t-1}, c)$
where context vector c is often $c = h_0$

**Encoder**: $h_0 = f_{\mathbf{W}}(\mathbf{z})$
where $z_t = \text{RNN}(x_t, u_{t-1})$
$f_{\mathbf{W}}(.)$ is MLP
u is the hidden RNN state

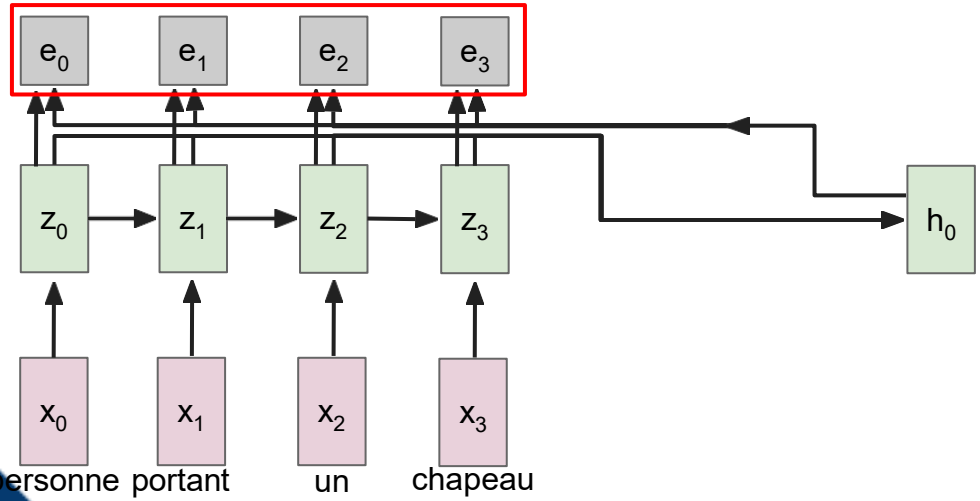


# Attention in NLP - Language translation example

Compute alignments scores (scalars):

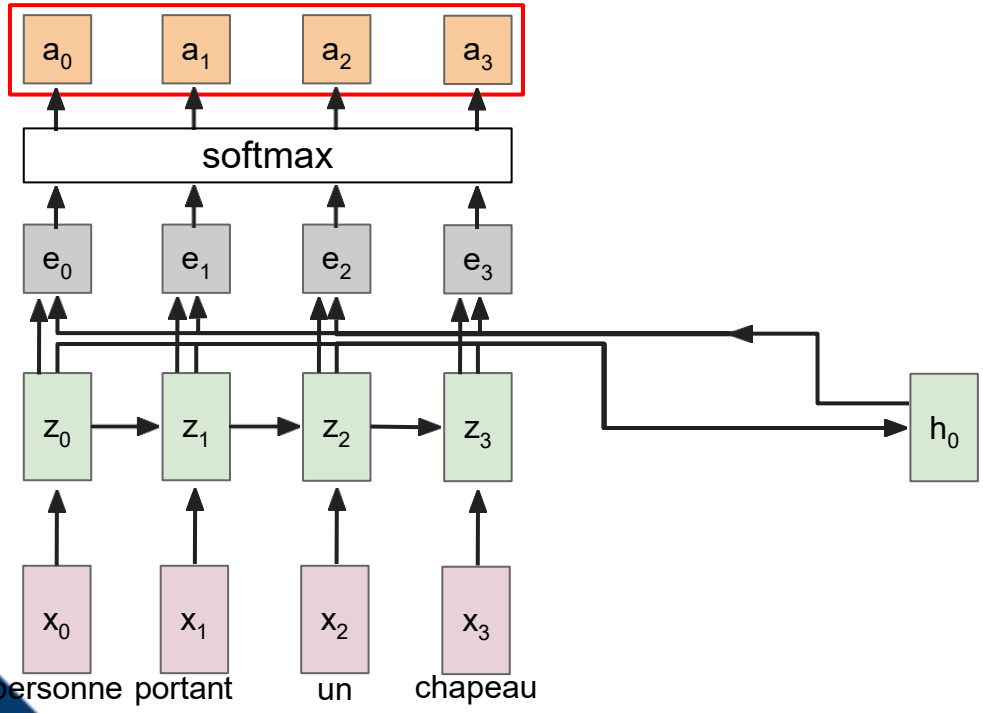
$$e_{t,i} = f_{att}(h_{t-1}, z_i)$$

$f_{att}(.)$ is an MLP




Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015

# Attention in NLP - Language translation example




Compute alignments scores (scalars):

$$e_{t,i} = f_{att}(h_{t-1}, z_i)$$

$f_{att}(.)$ is an MLP

Normalize to get attention weights:

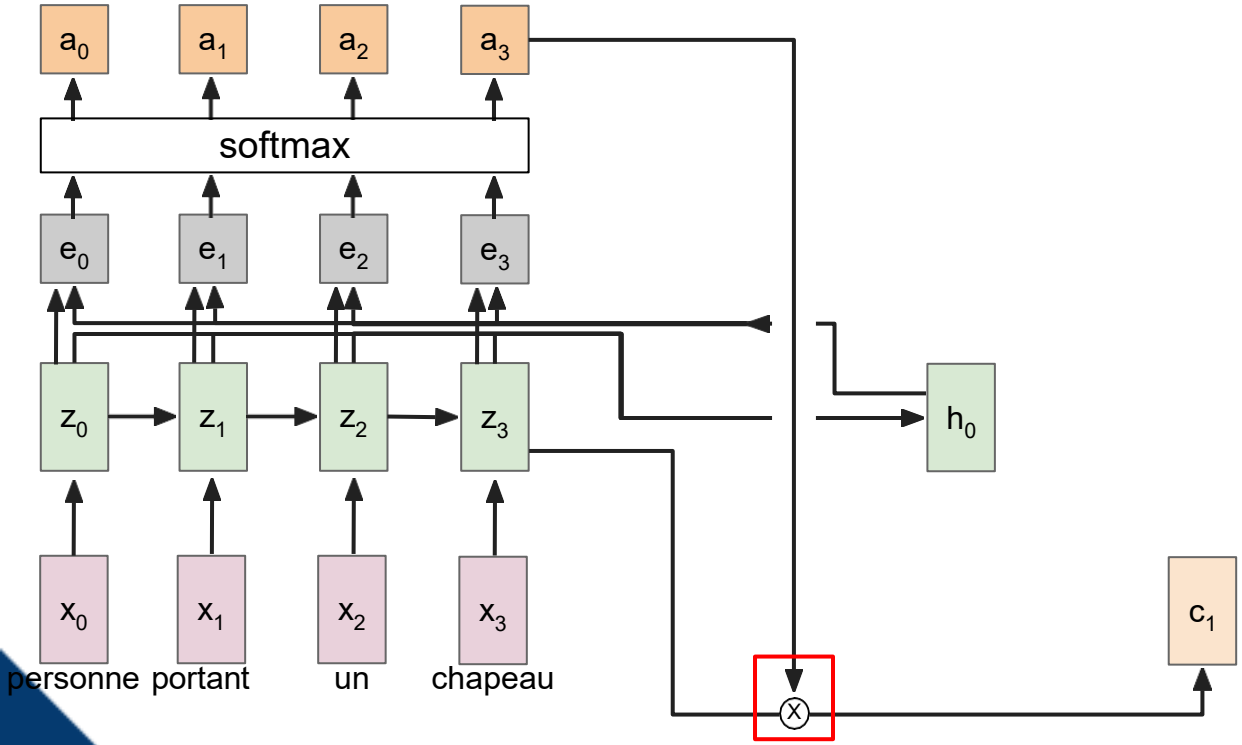
$$a_{t,:} = softmax(e_{t,:})$$

$0 < a_{t,i,j} < 1$,
attention values sum to 1

Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015



# Attention in NLP - Language translation example




Compute alignments scores (scalars):

$$e_{t,i} = f_{att}(h_{t-1}, z_i)$$

$f_{att}(.)$ is an MLP

Normalize to get attention weights:

$$a_{t,:} = softmax(e_{t,:})$$

$0 < a_{t,i,j} < 1$,
attention values sum to 1

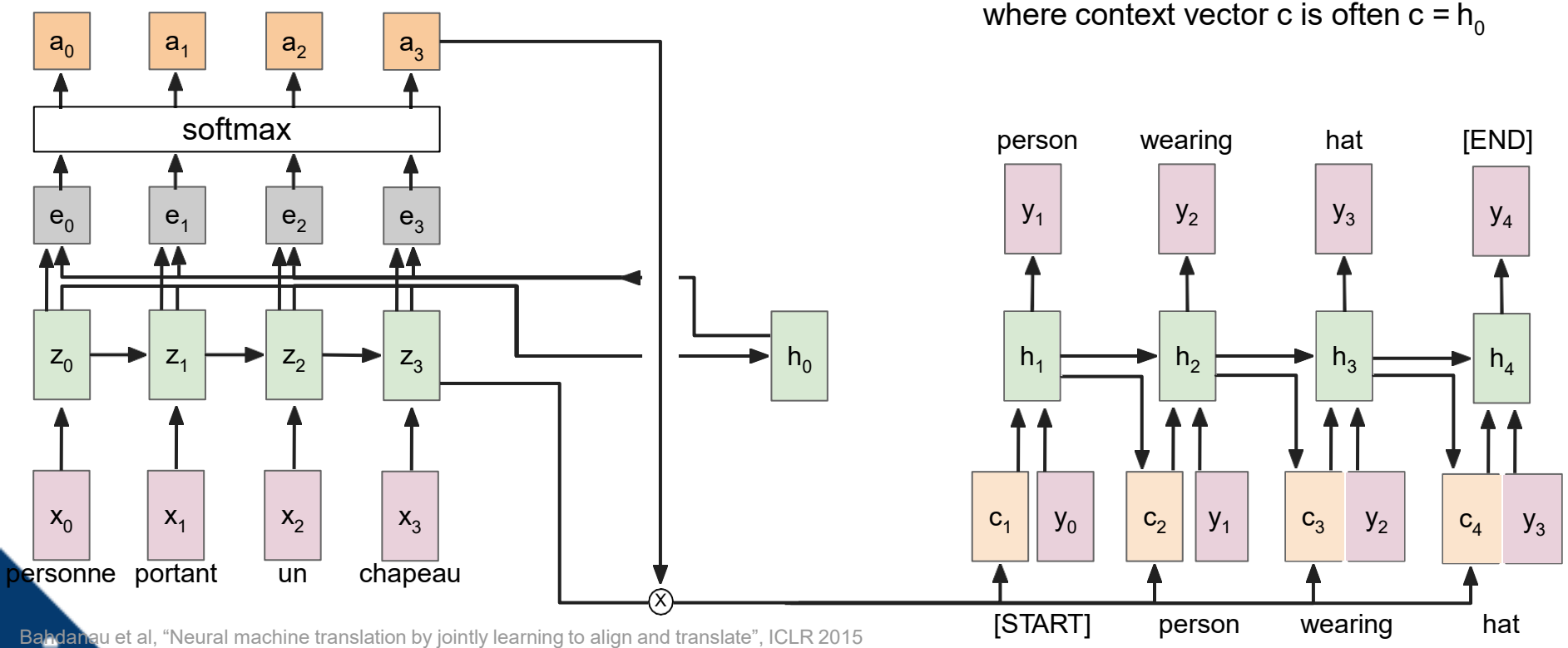
Compute context vector:

$$c_t = \sum_i a_{t,i} z_{t,i}$$

Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015

# Attention in NLP - Language translation example

**Decoder**: $y_t = g_V(y_{t-1}, h_{t-1}, c)$
where context vector c is often $c = h_0$




Bahdanau et al, “Neural machine translation by jointly learning to align and translate”, ICLR 2015

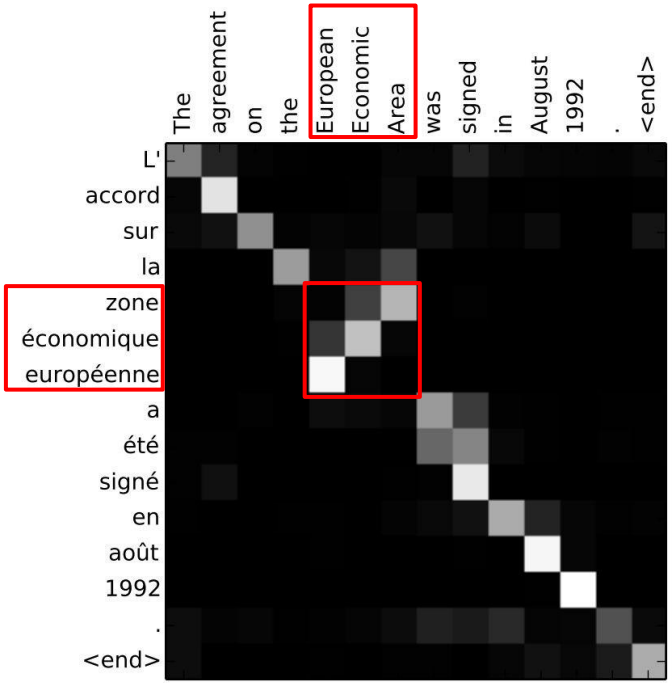
# Similar visualization of attention weights

English to French translation example:

**Input**: "The agreement on the European Economic Area was signed in August 1992."

**Output**: "L'accord sur la zone économique européenne a été signé en août 1992."

Without any attention supervision, model learns different word orderings for different languages



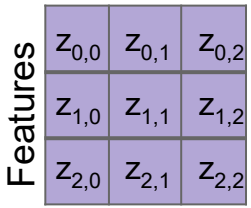

Bahdanau et al, "Neural machine translation by jointly learning to align and translate", ICLR 2015

# Today's Agenda:

- **Attention with RNNs**
  - In Computer Vision
  - In NLP
- **General Attention Layer**
  - Self-attention
  - Positional encoding
  - Masked attention
  - Multi-head attention
- **Transformers**



# Attention we just saw in image captioning

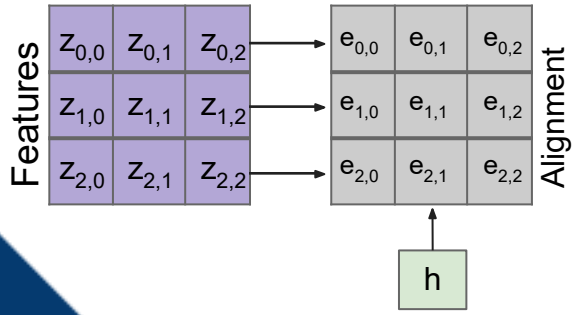



h

**Inputs**:
Features: **z** (shape: H x W x D)
Query: **h** (shape: D)

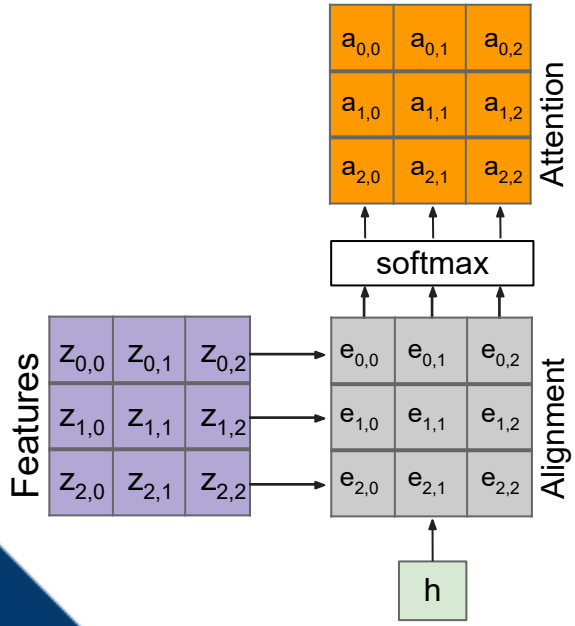
# Attention we just saw in image captioning

**Operations:**
Alignment: $e_{i,j} = f_{att}(h, z_{i,j})$




**Inputs**:
Features: **z** (shape: H x W x D)
Query: **h** (shape: D)

# Attention we just saw in image captioning

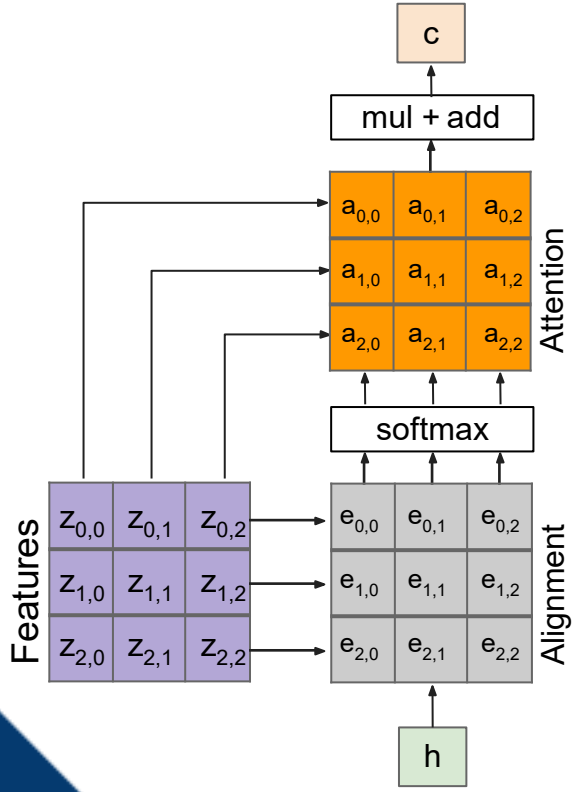



**Operations:**
Alignment: $e_{i,j} = f_{att}(h, z_{i,j})$
Attention: **a** = softmax(**e**)

**Inputs**:
Features: **z** (shape: H x W x D)
Query: **h** (shape: D)



# Attention we just saw in image captioning




**Outputs:**
context vector: **c** (shape: D)

**Operations:**
Alignment: $e_{i,j} = f_{att}(h, z_{i,j})$
Attention: **a** = softmax(**e**)
Output: **c** = $\sum_{i,j} a_{i,j} z_{i,j}$

**Inputs**:
Features: **z** (shape: H x W x D)
Query: **h** (shape: D)



Generalize

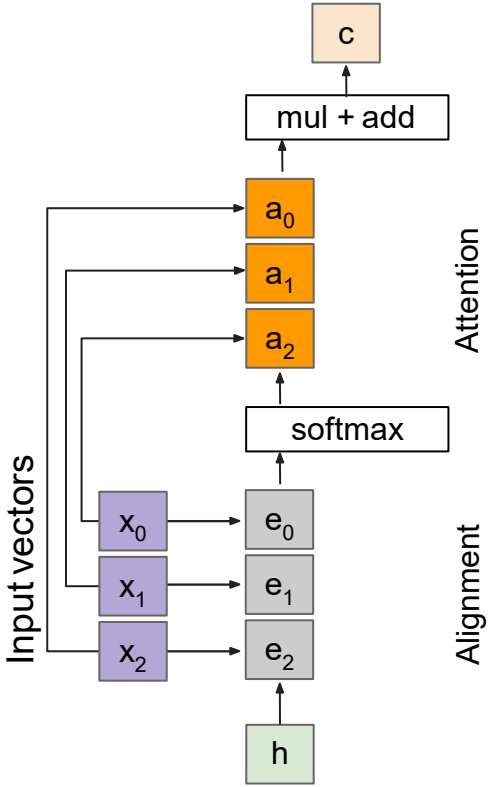
Attention in image captioning

to

General attention layer



# General attention layer




**Outputs:**
context vector: **c** (shape: D)

**Operations:**
Alignment: $e_i = f_{att}(h, x_i)$
Attention: **a** = softmax(**e**)
Output: $\mathbf{c} = \sum_i a_i x_i$

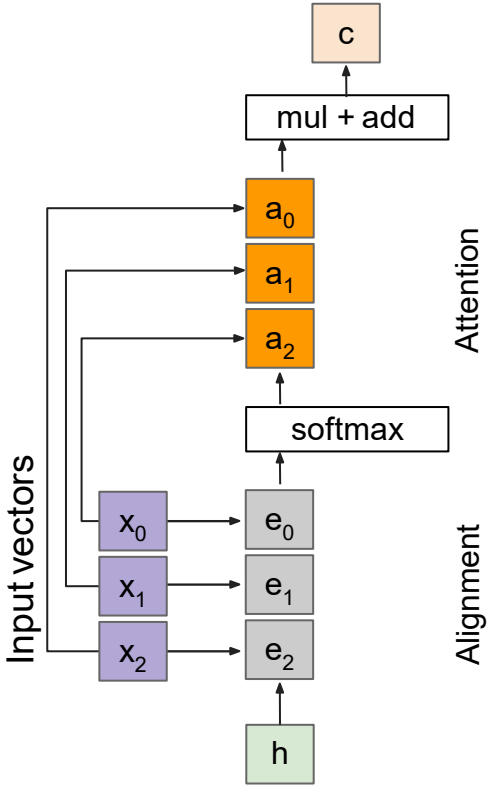
**Inputs**:
Input vectors: **x** (shape: N x D)
Query: **h** (shape: D)

Attention operation is **permutation invariant.**

- Doesn't care about ordering of the features
- Stretch H x W = N into N vectors



# General attention layer




**Outputs:**
context vector: **c** (shape: D)

**Operations:**
Alignment: $e_i = h \cdot x_i$
Attention: **a** = softmax(**e**)
Output: **c** = $\sum_i a_i x_i$

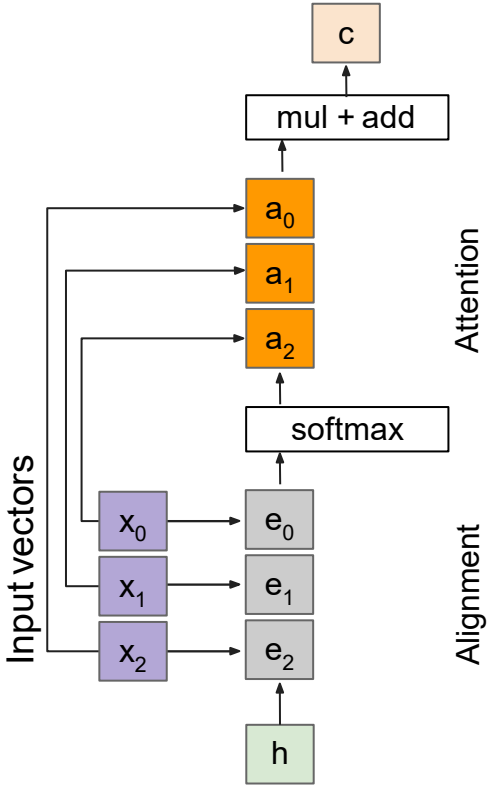
Change $f_{att}(.)$ to a simple dot product

- only works well with key & value transformation trick (will mention in a few slides)

**Inputs**:
Input vectors: **x** (shape: N x D)
Query: **h** (shape: D)



# General attention layer




**Outputs:**
context vector: **c** (shape: D)

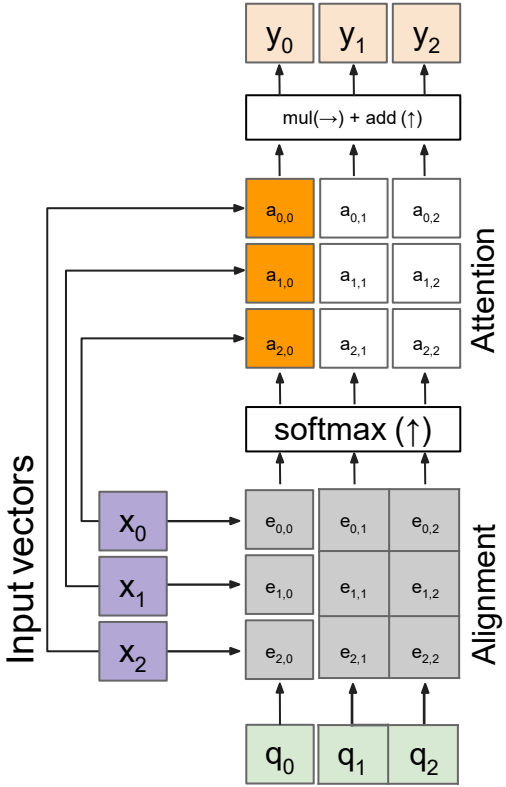
**Operations:**
Alignment: $e_i = h \cdot x_i / \sqrt{D}$
Attention: **a** = softmax(**e**)
Output: **c** = $\sum_i a_i x_i$

Change $f_{att}(.)$ to a scaled simple dot product

**Inputs**:
Input vectors: **x** (shape: N x D)
Query: **h** (shape: D)



# General attention layer




**Outputs:**
context vectors: **y** (shape: D)

Multiple query vectors
- each query creates a new output context vector

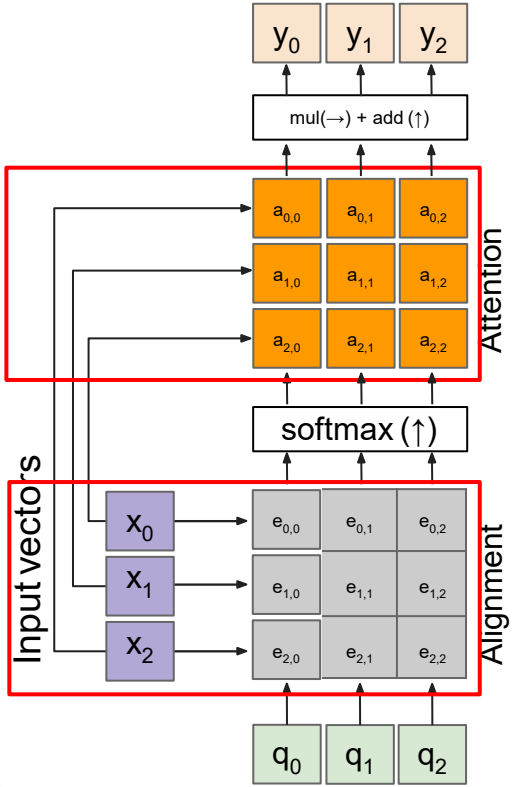
**Operations:**
Alignment: $e_{i,j} = q_j \cdot x_i / \sqrt{D}$
Attention: **a** = softmax(**e**)
Output: $y_j = \sum_i a_{i,j} x_i$

Multiple query vectors

**Inputs**:
Input vectors: **x** (shape: N x D)
Queries: **q** (shape: M x D)



# General attention layer




**Outputs:**
context vectors: **y** (shape: D)

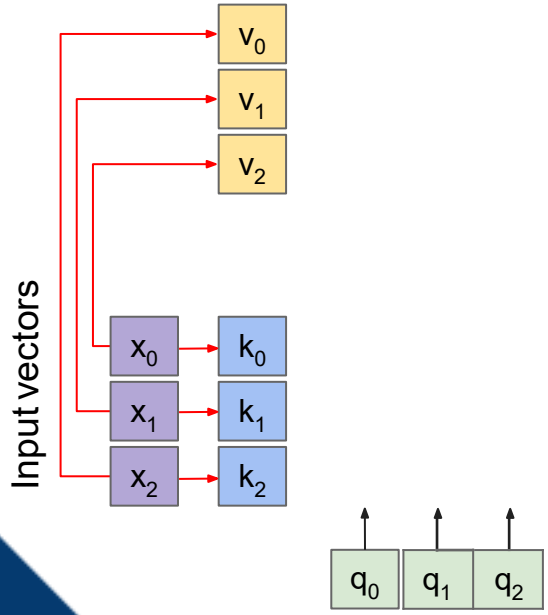
**Operations:**
Alignment: $e_{i,j} = q_j \cdot x_i / \sqrt{D}$
Attention: **a** = softmax(**e**)
Output: $y_j = \sum_i a_{i,j} x_i$

**Inputs**:
Input vectors: **x** (shape: N x D)
Queries: **q** (shape: M x D)

Notice that the input vectors are used for both the alignment as well as the attention calculations.

- We can add more expressivity to the layer by adding a different FC layer before each of the two steps.

# General attention layer



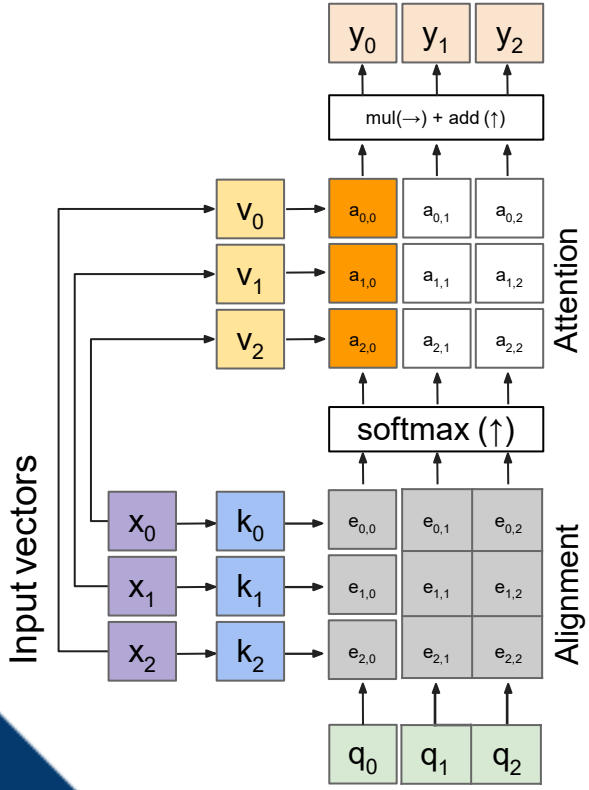

**Operations:**
Key vectors: $\mathbf{k} = \mathbf{x}W_k$
Value vectors: $\mathbf{v} = \mathbf{x}W_v$

**Inputs**:
Input vectors: **x** (shape: N x D)
Queries: **q** (shape: M x $D_k$)

Notice that the input vectors are used for both the alignment as well as the attention calculations.

- We can add more expressivity to the layer by adding a different FC layer before each of the two steps.

# General attention layer




**Outputs:**
context vectors: **y** (shape: $D_v$)

**Operations:**
Key vectors: $\mathbf{k} = \mathbf{x}W_k$
Value vectors: $\mathbf{v} = \mathbf{x}W_v$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

**Inputs**:
Input vectors: **x** (shape: N x D)
Queries: **q** (shape: M x $D_k$)

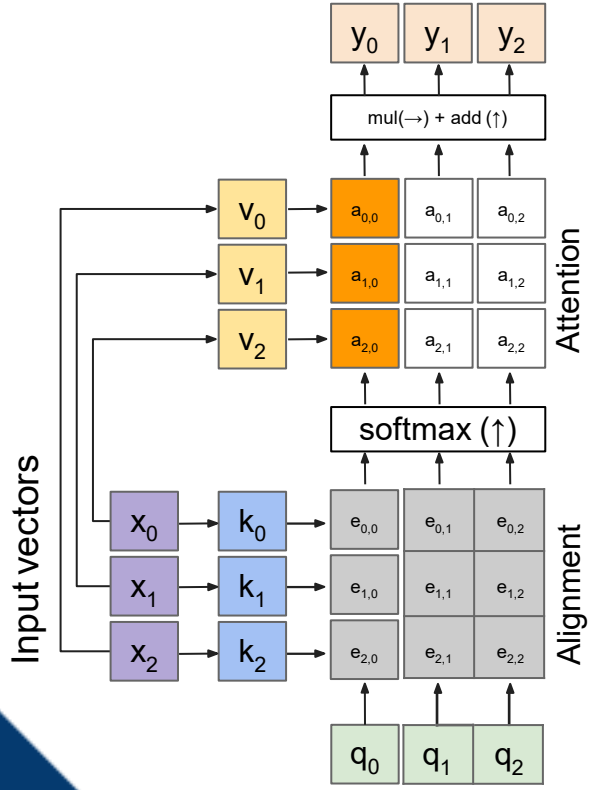
The input and output dimensions can now change depending on the key and value FC layers

Notice that the input vectors are used for both the alignment as well as the attention calculations.

- We can add more expressivity to the layer by adding a different FC layer before each of the two steps.



# General attention layer



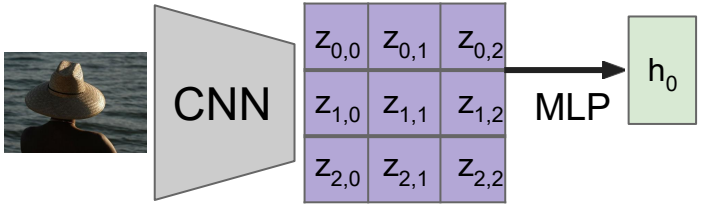

**Outputs:**
context vectors: **y** (shape: $D_v$)

**Operations:**
Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$
Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

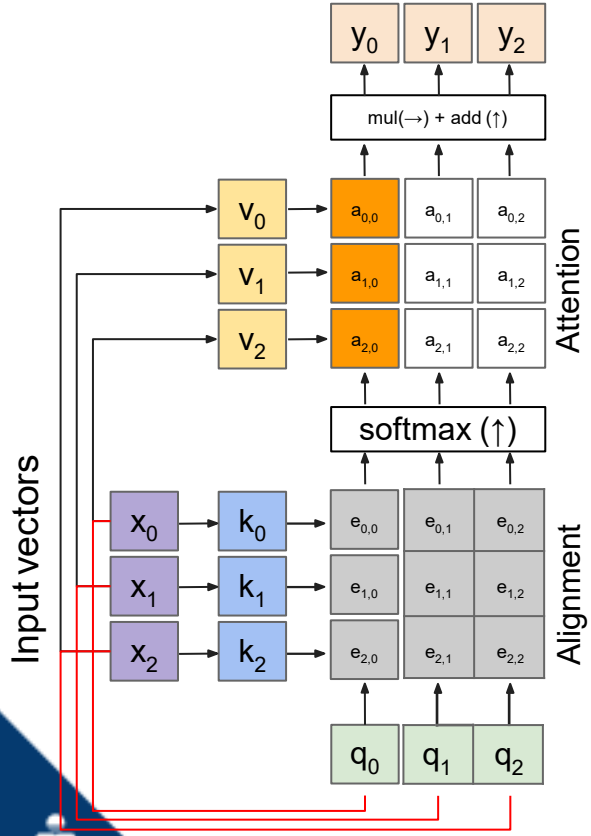
**Inputs**:
Input vectors: **x** (shape: N x D)
Queries: **q** (shape: M x $D_k$)

Recall that the query vector was a function of the input vectors

**Encoder**: $h_0 = f_W(\mathbf{z})$
where **z** is spatial CNN features
$f_W(.)$ is an MLP



# General attention layer ➡ Self-attention layer



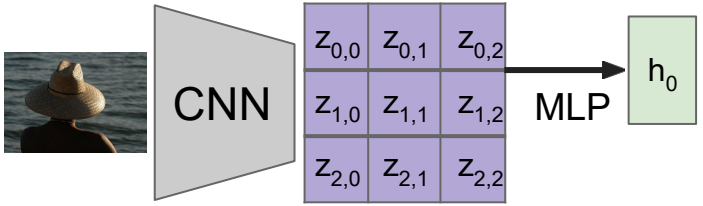

**Outputs:**
context vectors: **y** (shape: $D_v$)

**Operations:**
Key vectors: $\mathbf{k} = \mathbf{x}W_k$
Value vectors: $\mathbf{v} = \mathbf{x}W_v$
Query vectors: $\mathbf{q} = \mathbf{x}W_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

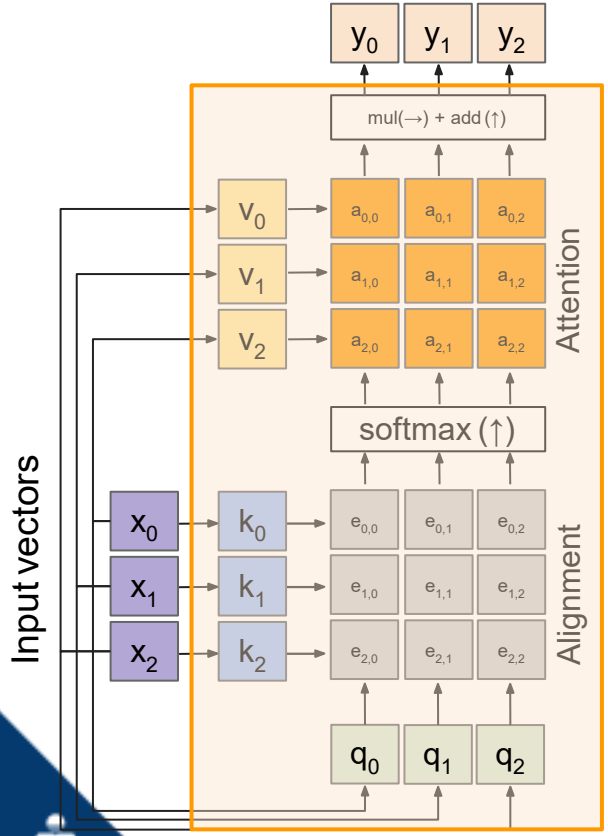
**Inputs**:
Input vectors: **x** (shape: N x D)
~~Queries: **q** (shape: M x $D_k$)~~

Recall that the query vector was a function of the input vectors

**Encoder**: $h_0 = f_W(\mathbf{z})$
where **z** is spatial CNN features
$f_W(.)$ is an MLP



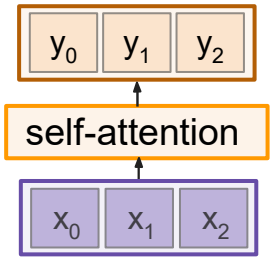
# Self attention layer - attends over sets of inputs




**Outputs:**
context vectors: **y** (shape: $D_v$)

**Operations:**
Key vectors: $\mathbf{k} = \mathbf{x}W_k$
Value vectors: $\mathbf{v} = \mathbf{x}W_v$
Query vectors: $\mathbf{q} = \mathbf{x}W_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

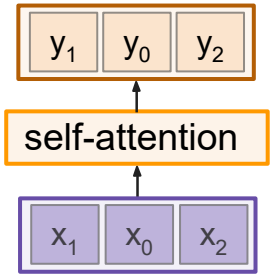
**Inputs**:
Input vectors: **x** (shape: N x D)

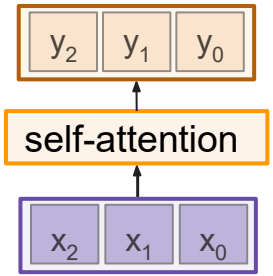


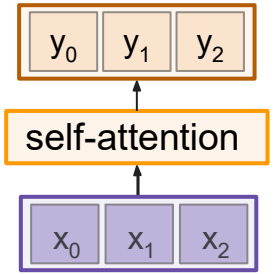
# Self attention layer - attends over sets of inputs

Permutation invariant

**Problem:** how can we encode ordered sequences like language or spatially ordered image features?

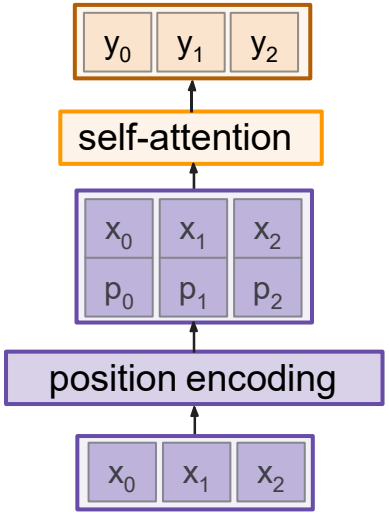





# Positional encoding




Concatenate special positional encoding $p_j$ to each input vector $x_j$

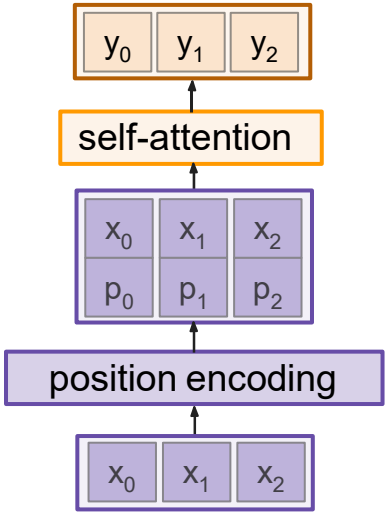
We use a function *pos*: $N \rightarrow R^d$ to process the position $j$ of the vector into a d-dimensional vector

So, $p_j = pos(j)$

Desiderata of *pos*(.) :

1. It should output a **unique** encoding for each time-step (word's position in a sentence)
2. **Distance** between any two time-steps should be consistent across sentences with different lengths.
3. Our model should generalize to **longer** sentences without any efforts. Its values should be bounded.
4. It must be **deterministic**.

# Positional encoding




Concatenate special positional encoding $p_j$ to each input vector $x_j$

We use a function *pos*: $N \rightarrow R^d$ to process the position $j$ of the vector into a d-dimensional vector

So, $p_j = pos(j)$

Options for *pos*(.)

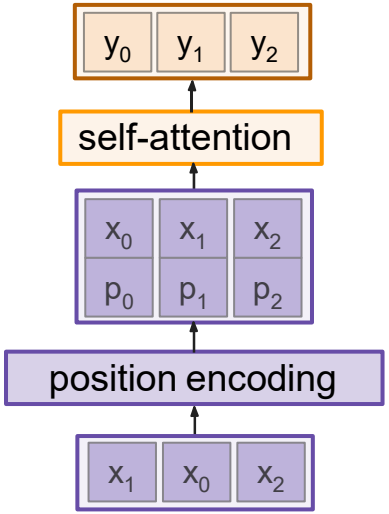
1. Learn a lookup table:
   - Learn parameters to use for *pos*(t) for $t \in [0, T)$
   - Lookup table contains T x d parameters.

Desiderata of *pos*(.) :

1. It should output a **unique** encoding for each time-step (word's position in a sentence)
2. **Distance** between any two time-steps should be consistent across sentences with different lengths.
3. Our model should generalize to **longer** sentences without any efforts. Its values should be bounded.
4. It must be **deterministic**.

Vaswani et al, "Attention is all you need", NeurIPS 2017

# Positional encoding




Concatenate special positional encoding $p_j$ to each input vector $x_j$

We use a function *pos*: N $\rightarrow R^d$ to process the position j of the vector into a d-dimensional vector

So, $p_j = pos(j)$

Options for *pos*(.)

1. Learn a lookup table:
   - Learn parameters to use for *pos*(t) for t ε [0, T)
   - Lookup table contains T x d parameters.

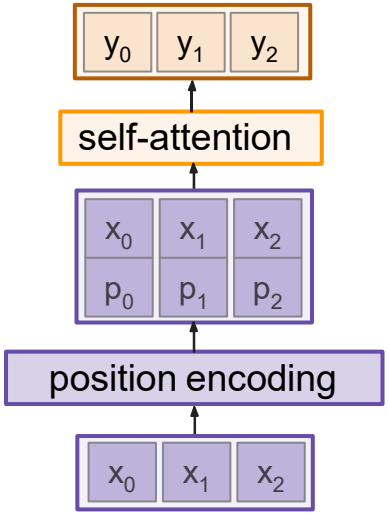
2. Design a fixed function with the desiderata
   -

$$\mathbf{p(t)} = \begin{bmatrix} \sin(\omega_1 . t) \\ \cos(\omega_1 . t) \\ \\ \sin(\omega_2 . t) \\ \cos(\omega_2 . t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} . t) \\ \cos(\omega_{d/2} . t) \end{bmatrix}_d \quad \text{where } \omega_k = \frac{1}{10000^{2k/d}}$$

Vaswani et al, "Attention is all you need", NeurIPS 2017



# Positional encoding




Concatenate special positional encoding $p_j$ to each input vector $x_j$

We use a function *pos*: N $\rightarrow R^d$ to process the position j of the vector into a d-dimensional vector

So, $p_j = pos(j)$

Options for *pos*(.)

1. Learn a lookup table:
   - Learn parameters to use for *pos*(t) for t ε [0, T)
   - Lookup table contains T x d parameters.

2. Design a fixed function with the desiderata
   -

$$\mathbf{p(t)} = \begin{bmatrix} \sin(\omega_1 . t) \\ \cos(\omega_1 . t) \\ \sin(\omega_2 . t) \\ \cos(\omega_2 . t) \\ \vdots \\ \sin(\omega_{d/2} . t) \\ \cos(\omega_{d/2} . t) \end{bmatrix}_d$$

Intuition:

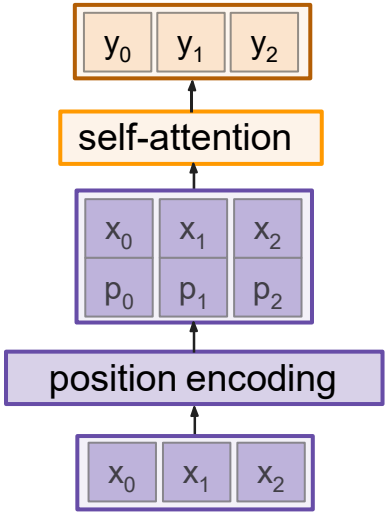
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 : | 0 | 0 | 0 | 0 | 8 : | 1 | 0 | 0 | 0 |
| 1 : | 0 | 0 | 0 | 1 | 9 : | 1 | 0 | 0 | 1 |
| 2 : | 0 | 0 | 1 | 0 | 10 : | 1 | 0 | 1 | 0 |
| 3 : | 0 | 0 | 1 | 1 | 11 : | 1 | 0 | 1 | 1 |
| 4 : | 0 | 1 | 0 | 0 | 12 : | 1 | 1 | 0 | 0 |
| 5 : | 0 | 1 | 0 | 1 | 13 : | 1 | 1 | 0 | 1 |
| 6 : | 0 | 1 | 1 | 0 | 14 : | 1 | 1 | 1 | 0 |
| 7 : | 0 | 1 | 1 | 1 | 15 : | 1 | 1 | 1 | 1 |

where $\omega_k = \frac{1}{10000^{2k/d}}$

Vaswani et al, "Attention is all you need", NeurIPS 2017



# Positional encoding




Concatenate special positional encoding $p_j$ to each input vector $x_j$

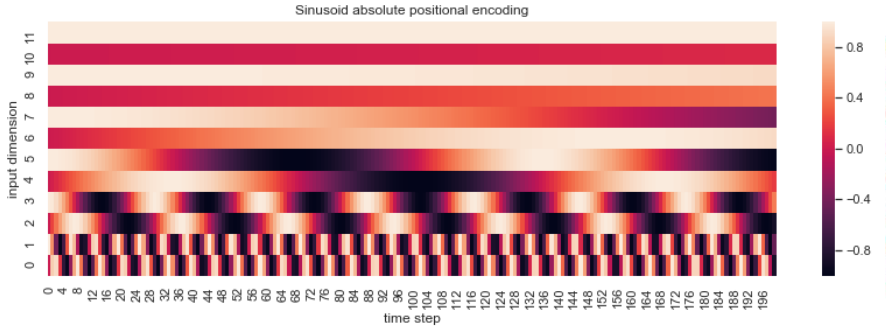
We use a function *pos*: N $\rightarrow R^d$ to process the position j of the vector into a d-dimensional vector

So, $p_j = pos(j)$

Options for *pos*(.)

1. Learn a lookup table:
   - Learn parameters to use for *pos*(t) for t ε [0, T)
   - Lookup table contains T x d parameters.

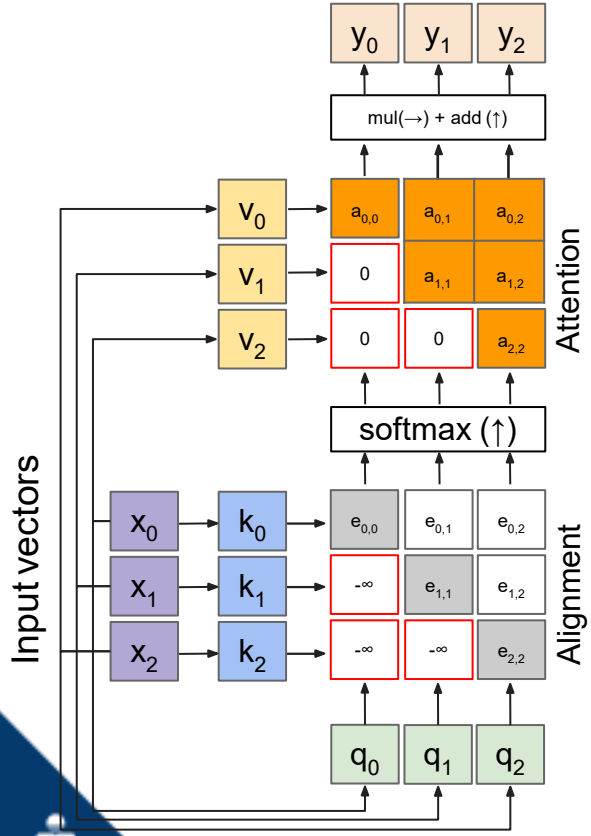
2. Design a fixed function




$$\begin{bmatrix} \sin(\omega_{d/2}.t) \\ \cos(\omega_{d/2}.t) \end{bmatrix}_d \quad \text{where} \quad \omega_k = \frac{1}{10000^{2k/d}}$$

Vaswani et al, "Attention is all you need", NeurIPS 2017

# Masked self-attention layer




**Outputs:**
context vectors: **y** (shape: $D_v$)

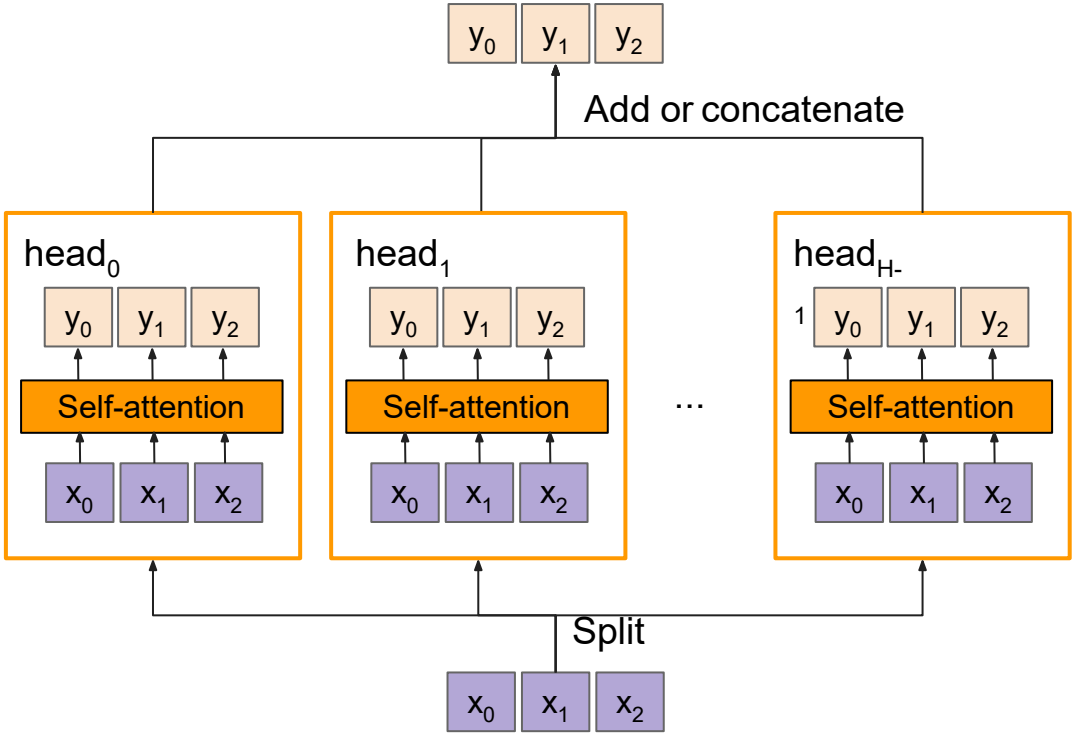
**Operations:**
Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$
Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$
Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

**Inputs**:
Input vectors: **x** (shape: N x D)

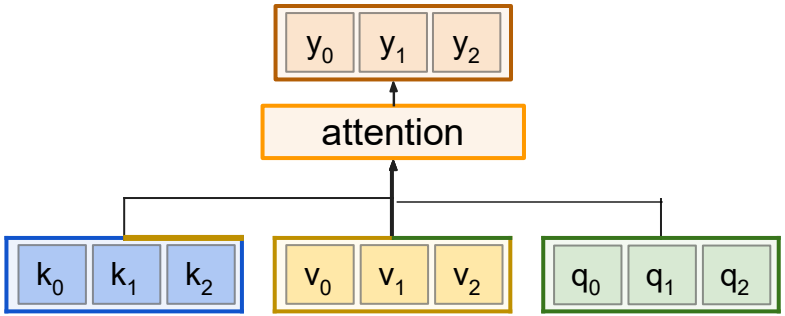
- Prevent vectors from looking at future vectors.
- Manually set alignment scores to -infinity

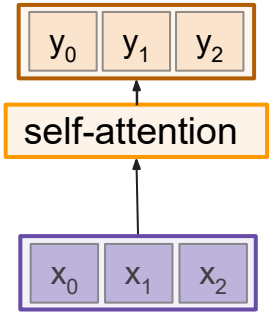
# Multi-head self attention layer

- Multiple self-attention heads in parallel



# General attention versus self-attention



# Comparing RNNs to Transformers

**RNNs**
(+) LSTMs work reasonably well for long sequences.
(-) Expects an ordered sequences of inputs
(-) Sequential computation: subsequent hidden states can only be computed after the previous ones are done.

**Transformers:**
(+) Good at long sequences. Each attention calculation looks at all inputs.
(+) Can operate over unordered sets or ordered sequences with positional encodings.
(+) Parallel computation: All alignment and attention scores for all inputs can be done in parallel.
(-) Requires a lot of memory: N x M alignment and attention scalers need to be calculated and stored for a single self-attention head. (but GPUs are getting bigger and better)



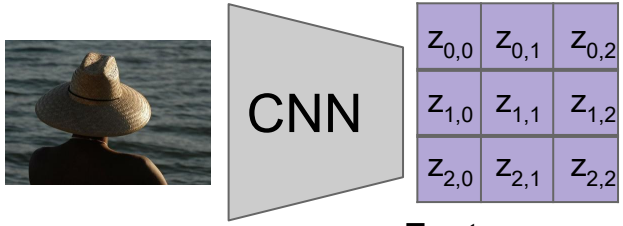
# Outline

- **Attention with RNNs**
  - In Computer Vision
  - In NLP
- **General Attention Layer**
  - Self-attention
  - Positional encoding
  - Masked attention
  - Multi-head attention
- **Transformers**



# Image Captioning using transformers

**Input**: Image **I**
**Output:** Sequence **y** = $y_1, y_2, ..., y_T$



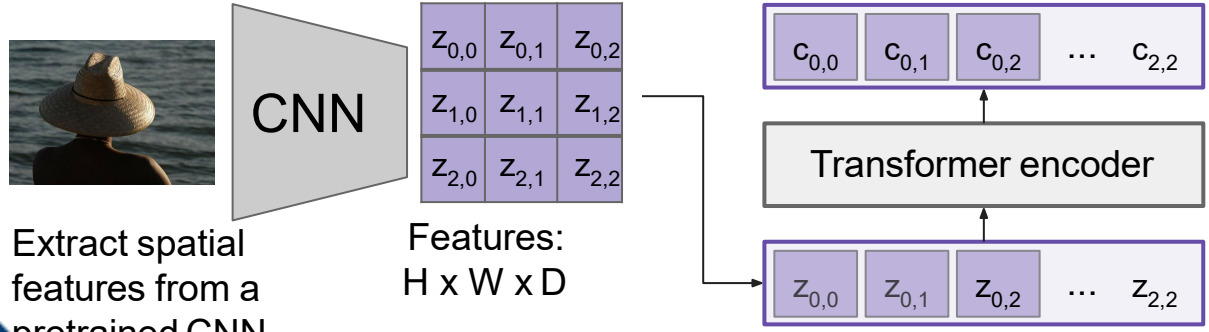

Extract spatial features from a pretrained CNN

Features: H x W x D

# Image Captioning using transformers

**Input**: Image **I**
**Output:** Sequence **y** = $y_1, y_2, ..., y_T$

**Encoder**: **c** = $T_W(\mathbf{z})$
where **z** is spatial CNN features
$T_W(.)$ is the transformer encoder




Extract spatial features from a pretrained CNN

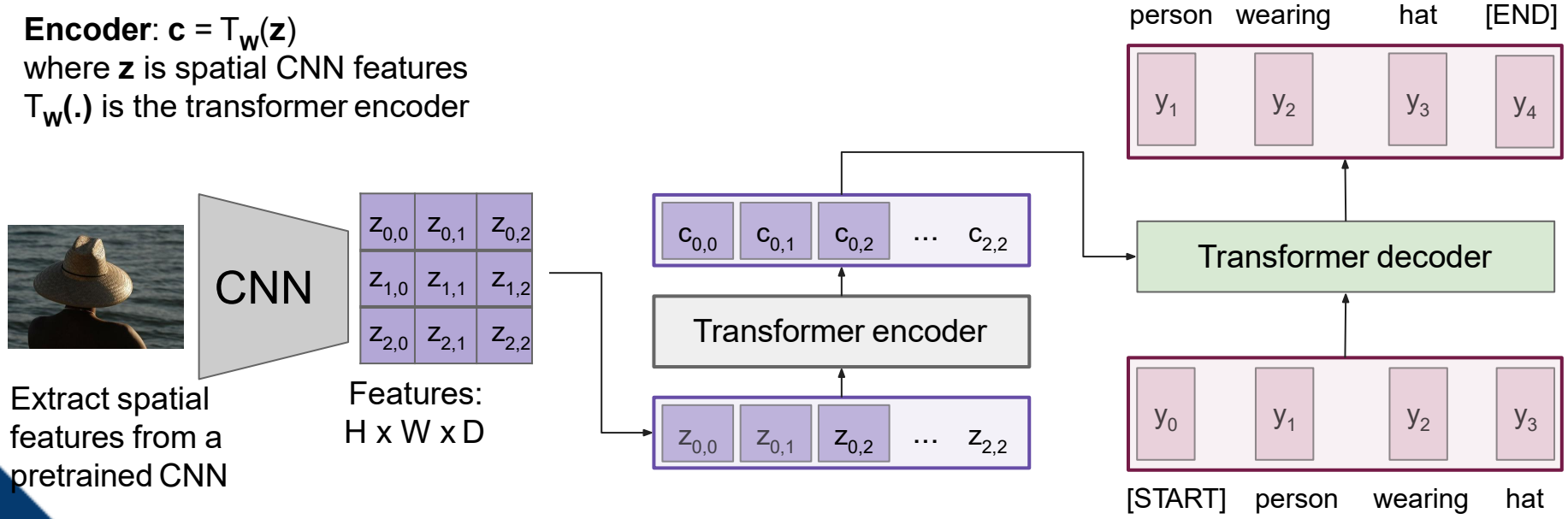
Features: H x W x D

# Image Captioning using transformers

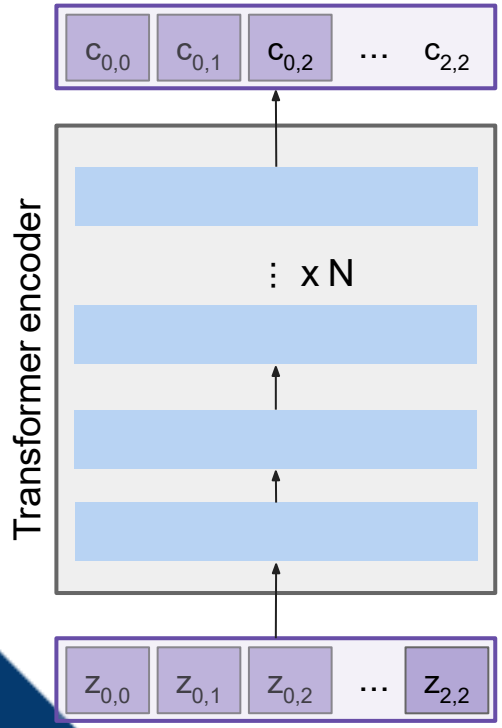
**Input**: Image **I**
**Output:** Sequence **y** = $y_1, y_2, ..., y_T$

**Decoder**: $y_t = T_D(\mathbf{y}_{0:t-1}, \mathbf{c})$
where $T_D(.)$ is the transformer decoder

**Encoder**: $\mathbf{c} = T_W(\mathbf{z})$
where **z** is spatial CNN features
$T_W(.)$ is the transformer encoder



# The Transformer encoder block

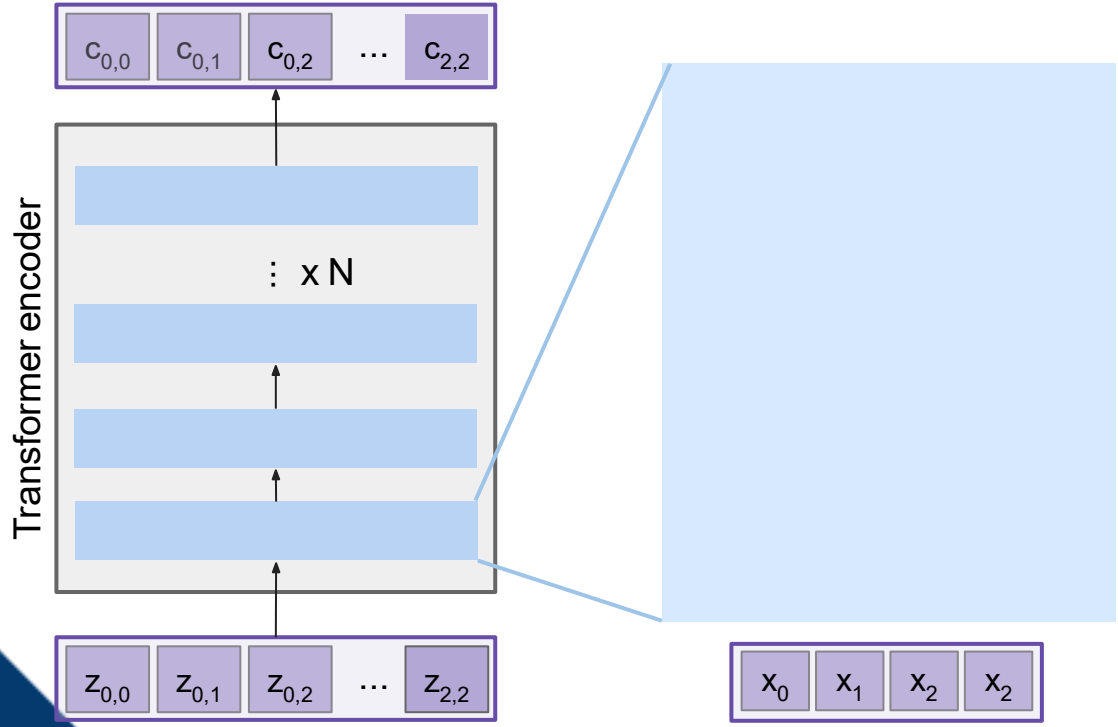



Made up of N encoder blocks.

In vaswani et al. N = 6, $D_q$= 512

Vaswani et al, “Attention is all you need”, NeurIPS 2017

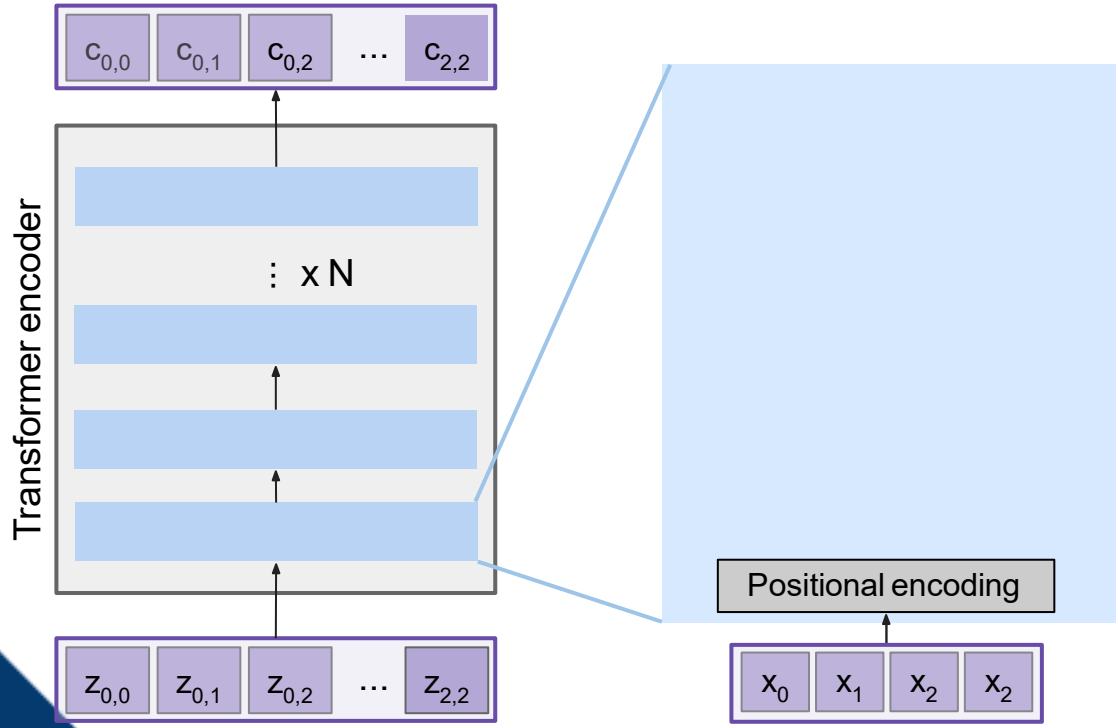
# The Transformer encoder block




Let's dive into one encoder block

Vaswani et al, “Attention is all you need”, NeurIPS 2017

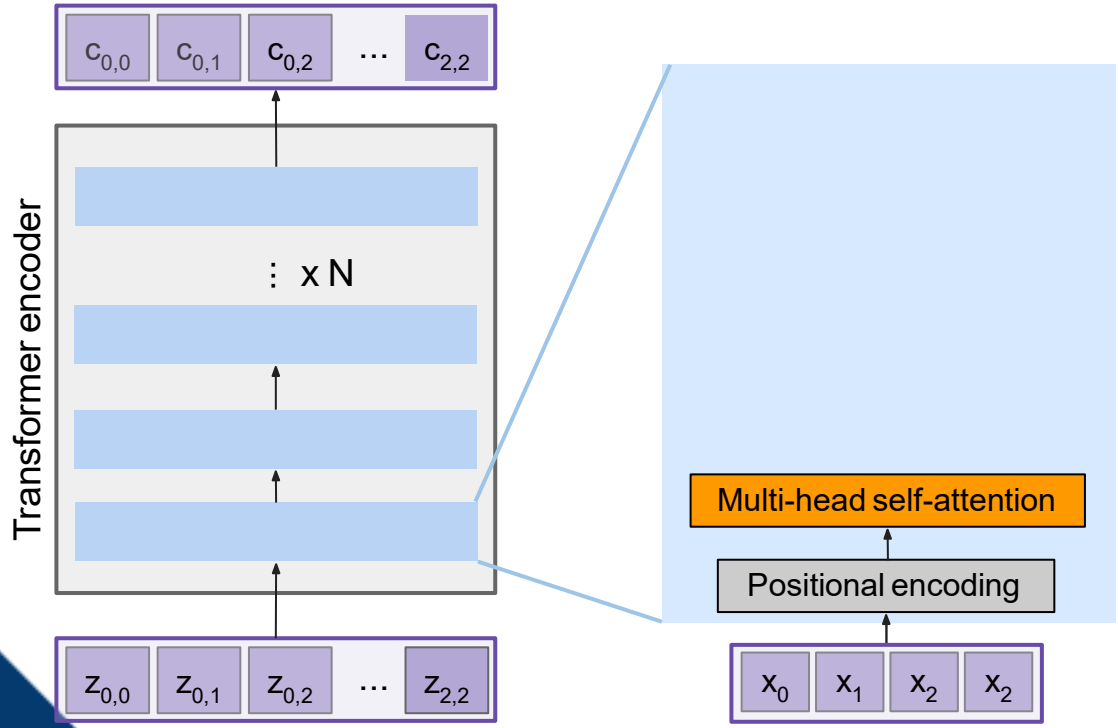
# The Transformer encoder block




Add positional encoding

Vaswani et al, “Attention is all you need”, NeurIPS 2017

# The Transformer encoder block

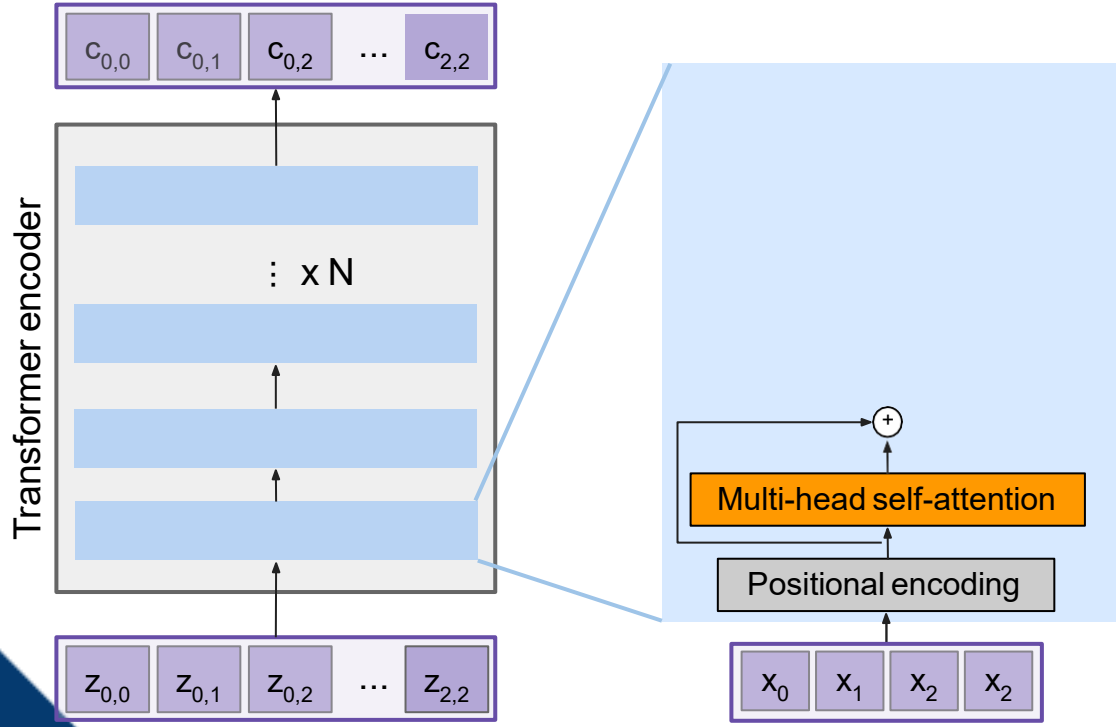



Attention attends over all the vectors

Add positional encoding

Vaswani et al, “Attention is all you need”, NeurIPS 2017

# The Transformer encoder block



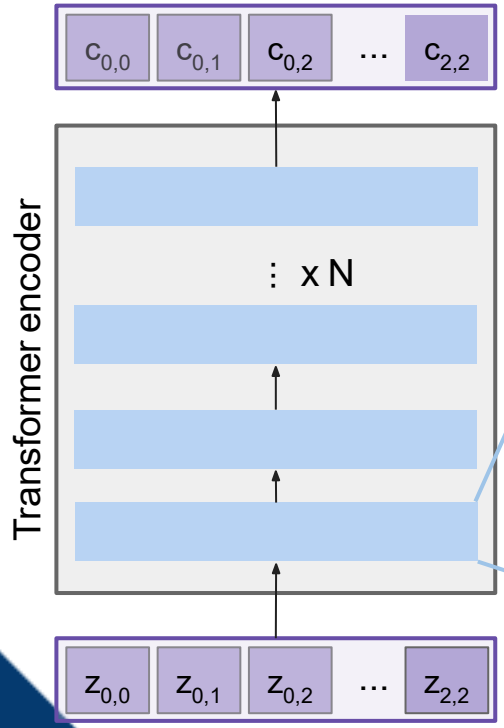

Residual connection

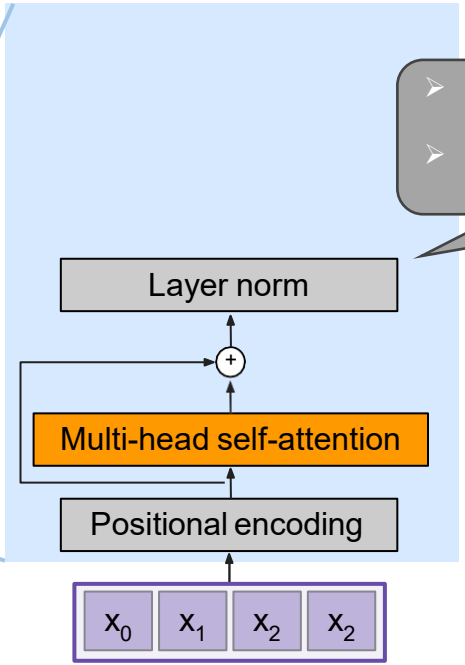
Attention attends over all the vectors

Add positional encoding

Vaswani et al, “Attention is all you need”, NeurIPS 2017

# The Transformer encoder block




- BatchNorm: normalize across batch (same feature across samples)
- LayerNorm: normalize across features (within one sample)

LayerNorm over each vector individually

Residual connection

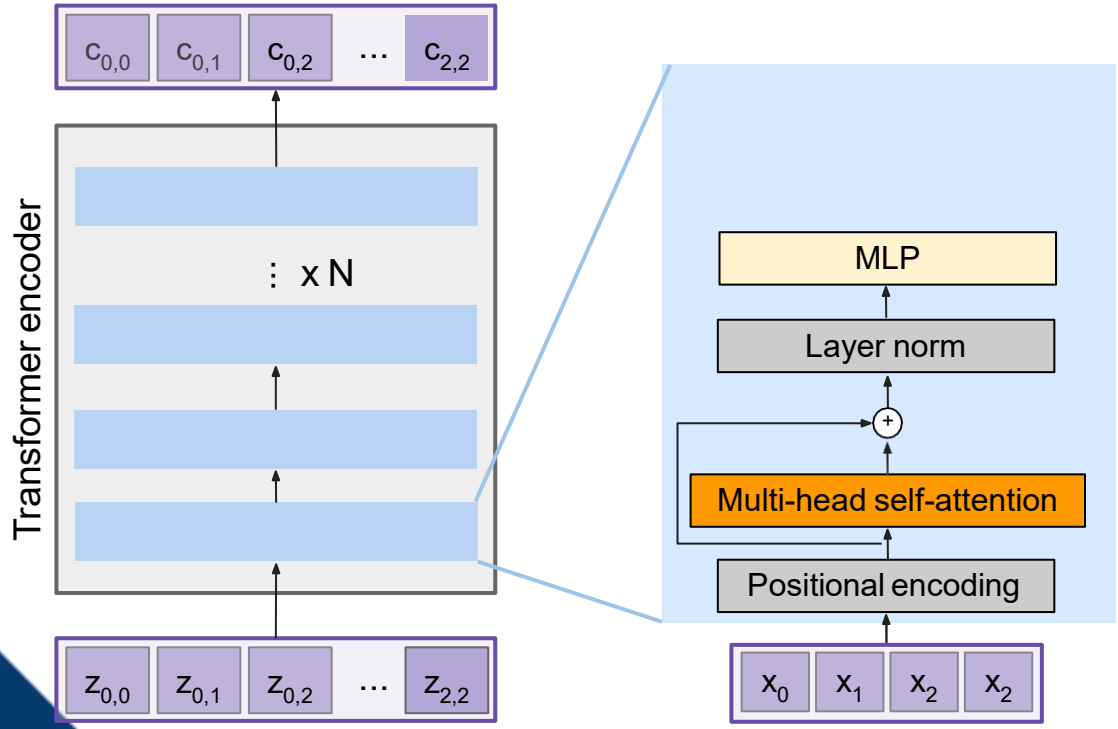
Attention attends over all the vectors

Add positional encoding

Vaswani et al, “Attention is all you need”, NeurIPS 2017



# The Transformer encoder block




MLP over each vector individually

LayerNorm over each vector individually

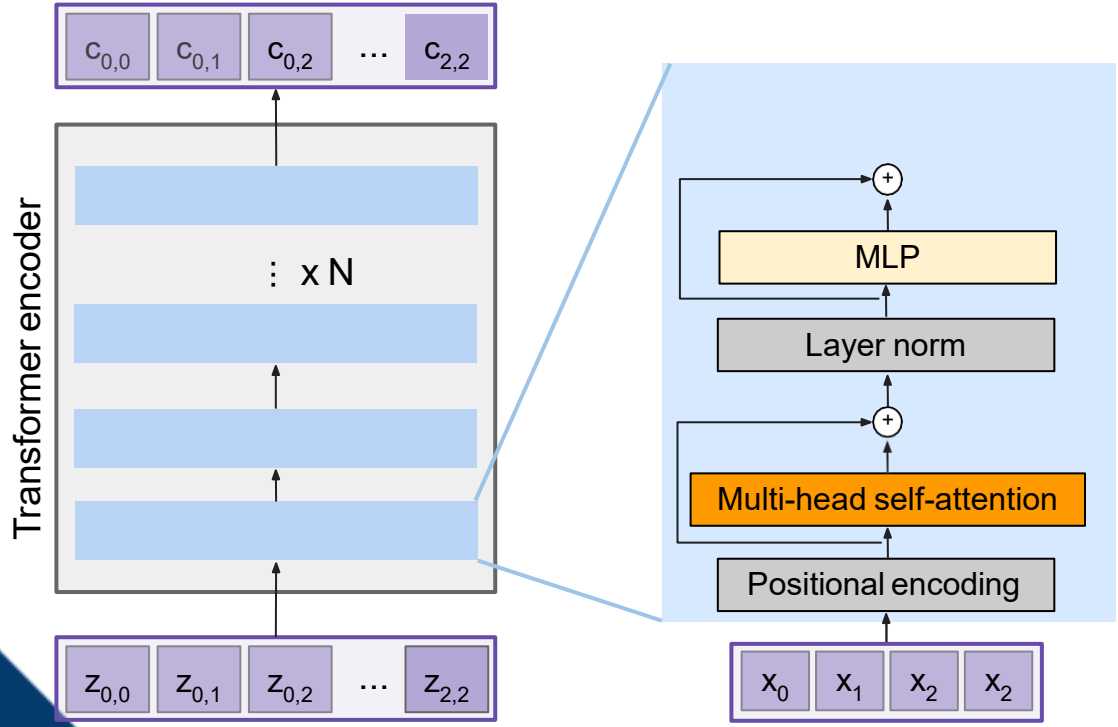
Residual connection

Attention attends over all the vectors

Add positional encoding

Vaswani et al, “Attention is all you need”, NeurIPS 2017

# The Transformer encoder block




Residual connection

MLP over each vector individually

LayerNorm over each vector individually

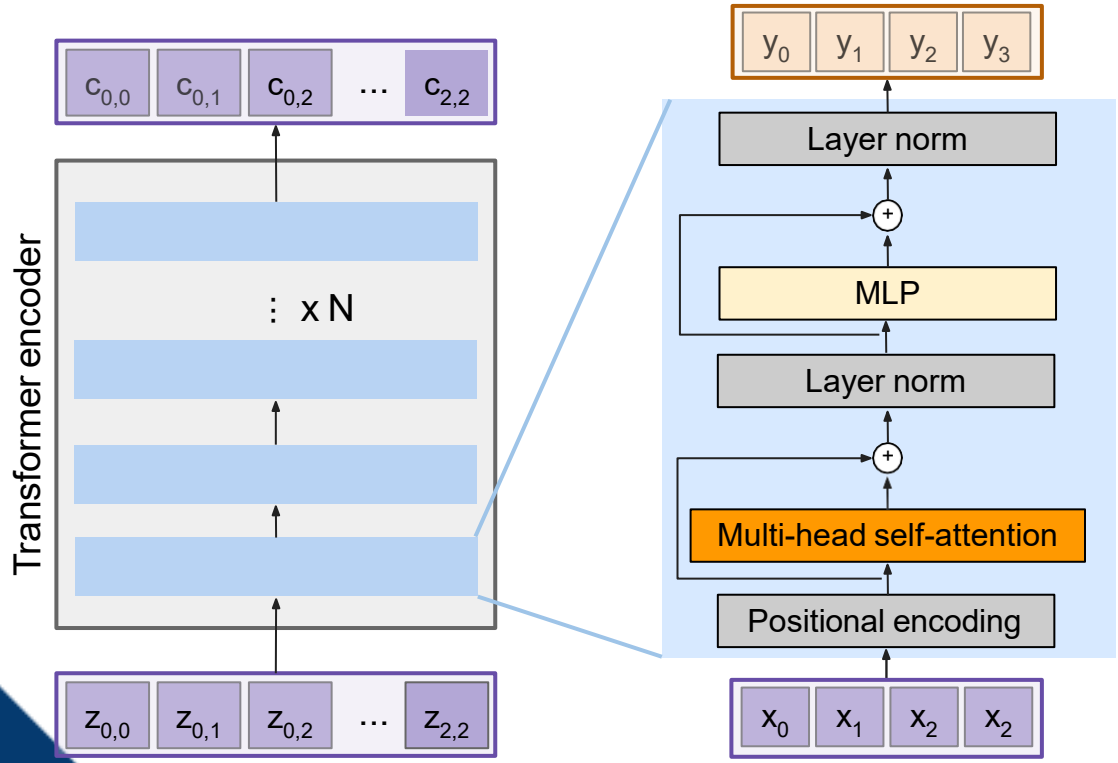
Residual connection

Attention attends over all the vectors

Add positional encoding

Vaswani et al, “Attention is all you need”, NeurIPS 2017

# The Transformer encoder block




**Transformer Encoder Block:**

**Inputs**: Set of vectors **x**
**Outputs**: Set of vectors **y**

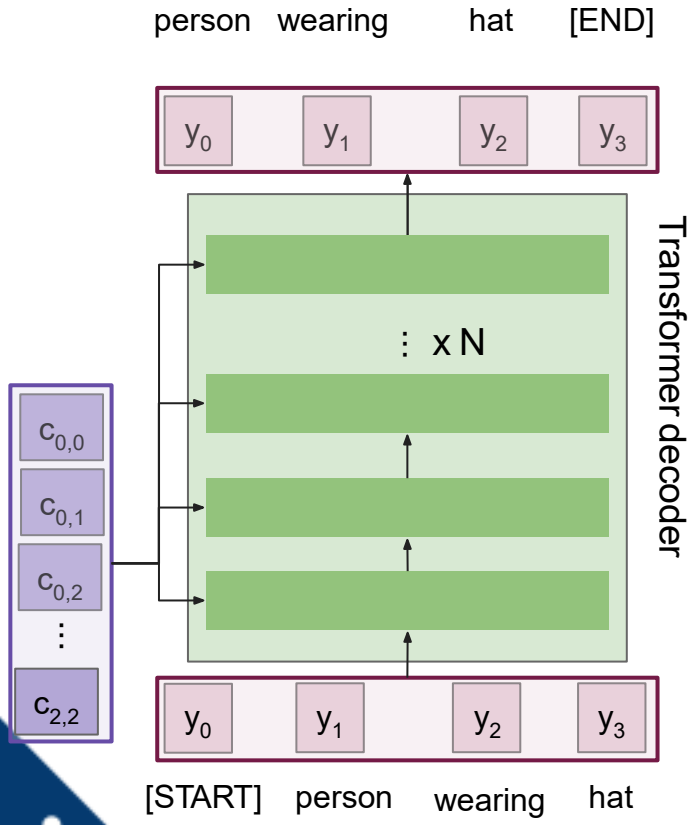
Self-attention is the only interaction between vectors.

Layer norm and MLP operate independently per vector.

Highly scalable, highly parallelizable, but high memory usage.

Vaswani et al, “Attention is all you need”, NeurIPS 2017

# The Transformer Decoder block

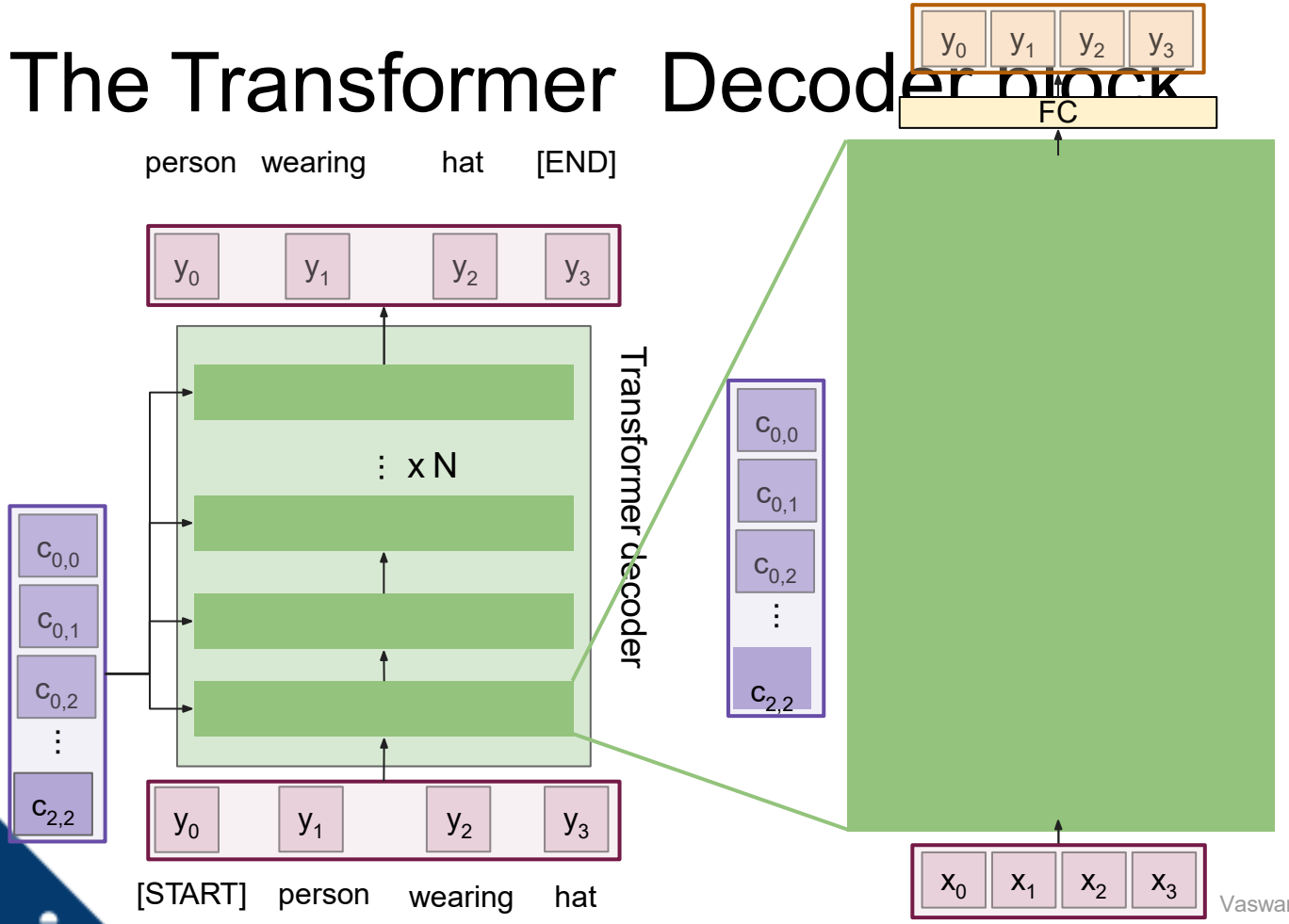



Made up of N decoder blocks.

In vaswani et al. N = 6, $D_q$= 512

Vaswani et al, “Attention is all you need”, NeurIPS 2017

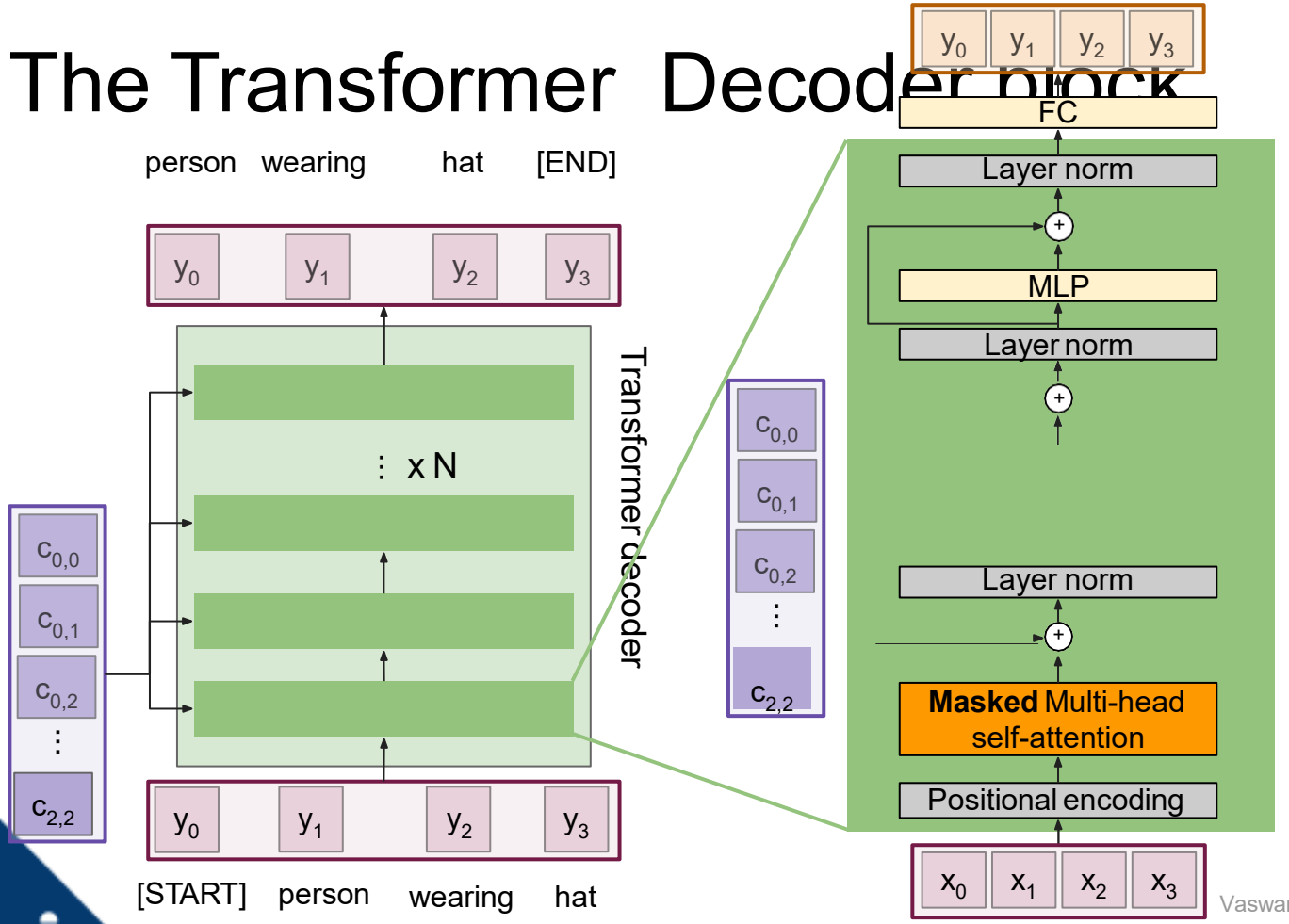
# The Transformer Decoder block




Let's dive into the transformer decoder block

Vaswani et al, "Attention is all you need", NeurIPS 2017

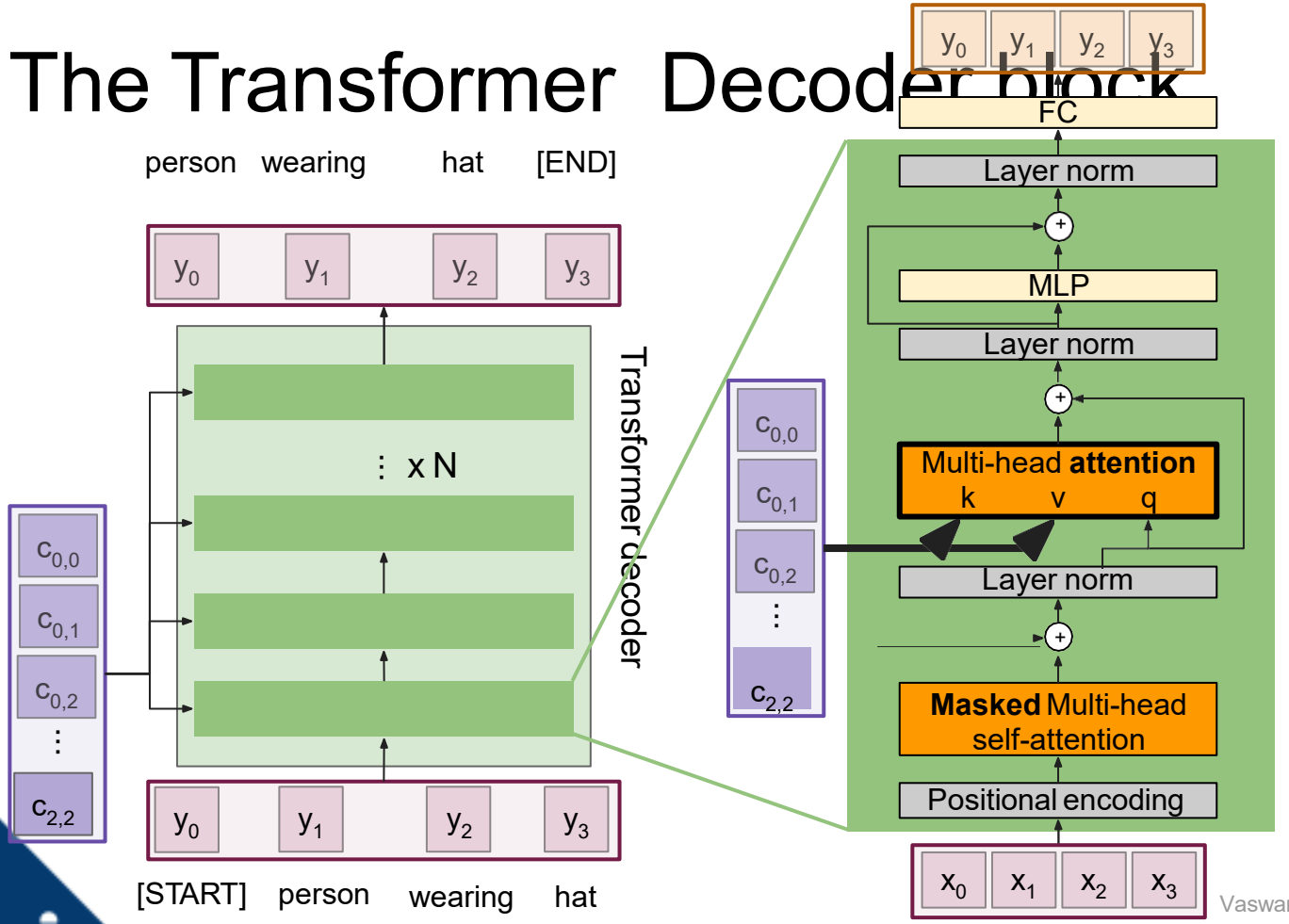
# The Transformer Decoder block




Most of the network is the same the transformer encoder.

Vaswani et al, "Attention is all you need", NeurIPS 2017

# The Transformer Decoder block

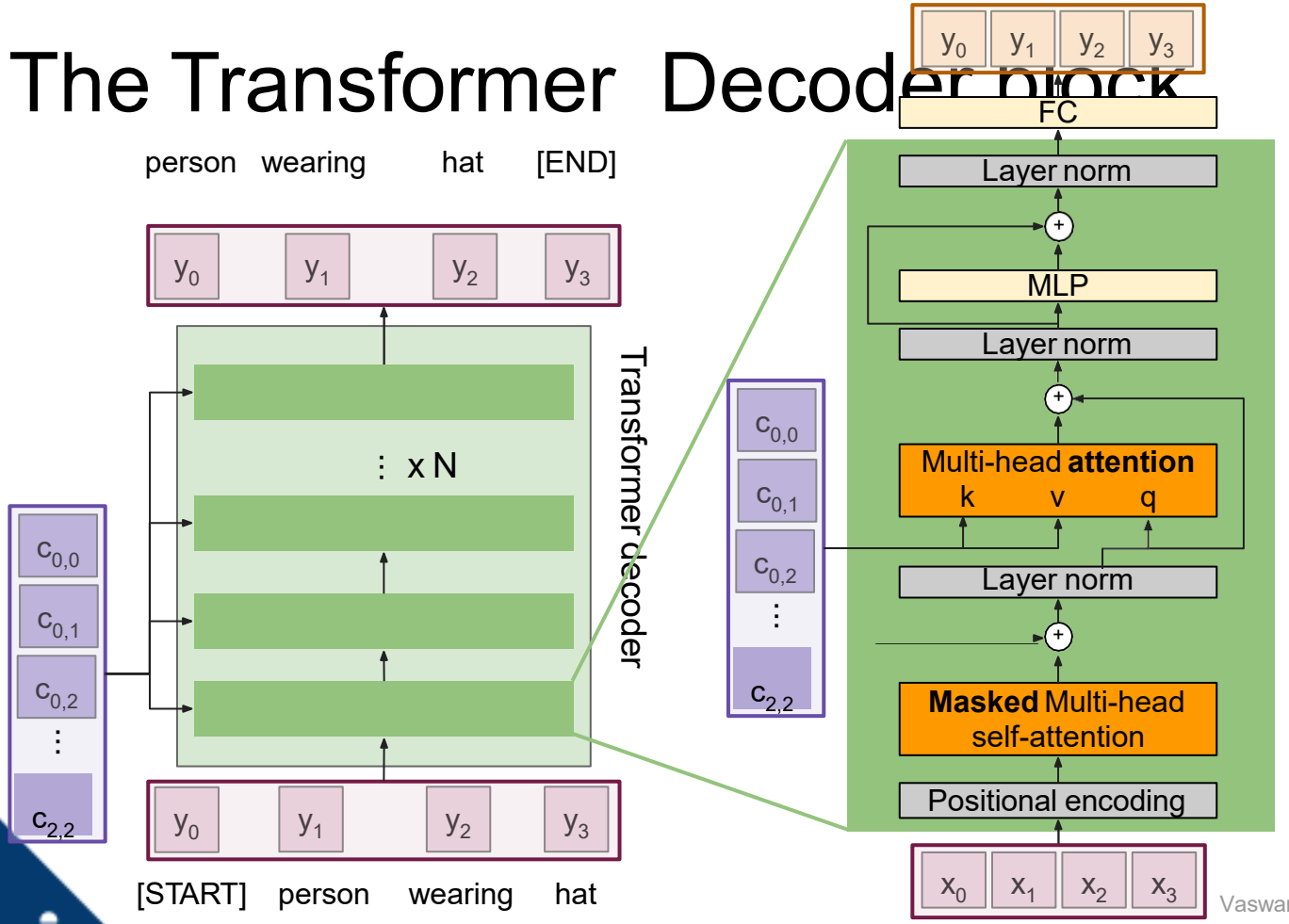



Multi-head attention block attends over the transformer encoder outputs.

For image captions, this is how we inject image features into the decoder.

Vaswani et al, "Attention is all you need", NeurIPS 2017

# The Transformer Decoder block




**Transformer Decoder Block:**

**Inputs**: Set of vectors **x** and Set of context vectors **c.**
**Outputs**: Set of vectors **y.**

Masked Self-attention only interacts with past inputs.

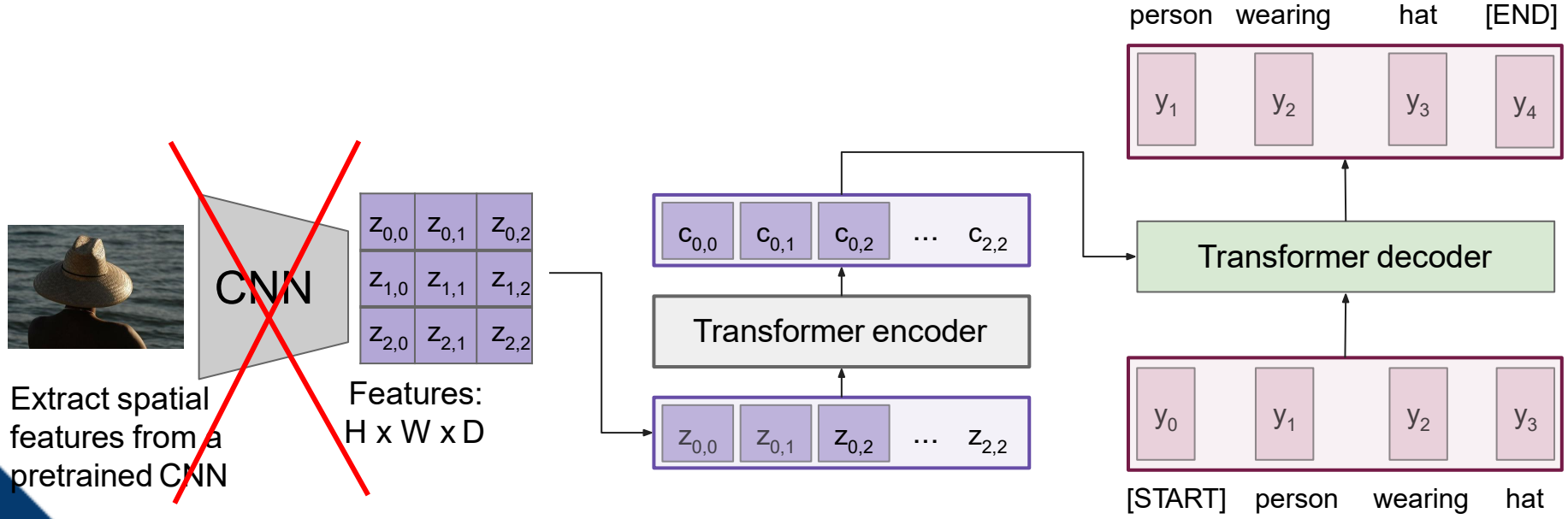
Multi-head attention block is NOT self-attention. It attends over encoder outputs.

Highly scalable, highly parallelizable, but high memory usage.

Vaswani et al, "Attention is all you need", NeurIPS 2017

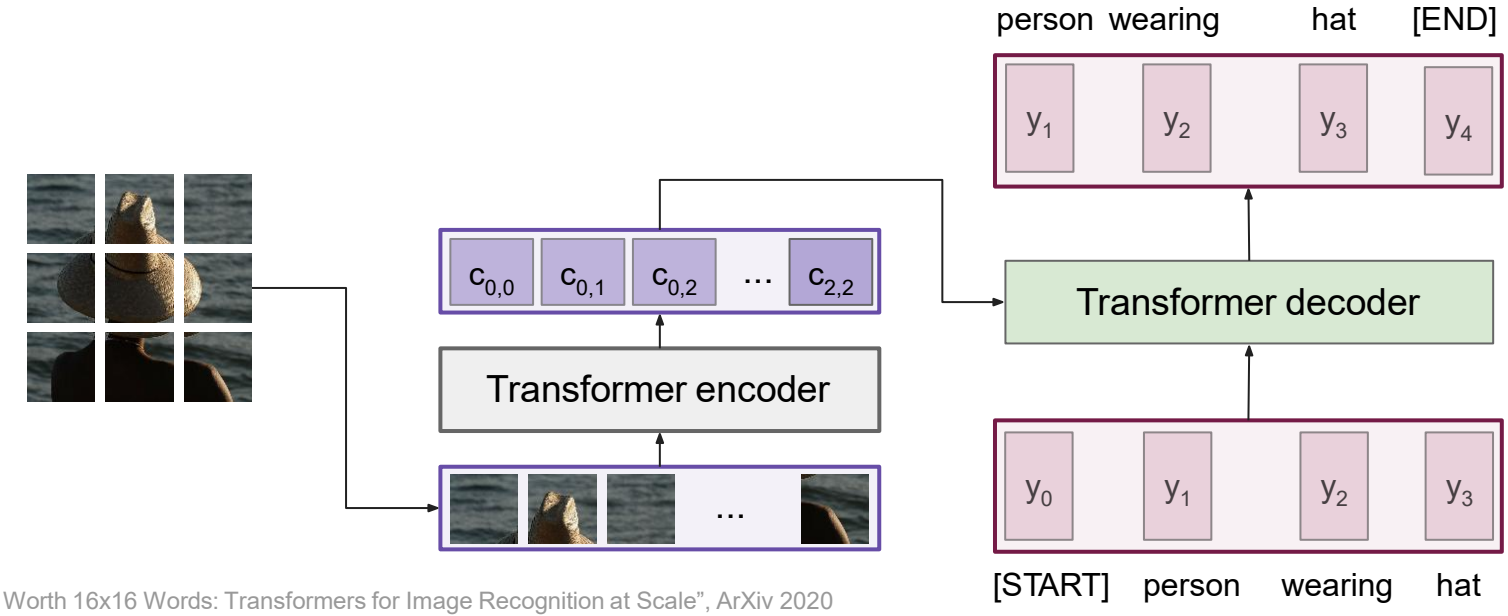
# Image Captioning using transformers

- **Perhaps we don't need convolutions at all?**



# Image Captioning using ONLY transformers

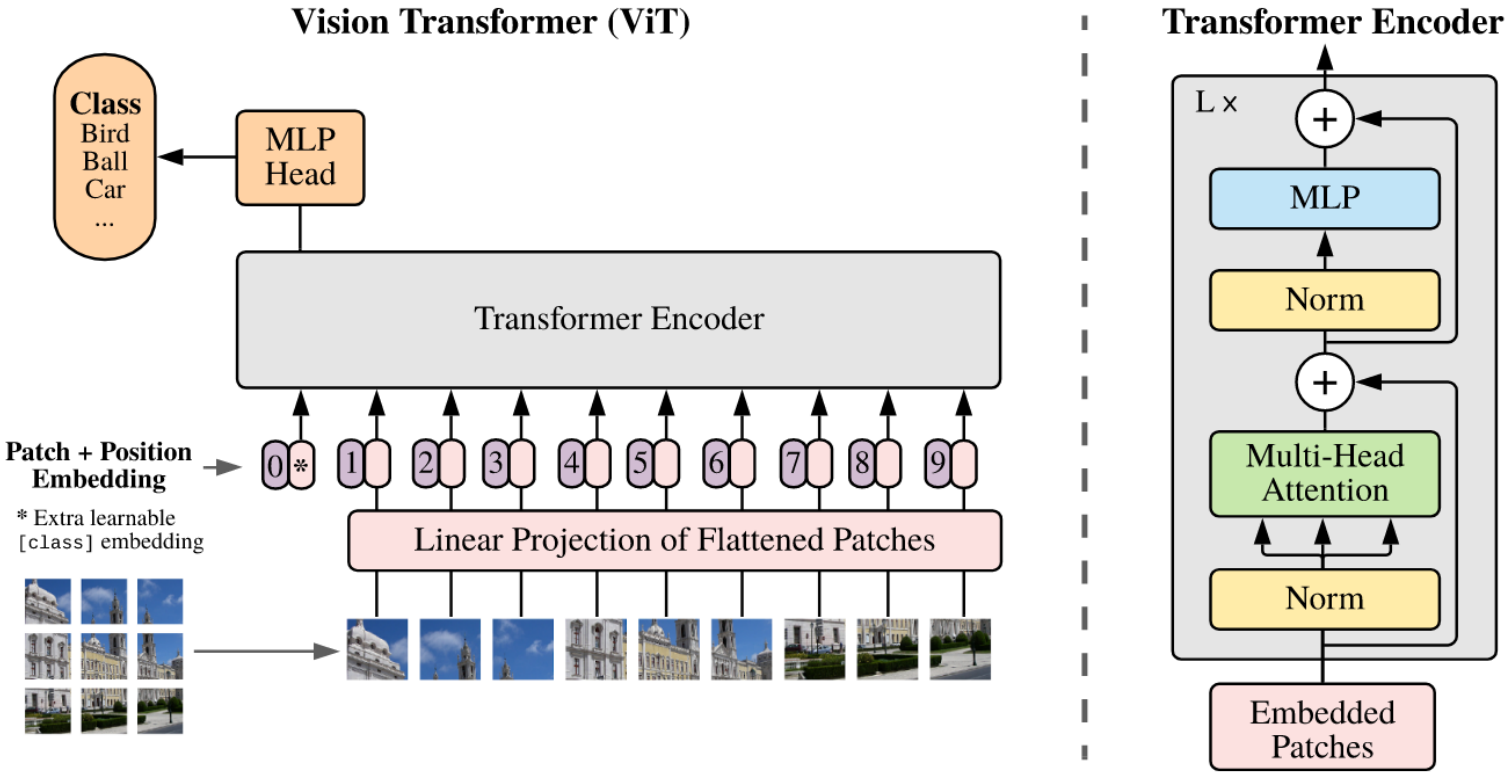
- **Transformers from pixels to language**




Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ArXiv 2020

Colab link to an implementation of vision transformers

# Visual Transformer



# Summary

- Adding **attention** to RNNs allows them to "attend" to different parts of the input at every time step
- The **general attention layer** is a new type of layer that can be used to design new neural network architectures
- **Transformers** are a type of layer that uses **self-attention** and layer norm.
  - It is highly **scalable** and highly **parallelizable**
  - **Faster** training, **larger** models, **better** performance across vision and language tasks
  - They are quickly replacing RNNs, LSTMs, and may even replace convolutions.





**Anonymous**
**Class Feedback**

# Q&A